

Master Thesis

SHAPE OPTIMISATION USING DEEP REINFORCEMENT LEARNING METHODS

Submitted in partial fulfilment of the requirements for the award of

Master of Science Degree in

Automation and Robotics

of the Technische Universität Dortmund

PHILIP VARGHESE MODAYIL

Matriculation Number: 229225

External Supervisor: **Dr.-Ing. Werner John**(Associate Lecturer)
Supervisor: **Prof. Dr.-Ing. Jürgen Götze**
Co-Supervisor: **M.Sc. Emre Ecik**
Submitted On: **29 June 2024**

Eidesstattliche Versicherung

(Affidavit)

MODAYIL, PHILIP VARGHESE

229225

Name, Vorname
(surname, first name)

Matrikelnummer
(student ID number)

☐ Bachelorarbeit
(Bachelor's thesis)

☒ Masterarbeit
(Master's thesis)

Titel
(Title)

SHAPE OPTIMISATION USING DEEP REINFORCEMENT LEARNING METHODS

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

DORTMUND, 29-06-2024

Ort, Datum
(place, date)

Unterschrift
(signature)



Belehrung:

Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin“) zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:

Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

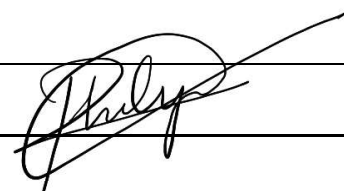
As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

DORTMUND, 29-06-2024

Ort, Datum
(place, date)

Unterschrift
(signature)



***Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.**

Abstract

This thesis focuses on optimising the shape of a potential curve within a microstrip arrangement environment, following Thomson's theorem of minimum energy, where the optimal potential curve will have the least energy within the environment. A near-global solution was achieved by navigating through the microstrip arrangement environment using reinforcement learning (RL). Through various experiments, RL has proved to be a transformative method that enabled exploration within an unknown environment. In addition to RL, a genetic algorithm (GA) was employed to optimise the RL-predicted solution further and achieve the global solution.

Contents

List of Abbreviations	i
List of Figures	v
List of Tables	viii
List of Source Codes	viii
List of Algorithms	ix
1 Introduction	1
1.1 Background	1
1.1.1 Machine Learning Methods	1
1.1.2 Basic Principles: Deep Reinforcement Learning	2
1.1.3 Basic Principles: Mixed Boundary Value Problem	3
1.1.4 Shape optimisation and Deep Reinforcement Learning	4
1.2 Motivation	7
2 State Of The Art	9
2.1 Deep Reinforcement Learning	9
2.1.1 Proximal Policy optimisation (PPO)	11
2.1.2 Advantage Actor Critic (A2C)	15
2.1.3 Deep Deterministic Policy Gradient (DDPG)	18
2.1.4 Twin Delayed Deep Deterministic Policy Gradient (TD3)	20
2.1.5 Soft Actor-Critic (SAC)	22
2.2 Mixed Boundary Value Problem	25
2.2.1 Problem Definition	25
2.2.2 Mathematical Formulations for Solving the Mixed Boundary Value Problem	26

2.2.3	Solution Approach	27
2.2.4	Essential Equations	28
3	Realization Of Software Modules	31
3.1	Environment	31
3.2	Agent	32
3.3	Software optimisations	33
4	Training Metrics and Tensorboard Logging	35
4.1	Rewards	35
4.2	Value Loss/Critic Loss	36
4.3	Policy Gradient Loss/Actor Loss	36
5	Concept Development	38
5.1	Method 1: Scatter Points	38
5.2	Method 2: Decaying Exponential Function	42
5.3	Method 3: Bézier Curves, Threshold Reward	46
5.3.1	Bézier Curve	49
5.3.2	Control Points	50
5.3.3	Reward Scheme	50
5.3.4	Training Parameters	51
5.3.5	Learning Policy and Training	54
5.4	Method 4: Bézier Curves, Scaled Reward	58
5.4.1	Reward	58
5.4.2	Learning Policy and Training	62
5.4.3	Training Results	64
6	Test Cases	81
6.1	Test Case 2.2	82
6.2	Test Case 2.3	82
6.3	Test Case 2.4	88
7	Reinforcement Learning + Genetic Algorithm	91
7.1	Genetic Algorithm	91
7.2	Limitations Of RL	93
7.3	GA optimisation of RL Results	94
7.4	Software Implementation for GA	94
7.5	GA optimisation Results	95

7.6	Observations	100
7.7	Test Case 2 Series	101
7.7.1	Test Case 2.2	101
7.7.2	Test Case 2.3	103
7.7.3	Test Case 2.4	105
7.8	Impact of RL	107
8	Conclusion and Future Scope	110
8.1	Future Scope	110
8.2	Conclusion	111
	BIBLIOGRAPHY	112
A	Derivations	118
A.1	Calculation of the Transmission Line Parameters Z_D and ϵ_{eff}	120
A.2	Equations for Calculating the Potential Distribution using First Order Spline Functions	121
A.3	Calculation of Surface Charge Density using the Potential Coefficients	124
A.4	Calculation of the Energy of the Electric Fields for the areas A_1 and A_2 using the Potential Coefficients	126
B	Software Documentation	130
B.1	Microstrip Arrangement	130
B.2	Method 3: Microstrip Arrangement RL Environment	141
B.3	Method 4: Microstrip Arrangement RL Environment	148

List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
GA	Genetic Algorithm
PPO	Proximal Policy Optimisation
A2C	Advantage Actor Critic
DDPG	Deep Deterministic Policy Gradient
TD3	Twin Delayed Deep Deterministic Policy Gradient
SAC	Soft Actor Critic
s_t	State at time t
$\hat{A}_t$	Action at time t
$\hat{Adv}_t$	Advantage at time t
θ	Parameters
$\hat{R}_t$	Reward
$\pi(\theta)$	Policy
$V(s_t)$	Value function at state s_t
D_k	Replay Buffer
PCB	Printed Circuit Board
d	Half-width of the microstrip
a	Half-width of the microstrip arrangement
t	Thickness of microstrip
c	Height of substrate medium

b	Height of the microstrip arrangement
A_1	Area Medium 1
A_2	Area Medium 2
$'$	Plane field problem
W'	Electrical field energy (plane field problem)
w'	Electrical field energy density
ϵ	Permittivity of substance
ϵ_r	Relative permittivity of substance
$\vec{E}$	Electric Field
σ	Surface Charge Density
SB3	stable-baselines3
MLP	Multi-layer Perceptron
POSIX	Portable Operating System Interface

List of Figures

1.1	Taxonomy of Machine Learning Methods [1]	2
1.2	Deep Reinforcement Learning[2]	3
1.3	Microstrip Arrangement	5
1.4	Microstrip Arrangement Potential Distribution	5
2.1	Taxonomy of Deep Reinforcement Learning Algorithms[3]	10
2.2	Left: On-Policy Learning, Right: Off-Policy Learning[4]	11
2.3	Microstrip Arrangement under Study	26
2.4	Approximation of the Potential Distribution by Spline Functions of the First Order	28
5.1	Episode Reward Mean for Method 1 [5]	41
5.2	Value Loss(Left), Policy Gradient Loss(Right) for Method 1 [5]	41
5.3	Degenrate Deep Reinforcement Learning framework	42
5.4	Method 2 G-Points	46
5.5	Episode Reward Mean for Method 2 [5]	47
5.6	Value Loss(Left), Policy Gradient Loss(Right) for Method 2 [5]	47
5.7	Quadratic Bézier: Three control points[6]	49
5.8	Cubic Bézier: Four control points[6]	50
5.9	Threshold Line For Rewards	53
5.10	Experiment Results from M.Sc. Emre Ecik [7]	54
5.11	Method 3 Case D G-Points	56
5.12	Episode Reward Mean for Method 3 Case D [5]	56
5.13	Critic Loss(Left), Actor Loss(Right) for Method 3 Case D [5]	57
5.14	Case D Baseline Curve	61
5.15	Case L Baseline Curve	62
5.16	Episode Reward Mean for Method 4 Case D [5]	65
5.17	Critic Loss(Left), Actor Loss(Right) for Method 4 Case D [5]	66
5.18	Entropy Coefficient for Method 4 Case D [5]	66

5.19	G-Points at 20K Timesteps for Case D with SAC Agent	67
5.20	G-Points at 50K Timesteps for Case D with SAC Agent	67
5.21	Surface Charge Density at 20K Timesteps for Case D with SAC Agent	68
5.22	Surface Charge Density at 50K Timesteps for Case D with SAC Agent	68
5.23	Episode Reward Mean for Method 4 Case L [5]	69
5.24	Critic Loss(Left), Actor Loss(Right) for Method 4 Case L [5]	70
5.25	Entropy Coefficient for Method 4 Case l [5]	70
5.26	G-Points at 45K Timesteps for Case L with SAC Agent	71
5.27	G-Points at 50K Timesteps for Case L with SAC Agent	72
5.28	Surface Charge Density at 45K Timesteps for Case L with SAC Agent	72
5.29	Surface Charge Density at 50K Timesteps for Case L with SAC Agent	73
5.30	Multiple Agents Episode Reward Mean for Method 4 Case D [5] . . .	78
5.31	Multiple Agents Episode Reward Mean for Method 4 Case L [5] . . .	78
5.32	Test Case 2.1 Case L Prediction with Case D Model	79
5.33	Test Case 2.1 Case D Prediction with Case L Model	80
6.1	Test Case 2.2 G-Points Case D with SAC Agent	83
6.2	Test Case 2.2 G-Points Case L with SAC Agent	83
6.3	Test Case 2.3 G-Points Case D with SAC Agent	86
6.4	Test Case 2.3 G-Points Case L with SAC Agent	86
6.5	Test Case 2.4 G-Points Case D with SAC Agent	89
6.6	Test Case 2.4 G-Points Case L with SAC Agent	89
7.1	Genetic Algorithm Flow Chart [8]	92
7.2	Case D Training Environment: Initial Population for GA	97
7.3	Case L Training Environment: Initial Population for GA	98
7.4	Case D Training Environment: GA Convergence	98
7.5	Case L Training Environment: GA Convergence	99
7.6	Case D Training Environment: GA Result	99
7.7	Case L Training Environment: GA Result	100
7.8	Case D Test Case 2.2: GA Result	102
7.9	Case L Test Case 2.2: GA Result	102
7.10	Case D Test Case 2.3: GA Result	104
7.11	Case L Test Case 2.3: GA Result	104
7.12	Case D Test Case 2.4: GA Result	106
7.13	Case L Test Case 2.4: GA Result	106
7.14	Test Case 2.1 Case D: GA Optimisation Starting Curves	108
7.15	Test Case 2.1 Case D: GA optimisation Results	109

7.16	Method 3 and Method 4 GA Convergence	109
8.1	Comparison of the Execution Times for Quicksort Algorithm [9] . . .	111
8.2	GUI Application for Potential Curve Shape Optimisation	112
A.1	Reduced Microstrip Arrangement under Study (use of symmetry characteristics of the arrangement)	118
A.2	Approximation of the Potential Distribution by Spline Functions of the First Order (symmetrical problem)	122

List of Tables

5.1	Action for Method 1	38
5.2	Observation Space for Method 1	39
5.3	Observation Space	43
5.4	Action for Method 2	44
5.5	Test Case 1: Method 2 Environment Parameters	45
5.6	Training Parameters	45
5.7	Prediction Energy Values Method 2	46
5.8	Test Case 2.1 Case D Environment Parameters	48
5.9	Test Case 2.1 Case L Environment Parameters	48
5.10	Action for Method 3	50
5.11	Bézier Control Points	51
5.12	Training Parameters	53
5.13	Predicted Action for Method 3 and Resulting Energy	55
5.14	Prediction Energy Values for Case D and Case L with Method 3 . . .	55
5.15	Capacitance for Case D and Case L with Method 3	55
5.16	Characteristic Impedance for Case D and Case L with Method 3 . . .	57
5.17	Phase Velocity and Effective Dielectric Constant with Method 3 . . .	57
5.18	Test Case 2.1 Case D Environment Parameters	58
5.19	Test Case 2.1 Case L Environment Parameters	59
5.20	Energy Values	60
5.21	Control Points: Baseline Potential Curve	60
5.22	Action for Method 4	63
5.23	Action Values for Case D	65
5.24	Action Values for Case L	71
5.25	Prediction Energy Values for Case D and Case L with Method 4 . . .	75
5.26	Capacitance for Case D and Case L with Method 4	75
5.27	Characteristic Impedance for Case D and Case L with Method 4 . . .	75
5.28	Phase Velocity and Effective Dielectric Constant with Method 4 . . .	75

5.29	Attribute Relative Error for Case D and Case L with Method 4 . . .	76
5.30	Attribute Relative Error for Case D and Case L with Method 3 . . .	76
5.31	Energy after Prediction	80
6.1	Test Parameters	81
6.2	Test Case 2.2 Case D Environment Parameters	82
6.3	Test Case 2.2 Case L Environment Parameters	82
6.4	Test Case 2.2: Prediction Energy Values for Case D and Case L . . .	84
6.5	Test Case 2.2: Capacitance for Case D and Case L	84
6.6	Test Case 2.2: Characteristic Impedance for Case D and Case L . . .	84
6.7	Test Case 2.2: Phase Velocity and Effective Dielectric Constant . . .	84
6.8	Test Case 2.2: Attribute Relative Error for Case D and Case L . . .	84
6.9	Test Case 2.3 Case D Environment Parameters	85
6.10	Test Case 2.3 Case L Environment Parameters	85
6.11	Test Case 2.3: Prediction Energy Values for Case D and Case L . . .	85
6.12	Test Case 2.3: Capacitance for Case D and Case L	85
6.13	Test Case 2.3: Characteristic Impedance for Case D and Case L . . .	87
6.14	Test Case 2.3: Phase Velocity and Effective Dielectric Constant . . .	87
6.15	Test Case 2.3: Attribute Relative Error for Case D and Case L . . .	87
6.16	Test Case 2.4 Case D Environment Parameters	88
6.17	Test Case 2.4 Case L Environment Parameters	88
6.18	Test Case 2.4: Prediction Energy Values for Case D and Case L . . .	90
6.19	Test Case 2.4: Capacitance for Case D and Case L	90
6.20	Test Case 2.4: Characteristic Impedance for Case D and Case L . . .	90
6.21	Test Case 2.4: Phase Velocity and Effective Dielectric Constant . . .	90
6.22	Test Case 2.4: Attribute Relative Error for Case D and Case L . . .	90
7.1	Genetic Algorithm Parameters	97
7.2	Genetic Algorithm Evaluation Time	97
7.3	Energy Values for Case D and Case L after GA optimisation	97
7.4	Run Time for RL Prediction and GA optimisation	100
7.5	Test Case 2.2 Case D Environment Parameters	101
7.6	Test Case 2.2 Case L Environment Parameters	101
7.7	Test Case 2.3 Case D Environment Parameters	103
7.8	Test Case 2.3 Case L Environment Parameters	103
7.9	Test Case 2.4 Case D Environment Parameters	105
7.10	Test Case 2.4 Case L Environment Parameters	105
7.11	Test Case 2.1 Case D: GA Optimisation Results	108

List of Source Codes

1	Gymnasium Example [10]	32
2	SAC Example SB3 [11]	33
3	Multiprocessing Example [12]	34
4	Gentic Algorithm Implementation Example [13]	96
B.1	Microstrip Arrangement	130
B.2	Method 3 Microstrip Arrangement RL Environment	141
B.3	Method 4 Microstrip Arrangement RL Environment	148

List of Algorithms

1	Proximal Policy optimisation (PPO) [14]	14
2	Advantage Actor-Critic (A2C) [15]	17
3	Deep Deterministic Policy Gradient (DDPG) [16]	19
4	Twin Delayed Deep Deterministic Policy Gradient (TD3) [17]	21
5	Soft Actor-Critic (SAC) [18]	24
6	Reward Scheme for Method 1	40
7	Reward Scheme for Method 2	44
8	Cubic Bézier curve	51
9	Reward Scheme Method 3	52
10	Reward Scheme	63
11	DRL Learning with SAC Policy	64

Chapter 1

Introduction

The world is filled with complex phenomena, necessitating clever methods to address these challenges. These real-life problems are non-linear in nature, and engineers have long relied on numerical optimisation techniques to tackle them and derive effective solutions.

One such problem that requires non-linear techniques is shape optimisation. This thesis considers the shape optimisation problem of a potential curve in a microstrip arrangement environment and explores the machine learning possibilities for the same.

1.1 Background

Let us first discuss the various machine-learning approaches and the shape optimisation problem we would like to solve.

1.1.1 Machine Learning Methods

Machine learning methods are broadly classified into Supervised, Unsupervised, and Reinforcement Learning.

Supervised learning uses labelled data for training where each input has a matching output. The model essentially aims to map a function between the inputs and outputs. Conversely, unsupervised learning doesn't require labelled data to function. In this approach, the model searches the dataset for hidden structures or patterns.

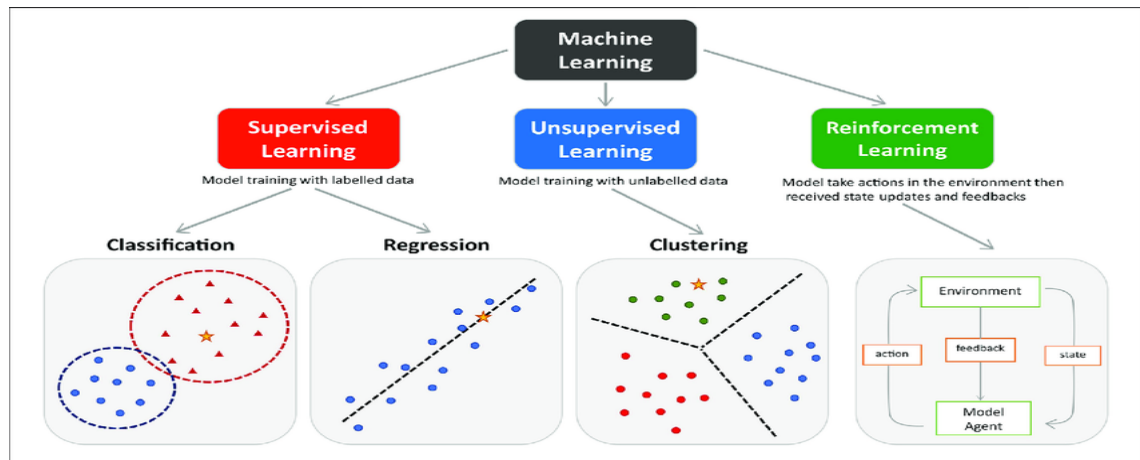


Figure 1.1: Taxonomy of Machine Learning Methods [1]

Unsupervised learning techniques concentrate on clustering similar data points together or lowering the dimensionality of the input, in contrast to supervised learning, where the model seeks to predict specific outputs.

With reinforcement learning, we enter an entirely new realm of machine learning. It focuses on teaching agents to make successive decisions through interaction with their surroundings. In contrast to supervised and unsupervised learning, which mostly concentrate on static datasets, reinforcement learning gains knowledge from the dynamic interactions with the environment across time.

Deep learning is a subset of machine learning that includes a hierarchical architecture of interconnected layers. Each layer consists of an individual unit called neurons. Deep learning is inspired by the structure and function of the brain's neural networks. [19]

1.1.2 Basic Principles: Deep Reinforcement Learning

Deep reinforcement learning combines the ideas of reinforcement learning and deep learning. Like humans, our agents discover how to implement effective tactics that yield the highest long-term benefits (Rewards). Our agents don't need domain heuristics or hand-engineered features; they create and acquire knowledge from unprocessed inputs like vision (direct interaction with the environment)[2].

Neural networks with deep learning are used to map the current state of the environment at timestep t , s_t to the action that needs to be taken $\hat{A}_t$. As shown in Figure 1.2, the Agent can output the action that needs to be taken for a given state in the environment, which would then be penalized or rewarded.

In general, with the help of conventional deep learning techniques, we could derive the optimal policy π^* or the optimal value function V^* , depending on the DRL algorithm we are using.

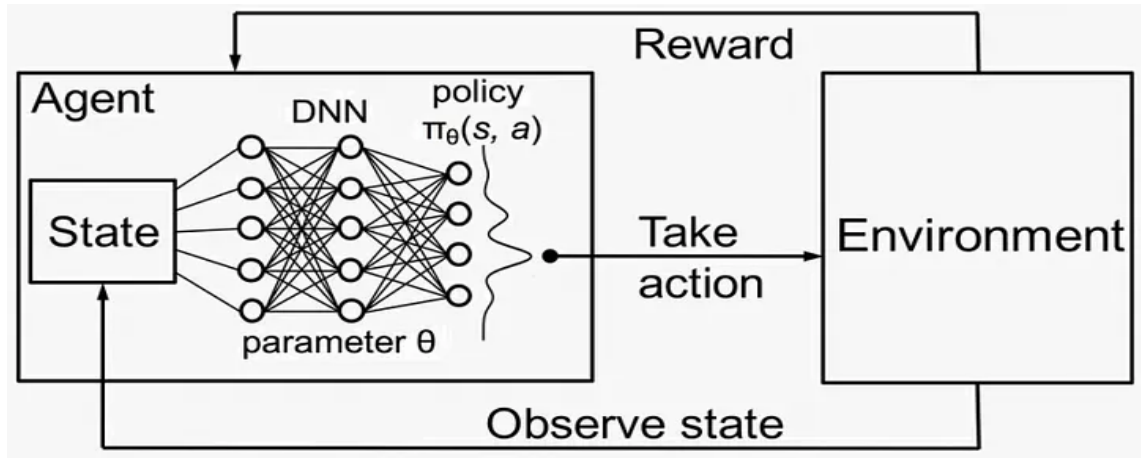


Figure 1.2: Deep Reinforcement Learning[2]

1.1.3 Basic Principles: Mixed Boundary Value Problem

A mixed boundary value problem exists if the potential is specified on some of the surfaces and its normal derivative is specified on the remaining surfaces. This is the case with the arrangement shown in Figure 1.3.

The arrangement shown in Figure 1.3 represents a symmetrical single microstrip transmission line. The microstrip transmission line consists of a very thin strip of conductive material on a dielectric substrate that propagates an electromagnetic wave in the axial z -direction, the existence of which requires axial field strength components for physical reasons.

In the frequency range up to 10 GHz, the axial field strength components are small compared to the transverse ones, so a transverse electromagnetic (TEM) wave (Q-

TEM case) can adequately describe the field. For many applications (SI simulations, for example), the parameters of single microstrip lines can accordingly be calculated with sufficient accuracy. [20] [21] [22] [23] [24] [25] [26]

For designing a Printed Circuit Board (PCB), it is essential to know the characteristic impedance and effective dielectric constant ϵ_{eff} for the microstrip arrangement on the board. This value is calculated based on the potential distribution across the microstrip (Figure 1.4), which follows Thomoson's minimum energy theorem. The theorem states that; "given a certain number of conductors, each one with a given charge, the charges distribute themselves on the conductor surfaces to minimize the electrostatic energy"[27]. This simply means that in a given microstrip arrangement, the potential distribution will take the shape of the curve with the least energy. Hence, the main task here is to Figure out the potential distribution(its optimal shape) across the microstrip.

For our problem, let us take the microstrip arrangement as shown in Figure 1.3, where we will assume the thickness of the strip to be zero. In this problem, ϵ_{r1} is 1, which corresponds to air. Hence, the arrangement represents the conductive microstrip placed on top of a printed circuit board (PCB).

This work presents an AI-based solution method for calculating the characteristic parameters of a simple (single) microstrip line (characteristic impedance; effective dielectric constant). The method combines an RL algorithm with a genetic algorithm (hybrid algorithm) to determine the desired potential function $V(x, y = c)$ (by figuring out the optimal shape) on the boundary layer.

It is important to note the symmetrical aspect of the arrangement and the potential distribution. This is key in reducing the calculations, as only one-half is required for it. We shall take advantage of this aspect and significantly reduce the complexity of the calculation. Moving forward, we will consider only one-half of the arrangement to derive the required equations.

1.1.4 Shape optimisation and Deep Reinforcement Learning

Shape optimisation plays a fundamental role across a spectrum of engineering disciplines, encompassing fields such as magnetostatics, acoustics, image restoration and segmentation, identification of composite materials, and nano-optics, among others.

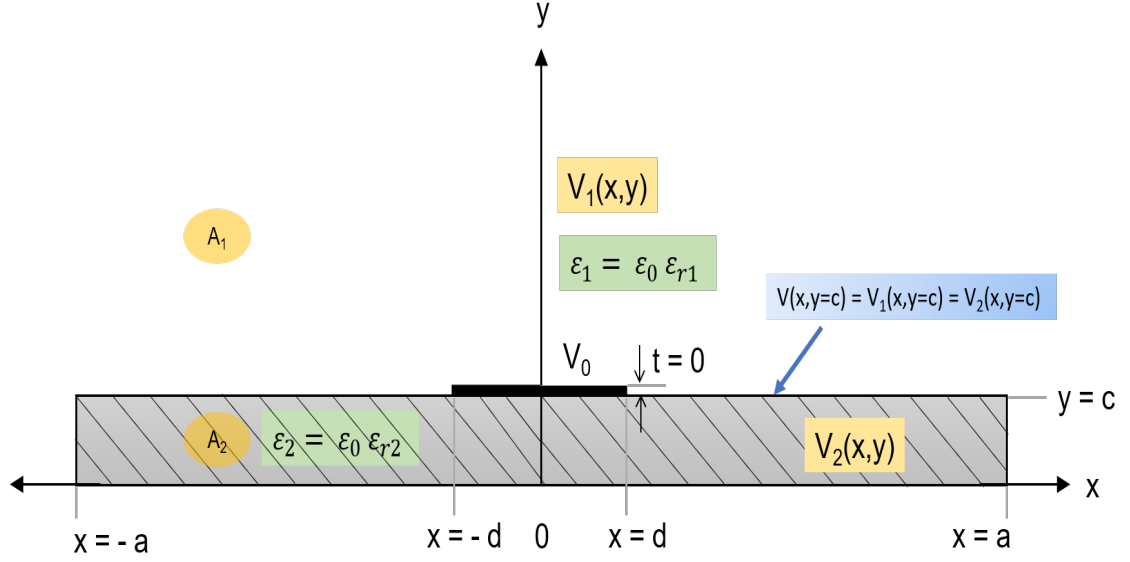


Figure 1.3: Microstrip Arrangement

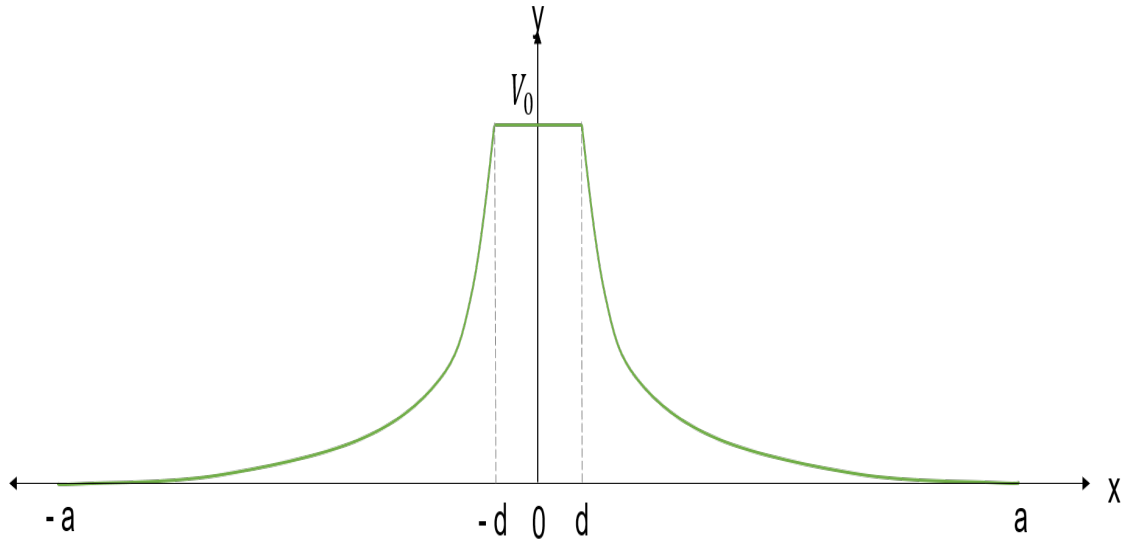


Figure 1.4: Microstrip Arrangement Potential Distribution

[28]

Historically, shape optimisation has been approached through two primary methods: gradient-based and gradient-free techniques. Gradient-based methods hinge on evaluating the objective function's gradient concerning the design parameters. They have demonstrated effectiveness in vast optimisation spaces, especially when employing the adjoint method to compute the gradient, which offers comparable computational costs to solve the governing equations. This stands in contrast to more resource-intensive alternatives like variance-based and regression-based methods, which may necessitate repeated equation solving, potentially up to a hundred times. However, gradient-based algorithms are prone to getting stuck in local optima, particularly when addressing stiff nonlinear problems and relying heavily on the initial guess.[28]

On the other hand, gradient-free methods offer greater robustness against local optima but can present implementation challenges. While genetic algorithms, particle swarm optimisation, and metropolis algorithms boast strong global optimisation capabilities, their performance may hinge significantly on heuristically chosen meta-parameters. Additionally, their computational cost tends to be higher, sometimes surpassing available budgets and limiting the number of design parameters.[28]

It's worth noting that both method classes can leverage inexpensive surrogate models to approximate costly objective and constraint functions without relying exclusively on numerical simulations. Various techniques exist for constructing surrogate models, including polynomial response surfaces, radial basis functions, kriging, and supervised artificial neural networks. Geometric parameterization is crucial in these models, impacting achievable geometries and the optimisation process's manageability.[28]

Deep reinforcement learning involves training an agent to make sequential decisions in an environment to maximize the cumulative reward over time. For a shape optimisation problem, the environment could be a simulation environment that evaluates the performance of different shapes based on some objective function. The agent will take actions corresponding to the shape's design parameters; hence, figuring out the highest reward action will give us the best-performing shape.

One advantage of deep reinforcement learning is its ability to handle complex optimisation problems. This is because DRL does not require explicit knowledge of the

gradient of the objective function. On the other hand, numerical optimisation relies solely on that. This ability of DRL is particularly beneficial when the objective function is difficult to evaluate analytically.

Furthermore, DRL algorithms are based on their interactions with the environment and can be tuned to explore the environment efficiently. This would mitigate the problem of getting stuck at local optima that other methods may encounter. This ability of DRL is known as the exploration-exploitation trade-off.

1.2 Motivation

Shape optimisation is a problem for which certain physical conditions define the optimal state/shape. In the case of a Mixed Boundary Value problem, we depend on Thomson's theorem, which states that; "given a certain number of conductors, each one with a given charge, the charges distribute themselves on the conductor surfaces to minimize the electrostatic energy". Hence, the optimal shape of the potential distribution will be the one with the least energy [27].

Here, we do not know the optimal state like that in a labelled dataset. This limitation of the problem forces us to deviate from classical deep learning techniques. Deep reinforcement learning can explore and navigate through a given environment and figure out the best possible state without explicit knowledge.

In [29], researchers have implemented the shape optimisation of an aeroplane wing under aerodynamic conditions with deep reinforcement learning. They have shown that, provided an adequate reward function based on the lift-to-drag ratio, their agent generated wing-like optimal shapes without prior knowledge of aerodynamic concepts.

In [28], researchers have shown a single-step DRL approach that learns the optimal shape for fluid dynamics problems. The numerical reward fed to the learning network is calculated with a finite elements CFD environment, solving stabilized weak forms of the governing equations (Navier–Stokes, otherwise uRANS with negative Spalart–Allmaras as turbulence model) with a variational multiscale approach, immersed volume method, and anisotropic mesh adaptation. The agent is concerned only with maximizing the numerical reward fed to it and is completely unaware of the complex physics of the environment.

Inferring from the above examples, we can conclude that DRL is a viable approach for shape optimisation problems where the environment is complex for gradient-based optimisation approaches.

Chapter 2

State Of The Art

2.1 Deep Reinforcement Learning

Here, we shall examine the various state-of-the-art DRL algorithms and how DRL is classified based on its characteristics. DRL can be categorized based on two main dimensions: model-based vs. model-free and on-policy vs. off-policy.

One of the critical branching points in a reinforcement learning (RL) algorithm is whether the agent possesses or learns a model of the environment. When referring to a model of the environment, we're indicating a function capable of forecasting state transitions and rewards.

Having a model in reinforcement learning offers a significant advantage by enabling the agent to engage in forward planning, envisioning potential outcomes for various choices and making explicit decisions among alternatives. Through planning, agents can distil insights into a learned policy, as exemplified by the widely recognized AlphaZero approach. This method often leads to a notable enhancement in sample efficiency compared to approaches lacking a model.

However, a significant drawback arises from the unavailability of a ground-truth model of the environment to the agent. In cases where an agent seeks to utilize a model, it must learn the model solely from experiential data, presenting numerous challenges. The primary hurdle lies in the potential exploitation of biases within the learned model by the agent, resulting in performance discrepancies between the model's learned environment and the natural environment. Given the inherent

complexity of model learning, even extensive efforts involving substantial time and computational resources may not yield the desired outcomes.

Algorithms that incorporate a model are termed model-based methods, while those that do not are categorized as model-free. While model-free methods forgo the potential benefits of sample efficiency by utilizing a model, they often present a more straightforward implementation and tuning process.[3]

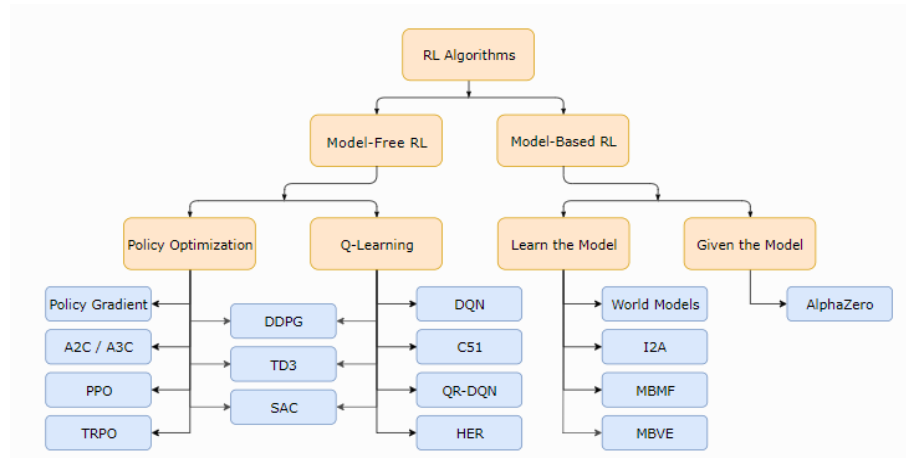


Figure 2.1: Taxonomy of Deep Reinforcement Learning Algorithms[3]

Another taxonomy of RL algorithms is based on how it is trained. With this, we can classify them into on-policy and off-policy algorithms. On-policy methods involve refining a policy that generates control actions within the environment, known as the behaviour policy. In contrast, off-policy methods utilize data from the behaviour policy to train a separate target policy for optimisation objectives. This distinction in updating policies with training data significantly influences the learning behaviour of different RL algorithms.

In on-policy RL, actions are generated based on observed environment states using a specific RL policy. The consequences of these actions are gathered and utilized to adjust the parameters of the same policy iteratively. Consequently, on-policy RL employs a single behaviour policy(π_B) and a target policy(π_T), which serves two primary purposes: (1) exploring the state and action spaces and (2) optimizing the learning objective based on the accumulated data. Most on-policy algorithms incorporate some action randomness to balance exploration and exploitation goals.

Thus, the RL agent may explore by occasionally selecting an action with uncertain outcomes. Other times, it may exploit the latest policy by choosing the action with the highest expected return for the current state. The result of the selected action is then used to update the policy parameters, thereby influencing subsequent behaviour iteratively. SARSA is an example of on-policy learning.

Off-policy RL involves managing a behaviour policy(π_B) and a target policy(π_T). While the behaviour policy is responsible for generating control actions corresponding to observed environment states, the target policy is continually trained using the resulting outcomes of these actions. Consequently, off-policy RL separates the process of data collection from policy training. A notable advantage of off-policy methods is their ability to learn an optimal target policy, such as a reward-maximizing policy, irrespective of the behaviour policy's exploration strategy. In off-policy learning, it is common practice to periodically update the behaviour policy with the most recent target policy to maximize learning efficiency. Q-learning is an example of off-policy learning. [4]

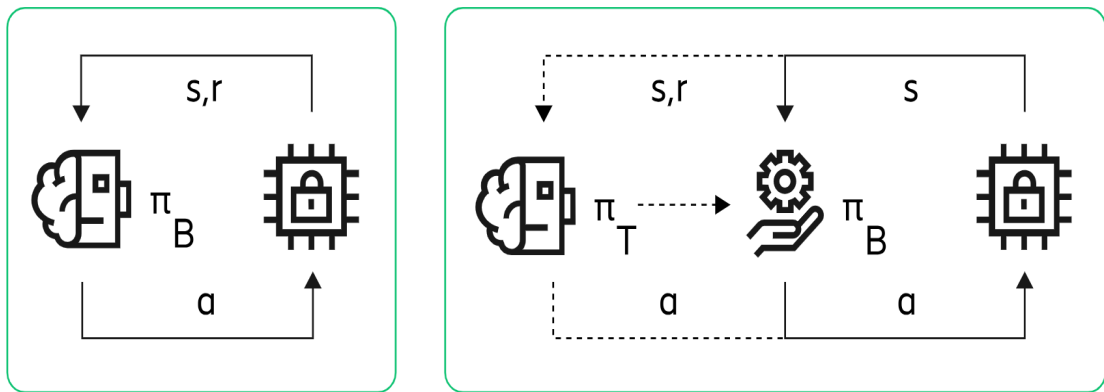


Figure 2.2: Left: On-Policy Learning, Right: Off-Policy Learning[4]

We shall now discuss the various state-of-the-art DRL algorithms.

2.1.1 Proximal Policy optimisation (PPO)

It belongs to a fresh lineage of policy gradient methods for reinforcement learning, distinguished by its methodology of alternating between data sampling through in-

teraction with the environment and optimizing a "surrogate" objective function via stochastic gradient ascent. Unlike conventional policy gradient methods, which typically execute one gradient update per data sample, PPO alternates between data sampling from the policy and executing multiple epochs of minibatch updates to improve sample efficiency.

PPO endeavours to take substantial steps toward policy enhancement based on the current dataset, ensuring advancement without risking performance deterioration (more stable). It incorporates a "clip" term in the surrogate objective function, which limits the extent of policy updates to prevent large deviations from the current policy. This helps prevent catastrophic policy collapses when policy updates are too aggressive. In simple words, we want to avoid having policy updates that are too large. Hence, the clip term is a ratio that determines how much the current policy has changed in comparison to the previous one limited within the limits $[1 - \epsilon, 1 + \epsilon]$. ϵ is a hyperparameter that helps us to define this clip range (in the paper, $\epsilon = 0.2$). The objective function

$$J(\theta) = \arg \max_{\theta} \mathbb{E} \left[\min \left(r(\theta) \hat{A} v_t, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A} v_t \right) \right] \quad (2.1)$$

$$r(\theta) = \frac{\pi_{\theta}(\hat{A}_t | s_t)}{\pi_{\theta_k}(\hat{A}_t | s_t)}, \text{ where } \pi_{\theta}: \text{ policy, } \hat{A}_t: \text{ action} \quad (2.2)$$

$$\hat{A} v_t^{GAE(\gamma, \lambda)} = \sum_{i=0}^{\infty} (\gamma \lambda)^i \delta_{t+i} \quad (2.3)$$

where:

$$\delta_t = R_t + \gamma V(s_{t+1}) - V(s_t)$$

R_t is the reward at time step t

$V(s_t)$ is the value function at time step t

γ is the discount factor

λ is a hyperparameter that controls the trade-off between bias and variance

PPO trains a stochastic policy in an on-policy manner, implying that it explores by selecting actions based on the most recent iteration of its stochastic policy. Initial conditions and the training process influence the degree of randomness in action selection. The policy usually exhibits diminishing randomness as training progresses, driven by the update rule's encouragement to exploit previously discovered rewards. However, this tendency can potentially lead the policy to become trapped in local optima.

PPO, therefore, is an on-policy deep reinforcement learning algorithm that balances sample efficiency and stability during training. While prioritising these aspects, it also incorporates mechanisms to manage the exploitation-exploration trade-off, such as stochastic action selection and policy update clipping. Nonetheless, like many on-policy methods, PPO may still face challenges with local optima during training.[14]

Algorithm 1 Proximal Policy optimisation (PPO) [14]

- 1: **Initialize:** policy parameters θ_0 , initial value function parameters ϕ_0
- 2: **Initialize:** replay buffer D
- 3: **for** $k = 0, 1, 2, \dots$ **do**
- 4: Collect set of trajectories $D_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
- 5: Compute rewards-to-go $\hat{R}_t$.
- 6: Compute advantage estimates $\hat{A}dv_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k} .
- 7: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|D_k|T} \sum_{\tau \in D_k} \sum_{t=0}^T \min \left(\frac{\pi_{\theta}(\hat{A}_t | s_t)}{\pi_{\theta_k}(\hat{A}_t | s_t)} \hat{A}dv^{\pi_{\theta_k}}(\hat{A}_t | s_t), g(\epsilon, \hat{A}dv^{\pi_{\theta_k}}(\hat{A}_t | s_t)) \right)$$

where,

$$g(\epsilon, \hat{A}dv) = \begin{cases} (1 + \epsilon)\hat{A}dv, & \text{if } \hat{A}dv \geq 0 \\ (1 - \epsilon)\hat{A}dv, & \text{if } \hat{A}dv < 0 \end{cases} \quad (2.4)$$

typically via stochastic gradient ascent with Adam optimizer. $\triangleright \hat{A}_t$ is action at state s_t $\triangleright \epsilon$ is a hyperparameter which roughly says how far away the new policy is allowed to go from the old

- 8: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|D_k|T} \sum_{\tau \in D_k} \sum_{t=0}^T \left(V_{\phi}(s_t) - \hat{R}_t \right)^2 \quad (2.5)$$

typically via some gradient descent algorithm

- 9: **end for**
-

2.1.2 Advantage Actor Critic (A2C)

The Advantage Actor-Critic (A2C) algorithm is a reinforcement learning technique that combines the advantages of both policy gradient methods and value-based methods. This hybrid approach helps to stabilize the training by reducing the variance.

In RL, we want to increase the probability of actions in a trajectory proportional to the returned rewards. The higher the reward, the higher the probability for the state-action pair, and vice versa. This makes the system unbiased as we are not estimating the reward, but the variance is high since the agent's interactions with the environment are purely stochastic (meaning it is random). The same starting state can lead to very different returns. Because of this, the return starting at the same state can vary significantly across episodes. The method to mitigate variance is to collect multiple episodes and increase the sample space. This would then affect the sample efficiency.

Actor-critic methods are useful for this problem as both networks learn from the current experience (on-policy). The actor-network takes the current state s_t and outputs the action $\hat{A}_t$, and the critic network takes s_t and $\hat{A}_t$ to output the Q-value. With this, we can reduce the stochastic nature of the rewards as the Q-value is based on the critic-network.

We can stabilize the learning further by introducing the term Advantage function, similar to TD(Temporal Difference) learning. The advantage function calculates how better taking that action at a state is compared to the average value of the state. It subtracts the mean value of the state from the state-action pair as in equation 2.6. Where $V(s)$ is the average value of that state (reward estimation). In other words, this will calculate the advantage(extra reward) we get for taking a particular state-action pair compared to the mean reward from that state.

$$Adv(s, \hat{A}) = Q(s, \hat{A}) - V(s) \quad (2.6)$$

If $Adv(s, \hat{A}) > 0$, we push the gradient in that direction, otherwise in the opposite direction. Since equation 2.6 requires two value functions, it becomes difficult, but fortunately, this is similar to TD learning. With that, we can rewrite equation 2.6

to equation 2.7 similar to equation 2.3.

$$\hat{A}dv(s, \hat{A}) = r + \gamma V(s') - V(s) \quad (2.7)$$

We would then get the objective function like equation 2.8

$$J(\theta) = \arg \max_{\theta} \mathbb{E} \left[\sum_{t=0}^T \left(\log \pi_{\theta}(\hat{A}_t | s_t) \cdot \hat{A}dv(s_t, \hat{A}_t) \right) \right] \quad (2.8)$$

One of the distinguishing features of A2C is its use of multiple parallel actor-learner threads to collect experiences from the environment concurrently. These parallel threads allow A2C to efficiently sample experiences and improve exploration by collecting diverse experiences. Parallel learning and sample-efficient actor-critic networks make A2C a popular algorithm for continuous actions. [15]

Algorithm 2 Advantage Actor-Critic (A2C) [15]

```

1: Initialize: actor-network  $\theta_0$ , critic-network  $\phi_0$ 
2: Initialize: replay buffer  $D$ 
3: for  $episode = 1 \rightarrow M$  do
4:   Initialize state  $s_0$ 
5:   Initialize  $\tau = \{\}$ 
6:   for  $t = 1 \rightarrow T_{max}$  do  $\triangleright$  number of steps per episode
7:     Choose action  $\hat{A}_t$  using policy  $\pi_\theta$ 
8:     Take action  $\hat{A}_t$  in the environment, observe reward  $r_t$ , next state  $s_{t+1}$ 
9:     Append  $(s_t, a_t, r_t, s_{t+1}, d_t)$  to trajectory  $\tau$   $\triangleright d_t$ : done bit
10:    if  $t \bmod N = 0$  or  $d_t$  is true then
11:      Compute advantage estimate  $\hat{A}dv_t$ 
12:      Append  $(s_t, a_t, A_t)$  to replay buffer  $D$ 
13:      Update  $\theta$  by minimizing policy loss using samples from  $D$ 
14:      Update  $\phi$  by minimizing value function loss using samples from  $D$ 
15:    end if
16:    if  $d_t$  is true then
17:      break
18:    end if
19:    Set  $s_t = s_{t+1}$ 
20:  end for
21: end for

```

2.1.3 Deep Deterministic Policy Gradient (DDPG)

DDPG is an actor-critic algorithm that learns a deterministic policy (produces the action value and not probability distribution) that can efficiently handle high dimensional action spaces. It is an off-policy learning algorithm where the actor-network learns a deterministic policy, and the critic-network estimates the value function.

The actor-network in DDPG is responsible for mapping states directly to actions deterministically. Unlike stochastic policies that output a probability distribution over actions, the deterministic policy generated by the actor-network provides a specific action for a given state. This allows DDPG to effectively handle continuous action spaces without the need for sampling actions from a distribution.

Meanwhile, the critic network evaluates the quality of the actions selected by the actor-network by estimating the value function. The value function predicts the expected cumulative reward starting from a given state-action pair. By evaluating the actions' quality, the critic network provides feedback to the actor-network, facilitating learning of better action-selection strategies.

DDPG incorporates target networks for both the actor and critic networks to improve stability during training. These target networks are periodically updated to slowly track the learned networks' parameters, which helps mitigate the issues of oscillations and divergence during training. Additionally, DDPG utilizes an experience replay buffer to store and sample experiences encountered during interactions with the environment. This buffer enables more efficient use of data and improves sample efficiency by breaking temporal correlations.[30]

Due to the deterministic nature of the policy, if the agent solely explores on-policy at the initial stages, it might not sufficiently sample a diverse range of actions to uncover valuable learning signals. To enhance exploration within DDPG policies, noise is introduced to their actions during training. While the original DDPG paper proposed time-correlated Ornstein-Uhlenbeck (OU) noise, recent findings indicate that uncorrelated, zero-mean Gaussian noise is equally effective. This simpler alternative is preferred due to its ease of implementation. Employing such noise aids in obtaining higher-quality training data, leading to more effective learning processes. [16]

The objective function for DDPG is given by the equation 2.9.

$$J(\theta) = \arg \max_{\pi} \mathbb{E}[Q(s, \pi(s))] \quad (2.9)$$

Algorithm 3 Deep Deterministic Policy Gradient (DDPG) [16]

- 1: **Initialize:** policy parameters θ , Q-function parameters ϕ
- 2: **Initialize:** replay buffer D
- 3: Set target parameters to main parameters: $\theta_{\text{targ}} \leftarrow \theta, \phi_{\text{targ}} \leftarrow \phi$
- 4: **repeat**
- 5: Initialize a random process for action exploration
- 6: Receive initial observation state s_1
- 7: **for** timestep = 1 to T **do**
- 8: Select action $\hat{A}_t = \mu(s_t | \theta^\mu) + \mathcal{N}$
- 9: Execute action $\hat{A}_t$ and observe reward r_t and new state s_{t+1}
- 10: Store transition $(s_t, \hat{A}_t, r_t, s_{t+1})$ in D
- 11: Sample a random minibatch of transitions $(s_i, \hat{A}_i, r_i, s_{i+1})$ from D
- 12: Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$
- 13: Update critic by minimizing the loss: $\mathcal{L} = \frac{1}{N} \sum_i (y_i - Q(s_i, \hat{A}_i | \theta^Q))^2$
- 14: Update actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, \hat{A} | \theta^Q) |_{s=s_i, \hat{A}=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s_i} \quad (2.10)$$

- 15: Update the target networks:

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \quad (2.11)$$

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \quad (2.12)$$

- 16: **end for**
 - 17: **until** convergence
-

2.1.4 Twin Delayed Deep Deterministic Policy Gradient (TD3)

Although DDPG may occasionally exhibit impressive performance, it often proves fragile regarding hyperparameter settings and other tuning aspects. A prevalent challenge encountered with DDPG is the tendency for the learned Q-function to overestimate Q-values significantly. Consequently, this overestimation can cause the policy to become unstable as it exploits inaccuracies in the Q-function. Twin Delayed DDPG (TD3) emerges as a solution to this problem by incorporating three pivotal strategies:

First trick: Clipped Double-Q-Learning. The learning process involves training two Q-functions instead of just one, hence the term "twin." The algorithm then utilizes the lower of the two Q-values when constructing the targets in the Bellman error loss functions.

Second trick: Delayed Policy Updates. The policy, as well as the target networks, are updated less frequently compared to the Q-function. According to the paper's recommendations, for every two updates to the Q-function, only one update is performed for the policy.

Third trick: Target Policy Smoothing. Noise is introduced to the target action to make it more difficult for the policy to exploit errors in the Q-function. This noise smooths out the Q-values along with changes in action, thereby improving the robustness of the learning process.

Like in DDPG, exploration is achieved by adding noise to the actions during training. Overall, TD3 builds upon the strengths of DDPG while addressing some of its limitations, such as overestimation bias and instability. By introducing twin Q-functions, target policy smoothing, and delayed policy updates, TD3 achieves improved sample efficiency and stability, making it a powerful algorithm for reinforcement learning in continuous action spaces. [17]

Algorithm 4 Twin Delayed Deep Deterministic Policy Gradient (TD3) [17]

```
1: Initialize: policy parameters  $\theta$ , Q-function parameters  $\phi$ 
2: Initialize: target Q-function parameters  $\phi' \leftarrow \phi$ , target policy parameters  $\theta' \leftarrow \theta$ 
3: Initialize: replay buffer  $D$ 
4: repeat
5:   Initialize a random process for action exploration
6:   Receive initial observation state  $s_1$ 
7:   for timestep = 1 to  $T$  do
8:     Select action  $\hat{A}_t = \pi(s_t|\theta) + \mathcal{N}$ 
9:     Execute action  $\hat{A}_t$  and observe reward  $r_t$  and new state  $s_{t+1}$ 
10:    Store transition  $(s_t, \hat{A}_t, r_t, s_{t+1})$  in  $D$ 
11:    if it's time to update then
12:      for j in range(Timesteps) do
13:        Sample a random minibatch of transitions  $(s_i, \hat{A}_i, r_i, s_{i+1})$  from  $D$ 
14:        Set  $y_i = r_i + \gamma \min_{j=1,2} Q'(s_{i+1}, \pi'(s_{i+1}|\theta'))|_{\phi'_j}$ 
15:        Update Q-functions by minimizing the loss:
```

$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i (Q(s_i, \hat{A}_i|\phi) - y_i)^2$$

```
16:      if j mod policy_delay = 0 then
17:        Every  $d$  steps of critic update, update policy by maximizing the
        Q-value:
```

$$\nabla_{\theta} J \approx \frac{1}{N} \sum_i \nabla_{\hat{A}} Q(s, \hat{A}|\phi)|_{s=s_i, \hat{A}=\pi(s_i)} \nabla_{\theta} \pi(s|\theta)|_{s_i}$$

```
18:      Update target networks:
```

$$\phi' \leftarrow \rho\phi + (1 - \rho)\phi'$$

$$\theta' \leftarrow \rho\theta + (1 - \rho)\theta'$$

```
19:      end if
20:    end for
21:  end if
22: end for
23: until convergence
```

2.1.5 Soft Actor-Critic (SAC)

SAC presents an algorithm that optimizes a stochastic policy through off-policy methods, effectively bridging the gap between stochastic policy optimisation and DDPG-style approaches. While not directly succeeding TD3 (as both were published roughly concurrently), SAC incorporates the clipped double-Q trick. Furthermore, owing to the inherent stochasticity of SAC’s policy, it naturally reaps benefits akin to target policy smoothing.

A central feature of SAC lies in entropy regularization. Here, the policy undergoes training to maximize a trade-off between expected return and entropy, which measures the level of randomness in the policy. This concept closely relates to the exploration-exploitation trade-off: by increasing entropy, the policy engages in more exploration, thus potentially accelerating learning in later stages. Additionally, it serves as a safeguard against premature convergence to suboptimal local optima.

Entropy measures the degree of randomness exhibited by a random variable. For instance, consider a weighted coin: if it predominantly lands on heads, its entropy is low; conversely, if it is evenly balanced and equally likely to land on either side, its entropy is high. The entropy term is given by equation 2.13, where P is the probability mass or density of x . This is also known as Shannon entropy [31].

$$H(P) = \mathbb{E} [-\log P(x)], \text{ Shannon entropy [31]} \quad (2.13)$$

Like in TD3, SAC also uses MSBE (Mean Squared Bellman Error) minimization and regresses to a single shared target, where the shared target uses the clipped-double-Q trick. However, SAC includes the entropy regularization term, and unlike the target policy in TD3, the next state-action pair comes from the current policy. TD3 uses target policy smoothing by incorporating action noise, but SAC trains a stochastic policy, and the noise from the stochasticity is sufficient to get a similar effect. The Q-value approximation is given by equation 2.14.

$$Q^\pi(s, \hat{A}) = \mathbb{E} \left[r + \gamma Q^\pi(s', \hat{A}') + \alpha H(\pi(.|s')) \right] \quad (2.14)$$

The estimation is averaged across the samples, where the next state comes from the replay buffer, and the next action comes from the current policy. The policy should then learn to maximize the expected future reward and entropy, as in equation 2.15.

$$V^\pi(s) = \mathbb{E} \left[Q^\pi(s, \hat{A}) + \alpha H(\pi(\cdot|s')) \right] \quad (2.15)$$

Here, the Q^π term will be the minimum of the two Q-value networks maintained like in TD3. The policy $\pi(\theta)$ is from the current policy network with a sample of action drawn from equation 2.16. $\tanh$ is a squashing function that ensures the actions are bounded to a finite range.[18]

$$\tilde{A}_\theta(s, \xi) = \tanh(\mu_\theta(s) + \sigma_\theta(s) \cdot \xi), \quad \xi \sim N(0, 1) \quad (2.16)$$

SAC utilizes entropy regularization to train a stochastic policy and explores the environment in an on-policy manner. The coefficient α in the entropy regularization explicitly governs the trade-off between exploration and exploitation, where higher α values encourage more exploration, while lower values favour exploitation. Determining the optimal coefficient, which leads to stable learning with the highest rewards, may necessitate meticulous tuning and can vary across different environments.

The policy's ability to exploit its learned knowledge is assessed during testing by removing stochasticity and employing the mean action instead of sampling from the distribution. This adjustment typically enhances performance compared to using the original stochastic policy.

Algorithm 5 Soft Actor-Critic (SAC) [18]

- 1: **Initialize:** policy parameters θ , Value function parameters ϕ_1 , Q-function parameters ϕ_2
- 2: **Initialize:** replay buffer D
- 3: **Initialize:** target Value function parameters $\phi_{\text{targ}} \leftarrow \phi_1$, target Q-function parameters $\phi_{\text{targ}} \leftarrow \phi_2$ **repeat**
- 4: Observe state s and select action $\hat{A}\tilde{\pi}_\theta(\cdot|s)$
- 5: Execute $\hat{A}$ in the environment
- 6: Observe next state s' , reward r , and done signal d
- 7: Store $(s, \hat{A}, r, s', d)$ in replay buffer D
- 8: **if** it's time to update **then**
- 9: **for** j in range(Timesteps) **do**
- 10: Sample a batch of transitions (s, a, r, s') from D
- 11: Compute targets for the Q function

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \hat{A}') - \alpha \log \pi_\theta(\hat{A}'|s') \right), \hat{A}'\tilde{\pi}_\theta(\cdot|s') \quad (2.17)$$

- 12: Update Q-function by minimizing the loss:

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s') \sim D} [(Q(s, a|\phi_i) - y)^2], \text{ for } i = 1, 2$$

- 13: Update policy by maximizing the expected Q-value:

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{s \sim D} [\nabla_\theta \log \pi(a|s, \theta) Q(s, a|\phi)]$$

- 14: Update target networks:

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i$$

$$\theta_{\text{targ}} \leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta$$

- 15: **end for**
 - 16: **end if**
 - 17: **until** convergence
-

2.2 Mixed Boundary Value Problem

The TEM field of a shielded single microstrip line (Figure 1.3) can be approximated, for example, by applying the conformal mapping method [22] [23] (this makes it possible to calculate the line parameters using simple formulas) or by using Green's function [24] [25].

The characteristic impedance of a microstrip line arrangement depends primarily on the substrate height, the stripline width, and the substrate material's dielectric constant. If the substrate height and dielectric constant are predetermined, the trace width remains the only free variable.

in [20], a solution method for calculating the TEM field of the microstrip line according to Fig. 1 is presented, based on the formulation of a mixed boundary value problem and its iterative solution.

In [21], the solution of the Laplace equation is described by a Fourier series approach. The unknown series coefficients are determined using 1st order spline functions to describe the potential function $V(x, y=c)$ and the minimization of the electrostatic energy in the region under consideration.

2.2.1 Problem Definition

The mathematical/physical requirements for using Physically Based AI Methods to calculate the required characteristic line parameters are explained below.

As a model for calculating the electromagnetic field of a symmetrically arranged microstrip line, the planar arrangement shown in Figure 2.3 is considered, which is limited by the conductive coordinate surfaces $x = \pm a$ and $y = 0$ or $y = b$. The dielectric substrate material of dielectric constant ϵ_1 is located between the areas $y = c$ and $y = b$; the dielectric substrate material of dielectric constant ϵ_2 is located between the areas $y = 0$ and $y = c$.

The thin conductive strip ($t = 0$) is arranged on the substrate surface $y = c$ in the area $x \leq |d|$. The strip has the potential V_0 in relation to the conductive boundary. If the electrical energy of this arrangement and that of the arrangement without dielectric substrate material is known, the characteristic impedance and the effec-

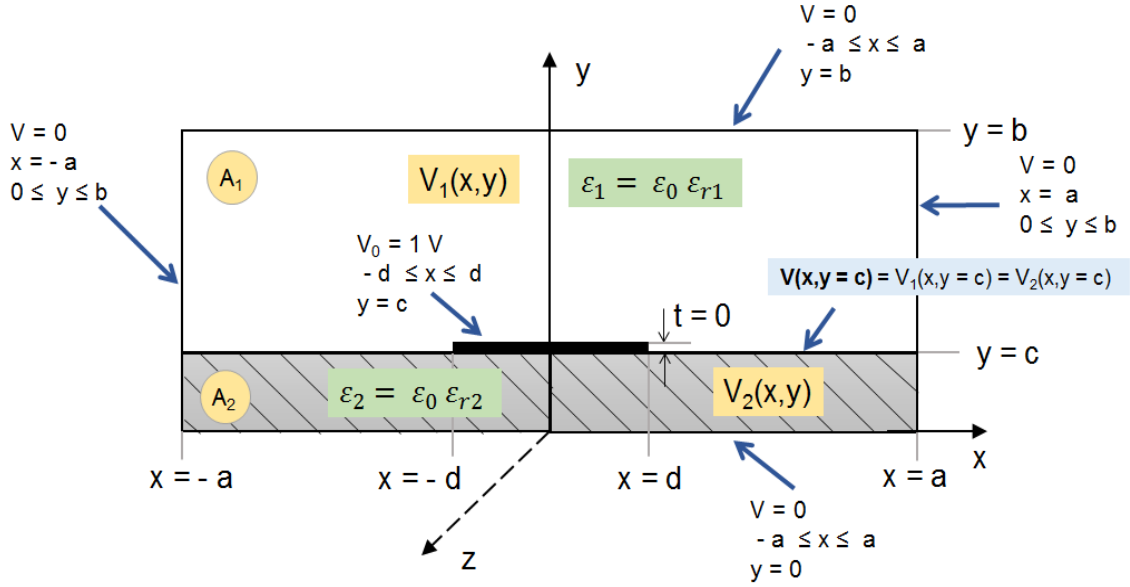


Figure 2.3: Microstrip Arrangement under Study

tive dielectric constant for the Q-TEM case can be calculated from this under the above-mentioned conditions.

This defines a mixed boundary value problem, for which the potential on the charged area $x \leq |d|$ (conductive layer) and the ideal conductive boundary is given but is unknown on the charge-free area $|d| < x \leq |a|$ outside the conductive strip.

2.2.2 Mathematical Formulations for Solving the Mixed Boundary Value Problem

In order to calculate the required characteristic conduction parameters Z_D (characteristic impedance) and ϵ_{eff} (effective dielectric constant) of the arrangement according to Figure 1.3, the potential curve $V(x, y = c)$ in the region $-a \leq x \leq a$ and the surface charge density $\sigma(x)$ in the region $-d \leq x \leq d$ or the electrical energy in the entire region $(A_1 + A_2)$, W' , must be known.

The following formulations are made in order to mathematically formulate the solution to the mixed boundary value problem:

- The arrangement shown in Figure 1.3 is provided with a conductive limitation of the potential $V = 0$. The dimensions b and a must be selected so that the characteristics of the Q-TEM field are not significantly affected;
- The ground plane at $y = 0$ ($-a \leq x \leq a$) also has the potential $V = 0$ (static case; Q-TEM);
- The conductive strip ($-d \leq x \leq d$) is assigned the constant potential V_0 (static case; Q-TEM);
- In the case considered here, the result is a plane field problem. This means that the required potentials $V_1(x, y)$ and $V_2(x, y)$ for the areas A1 and A2 (see Figure 2.3) are not dependent on the coordinate z .

2.2.3 Solution Approach

The following system-describing equation (Laplace equation) can be solved for the single microstrip arrangement shown in Figure 2.3.

$$\Delta V(x, y) = 0 \tag{2.18}$$

Using the separation of variables method (Bernoulli's product approach)

$$V(x, y) = X(x)Y(y) \tag{2.19}$$

By inserting it into the Laplace equation above, two ordinary differential equations are obtained for the functions $X(x)$ and $Y(y)$. These will be set up separately for areas A_1 and A_2 . The solutions are trigonometric functions in the x -direction and hyperbolic functions in the y -direction

2.2.4 Essential Equations

As mentioned in an earlier chapter, the energy values depend on the potential distribution across the microstrip arrangement. We shall now look at the equations that help us to approximate the potential distribution and its relation to the energy value.

Since the potential distribution curve is symmetrical across the microstrip, we can look at one-half of the curve for further analysis. Piecewise linear functions created by the g-points approximate the potential curve, as shown in Figure 2.4. The following equations define all the relations required to relate the shape of the potential curve and energy.

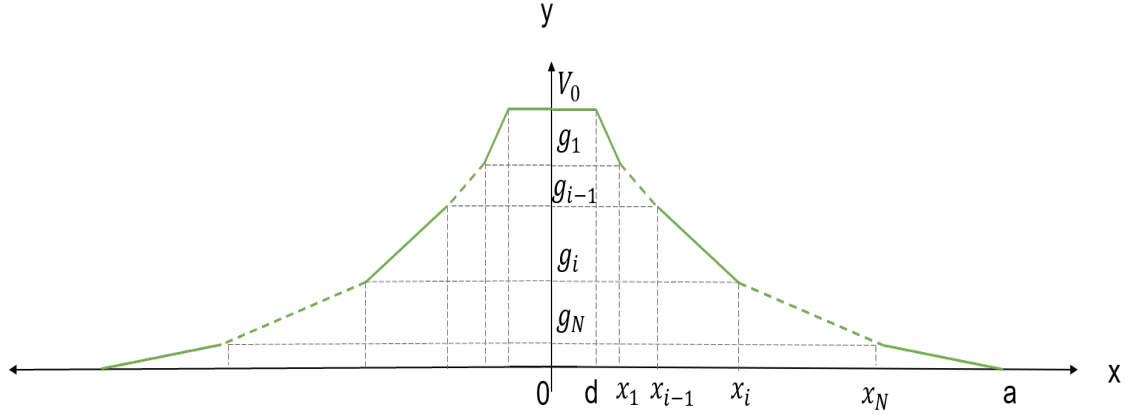


Figure 2.4: Approximation of the Potential Distribution by Spline Functions of the First Order

With this approximation, equations are derived to calculate the energy, surface charge density, characteristic impedance, effective dielectric constant, and phase velocity [32] [26]. Here, the equations are scaled to accommodate $V_0 = 1$, which can be scaled back by multiplying the actual potential value V .

The potential distribution on the surface of the microstrip ($y = c$) is given by:

$$V(x, y = c) = \sum_{n=0}^{\infty} v_n \cos \left[(2n + 1) \frac{\pi}{2a} x \right] \quad (2.20)$$

where v_n is given by,

$$\begin{aligned}
 v_n = & \frac{2}{a} V_0 \left(\frac{1}{[(2n+1)\frac{\pi}{2a}]^2} \right) \left\{ \frac{g_1 - 1}{x_1^G - d} \left[\cos \left((2n+1) \frac{\pi}{2a} x_1^G \right) - \cos \left((2n+1) \frac{\pi}{2a} d \right) \right] \right. \\
 & + \sum_{i=2}^N \frac{g_i - g_{i-1}}{x_i^G - x_{i-1}^G} \left[\cos \left((2n+1) \frac{\pi}{2a} x_i^G \right) - \cos \left((2n+1) \frac{\pi}{2a} x_{i-1}^G \right) \right] \\
 & \left. + \frac{g_N}{a - x_N^G} \cos \left((2n+1) \frac{\pi}{2a} x_N^G \right) \right\}
 \end{aligned} \tag{2.21}$$

the energy of the curve is then given by,

$$\begin{aligned}
 W_2'^1 = & \frac{\epsilon_1}{4} \left\{ \sum_{n=0}^{\infty} v_n^2 (2n+1) \pi \coth \left((2n+1) \frac{\pi}{2a} (b-c) \right) \right\} \\
 & + \frac{\epsilon_2}{4} \left\{ \sum_{n=0}^{\infty} v_n^2 (2n+1) \pi \coth \left((2n+1) \frac{\pi}{2a} c \right) \right\}
 \end{aligned} \tag{2.22}$$

the surface charge density is given by,

$$\sigma(x, y = c) = \sum_{n=0}^{\infty} \sigma_n \cos \left[(2n+1) \frac{\pi}{2a} x \right] \tag{2.23}$$

where,

$$\sigma_n = \psi_n v_n \tag{2.24}$$

and,

$$\psi_n = \left[(2n+1) \frac{\pi}{2a} \right] \left\{ \epsilon_2 \coth \left((2n+1) \frac{\pi}{2a} c \right) + \epsilon_1 \coth \left((2n+1) \frac{\pi}{2a} (b-c) \right) \right\} \tag{2.25}$$

With equations 2.20 - 2.25, we can approximate and calculate the attributes of the curve. From the energy calculations, we can then approximate the characteristic impedance of the system by considering two microstrip arrangements where one of them will have both mediums as air. Let us name those arrangements as Case D and L (Table 5.18 and Table 5.19)

Capacitance for the two cases will then be

$$W'_D = \frac{1}{2} C'_D V_0^2 \quad (2.26)$$

$$C'_D = 2 \frac{W'_D}{V_0^2} \quad (2.27)$$

$$W'_L = \frac{1}{2} C'_L V_0^2 \quad (2.28)$$

$$C'_L = 2 \frac{W'_L}{V_0^2} \quad (2.29)$$

Characteristic impedance will then be,

$$Z'_D = \frac{1'}{c_0 \sqrt{C'_D C'_L}} \quad (2.30)$$

$$Z'_L = \frac{1'}{c_0 C'_L} \quad (2.31)$$

The effective dielectric constant is then given by,

$$\epsilon_{eff} = \frac{C'_D}{C'_L} \quad (2.32)$$

and phase velocity,

$$v_{ph} = \frac{c_0}{\sqrt{\epsilon_{eff}}} \quad (2.33)$$

where, $\mu_r = 1$ and $c_0 = 299792458 m/s$

A detailed explanation and derivation of these equations are given in the appendix.

Chapter 3

Realization Of Software Modules

The DRL approach requires two fundamental units: the working environment and the agent. Since the shape optimisation problem cannot be realized in the real world, we require simulations of it in software. We shall now discuss the simulation environment and agent for our DRL approach.

3.1 Environment

We shall adopt our environment from the popular Gymnasium project by OpenAI. Gymnasium is a project designed to offer an API for all single-agent reinforcement learning environments, featuring implementations of popular environments such as cartpole, pendulum, mountain-car, mujoco, atari, and more.

The API comprises four main functions: `make`, `reset`, `step`, and `render`, which this introductory overview will cover. At the heart of Gymnasium lies `Env`, a Python class that serves as a high-level representation of a Markov Decision Process (MDP) from reinforcement learning theory. It's worth noting that this representation isn't a perfect reconstruction and may lack certain components of MDPs. Environments (MDPs) within Gymnasium are implemented as `Env` classes, complemented by `Wrappers`, which offer useful utilities and can modify the results provided to the user. [10]

The agent interacts with the environment like in Figure 1.2, and the example code for its implementation is shown in the Listing 1.

Listing 1 Gymnasium Example [10]

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset()

for _ in range(1000):
    # agent policy that uses the observation and info
    action = env.action_space.sample()
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

We can inherit the abstract class `gymnasium.Env` and create our own custom environment class. This custom environment will then be used for simulation while training.

3.2 Agent

We shall adopt our Agent from the popular reinforcement learning library `stable-baselines3` [11]. This library is built modular, making it easy to implement and develop with required customizations.

Unlike its predecessors, which were based on TensorFlow, `Stable Baselines3` is built on top of PyTorch [33]. This allows users to leverage PyTorch's flexibility and ease of use while developing and experimenting with reinforcement learning algorithms.

`Stable Baselines3` offers a straightforward and user-friendly API, simplifying reinforcement learning agents' training, evaluation, and deployment processes. By abstracting much of the intricacy associated with implementing reinforcement learning algorithms, the library enables users to concentrate on exploring various techniques and methodologies.

An example implementation of the SAC algorithm API from stable-baselines3 is shown in the Listing 2

Listing 2 SAC Example SB3 [11]

```
import gymnasium as gym

from stable_baselines3 import SAC

env = gym.make("Pendulum-v1", render_mode="human")

model = SAC("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=10000, log_interval=4)
model.save("sac_pendulum")

del model # remove to demonstrate saving and loading

model = SAC.load("sac_pendulum")

obs, info = env.reset()
while True:
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        obs, info = env.reset()
```

3.3 Software optimisations

Since the equations 2.21 and the rest have parallel operations, we can exploit multi-core parallel computing. This can be achieved using the native multiprocessing module of python [12].

The multiprocessing package provides a way to create processes similar to the threading module. It enables local and remote concurrency, bypassing the Global Interpreter Lock using subprocesses instead of threads. This allows programmers to utilize multiple processors effectively. It is compatible with both POSIX and Win-

dows systems.

Additionally, multiprocessing introduces features not found in the threading module. One notable feature is the Pool object, which simplifies parallelizing function execution across multiple input values, distributing data among processes (known as data parallelism). Below is an example demonstrating the typical practice of defining such functions in a module to ensure successful importation by child processes. Listing 3 showcases basic data parallelism using Pool. This multi-core processing will significantly improve the runtime speed of the program.

Listing 3 Multiprocessing Example [12]

```
from multiprocessing import Pool

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(5) as p:
        print(p.map(f, [1, 2, 3]))
```

Chapter 4

Training Metrics and Tensorboard Logging

During the training, we shall use tensorboard logging to log the various parameters related to the agent's performance[5]. In the context of Reinforcement Learning, the quality of the trained agent can be evaluated mainly based on cumulative rewards, value loss/ critic loss, and policy gradient loss/ actor loss.

4.1 Rewards

The cumulative reward is the fundamental metric upon which we shall evaluate the performance of our agent. As the core principle of Reinforcement Learning is to maximize the reward (unlike conventional machine learning algorithms, where the loss function is minimized), we shall try to train the agent until a maximum known reward is achieved.

However, such straightforward practices are not always practical. In real-world scenarios where reinforcement learning is usually deployed, the environment is mostly unknown, thereby reducing our prior knowledge of the maximum reward that can be obtained.

In this context, we shall slightly tweak our approach and look for the trend in reward, mainly if it is rising and when it converges. Setting rewards based on certain rules helps us better understand how the agent performs since we can correlate the reward value to the agent's state.

Speed of convergence is desired, but a fast convergence could also mean the agent is stuck at a non-global solution. This can only be monitored by testing the agent after training.

4.2 Value Loss/Critic Loss

In Reinforcement Learning, particularly within actor/critic algorithms, value loss refers to the disparity (or an average of such disparities) between the anticipated state value by the learning algorithm and the actual value observed empirically.

In essence, the state's value reflects the expected reward when initiating from that state. Immediate rewards contribute fully to this estimation, while potential rewards that might occur but don't immediately contribute diminish in value over time, a concept known as "discounting."

Expected value denotes the prediction of the state's value by the critic component of the algorithm. In the context of a critic realized through a neural network, it corresponds to the network's output when provided with the state as input.

Empirically observed value quantifies the accumulated rewards received upon leaving the state and any subsequent rewards (discounted over time) obtained immediately after for a specified number of steps (let's assume these steps lead to state X). Additionally, depending on the implementation, it may include a discounted value based on the anticipated value of state X.

A smaller value loss signifies an enhanced ability to forecast performance. However, this doesn't necessarily equate to the agent's improved ability. It only means that the network can predict the value from a given state. This is why exploration is vital for the agent to learn the unknown environment accurately.

4.3 Policy Gradient Loss/Actor Loss

When evaluating Policy Gradient Loss or Actor Loss during Reinforcement Learning training, attention should be directed towards several critical aspects.

The Policy Gradient Loss (or Actor Loss) should steadily converge to a low and consistent value over time. This trend signals that the policy network is effectively learning an optimal policy to maximise expected cumulative rewards.

Furthermore, assessing the correlation between the decreasing Policy Gradient Loss and the increasing cumulative reward throughout training is essential. A decline in loss should ideally coincide with an uptick in rewards. If the loss diminishes while rewards plateau or decline, it may suggest issues with the reward function or the approximation of the value function, particularly in actor-critic methods.

Stability is another key consideration. The Policy Gradient Loss should demonstrate a smooth and stable convergence pattern. Abrupt fluctuations or irregular spikes could indicate instability within the learning process, necessitating potential adjustments to hyperparameters, network architecture, or exploration strategies.

In some scenarios, incorporating an entropy term (entropy coefficient) into the Policy Gradient Loss helps foster exploration and prevents premature convergence to deterministic solutions. Monitoring the contribution of this entropy term to the overall loss is advisable.

In actor-critic methods, where the Policy Gradient Loss is weighted by the advantage function, assessing the quality of advantage estimates becomes crucial. Inaccurate estimates can lead to suboptimal policy updates, highlighting the importance of thorough analysis. Additionally, monitoring clipping or constraint terms in the loss function, as employed by certain policy gradient algorithms like PPO, helps prevent drastic policy updates and enhances stability.

Furthermore, comparing the Policy Gradient Loss with the Value Loss (or Critic Loss) in actor-critic methods is beneficial. If the Value Loss remains high despite a decrease in Policy Gradient Loss, it may indicate inaccuracies in value estimates provided by the critic, resulting in suboptimal policy updates.

By meticulously analyzing the Policy Gradient Loss (or Actor Loss) alongside metrics such as cumulative rewards, value loss, and exploration behaviour, valuable insights into the effectiveness of the policy network can be obtained. With these insights, we can make informed decisions regarding potential adjustments to the learning process.

Chapter 5

Concept Development

For any deep reinforcement learning algorithm, we must define the three basic units: Observation Space, Action Space, and Reward Scheme. Experiments with various methods were conducted to better understand the problem and explore different strategies.

5.1 Method 1: Scatter Points

The most intuitive approach for our problem is to ask the agent to generate the points for the piece-wise linear function. Here, the x-coordinates are preset, and we expect the agent to predict the corresponding y-coordinates. Since the potential curve is normalized to be within the range $(0,1)$, the action space is also limited between the range $(0,1)$. For this experiment, the action space is an array of size 10, where each value can vary within the range $(0,1)$, as shown in table 5.1.

Action	Type	Size	Value Range
g-points	continuous-space array	10	$(0,1)$

Table 5.1: Action for Method 1

The observation space for this experiment was set as an array of size 12, where the first 10 values were the y-coordinates for the g-points(piece-wise linear function points). The other two were d (half-width of the microstrip) and a (half-width of the

arrangement), as shown in table 5.2. The main idea was to give the parameters of the arrangement geometry and a starting point given by the g-points part in the observation space so that the agent can predict the next g-points for the given arrangement.

Observation Space	Value Range
g-points	(0,1)
d (half-width microstrip)	(0, Inf)
a (half-width arrangement)	(0, Inf)

Table 5.2: Observation Space for Method 1

The rewards are straightforward, where it checks for the necessary conditions first and then looks for the one with the least energy following Thomson’s theorem [27], and rewards it. Since reward maximization is needed, the inverse of the energy value is given as the reward, achieving the least energy prediction while maximizing the reward. This reward scheme also penalises the actions when the necessary conditions are unmet. The complete reward scheme is given in algorithm 6.

Due to the usage of continuous action space with a model-less approach, we shall proceed with the PPO algorithm with an MLP policy.

The agent was trained for one million timesteps, and the result was very poor. As we can observe from Figure 5.1, the mean reward did not even cross -1. From Figure 5.2, it can be observed that the loss has converged. The two plots will infer that the agent failed to learn anything. In this approach, we expected the agent to provide the coordinates based on the arrangement parameters. The complete randomness of the choices and since the degree of randomness increases with an increase in the number of points, it was expected to receive a poor result.

We can also see that the reward scheme provided for this approach used negative rewards to penalize when the conditions are not met. Since the rewards are given in a nested format, a negative reward will penalize the positive action that meets some conditions.

Algorithm 6 Reward Scheme for Method 1

```

1:  $W_{minimum} = \inf$  ▷ global minimum energy
2:  $x - coordinates - gpts_{prediction} \leftarrow$  generate points in the range( $d, a$ )
3:  $y - coordinates - gpts_{prediction} \leftarrow \hat{A}$  ▷ Action from agent
4:  $rewards \leftarrow 0$ 
5: if all  $gpts_{prediction}s > 0$  and all  $gpts_{prediction} < 1$  then
6:    $reward \leftarrow reward + 1$ 
7:   if  $gpts_{prediction}$  is monotone decreasing then
8:      $reward \leftarrow reward + 2$ 
9:     if  $gpts_{prediction}$  is convex then
10:       $reward \leftarrow reward + 2$ 
11:       $W_{prediction} \leftarrow energy(gpts_{prediction})$ 
12:      if  $W_{prediction} < W_{minimum}$  then
13:         $W_{minimum} \leftarrow W_{prediction}$ 
14:         $reward \leftarrow reward + \frac{1}{W_{prediction}}$ 
15:      end if
16:    else
17:       $reward \leftarrow reward - 2$ 
18:    end if
19:  else
20:     $reward \leftarrow reward - 2$ 
21:  end if
22: else
23:    $reward \leftarrow reward - 1$ 
24: end if

```

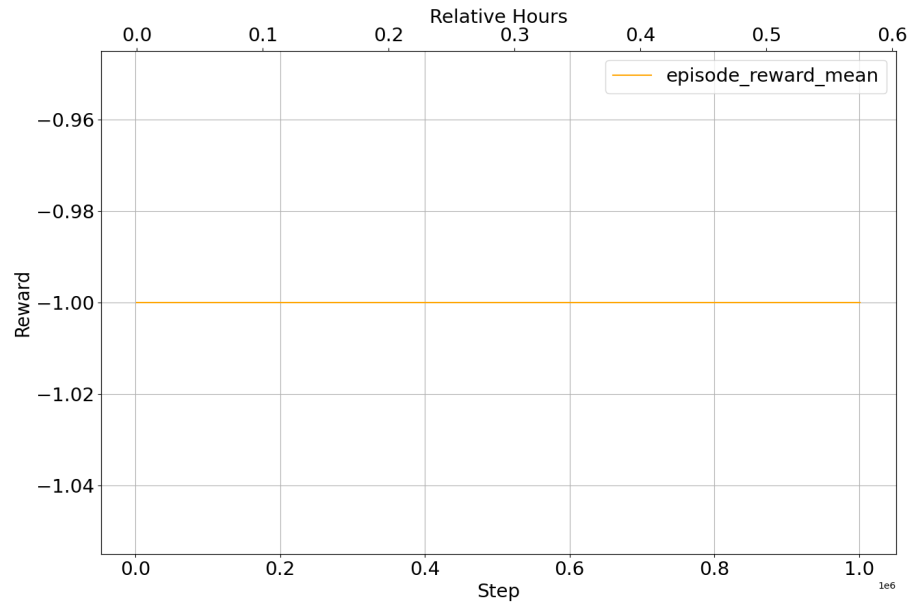


Figure 5.1: Episode Reward Mean for Method 1 [5]

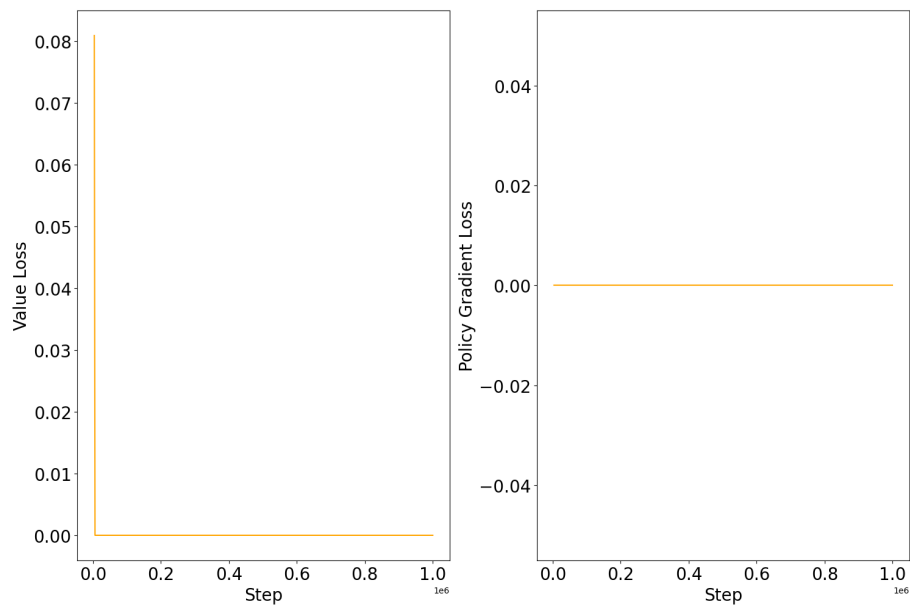


Figure 5.2: Value Loss(Left), Policy Gradient Loss(Right) for Method 1 [5]

5.2 Method 2: Decaying Exponential Function

The optimisation problem requires finding the optimal shape of the potential distribution for a microstrip arrangement like that shown in fig 1.3. In [28] and [29], researchers have introduced a new approach to how deep reinforcement learning is implemented called Degenerate Deep Reinforcement Learning (single-step DRL). As shown in the Figure 5.3, this approach does not have intermediate states.

Unlike conventional DRL problems, shape optimisations do not have intermediate states, and they expect to obtain the optimal shape given the characteristics of the environment, which is the observation space here. Hence, one training episode includes a single look at the environment characteristics and predicts the parameters for the optimal shape. We can consider this approach as a black-box optimizer, as mentioned in [28], as the optimal behaviour to be learnt is independent of states, as is notably the case in open-loop control problems.

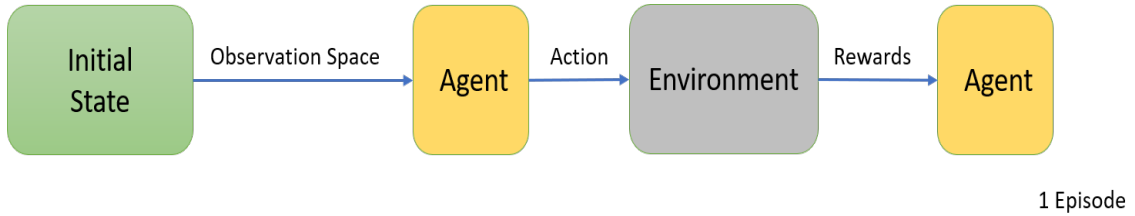


Figure 5.3: Degenrate Deep Reinforcement Learning framework

The novelty of this approach can be summarized as follows: in traditional deep reinforcement learning (DRL), a deep neural network (DNN) learns the optimal sequence of actions based on observations to maximize rewards. However, in single-step DRL, the focus shifts to learning the optimal mapping f_{θ^*} such that the optimal action $action^*$ is obtained as $action^* = f_{\theta^*}(s_0)$, where s_0 represents an input state (often a fixed vector) repeatedly presented to the agent. Consequently, the optimal policy transforms the initial state s_0 into the desired action $action^*$.

As a result, single-step DRL algorithms can employ significantly smaller networks compared to typical agent architectures in other DRL approaches. This reduction is possible because the agent is not tasked with learning complex state-action relationships but rather focuses on learning a transformation from a constant input

state to a specified action.

Hence, the observation space needs to reflect the parameters of the microstrip arrangement, namely the dimensions and the dielectric constants of the second medium present in the arrangement (the first medium is air, and hence it is not considered here). We shall set our observation space as a 1-D array with size five that includes the characteristics of the arrangement as shown in table 5.3.

Observation Space	Value Range
d (half-width microstrip)	(0, Inf)
a (half-width arrangement)	(0, Inf)
b (height of arrangement)	(0, Inf)
c (height of dielectric)	(0, Inf)
ϵ_{r2} (dielectric constant)	(0, Inf)

Table 5.3: Observation Space

From the previous methods, we understand that randomly generating a shape and waiting for the agent to learn to provide a monotone, decreasing, and convex shape (necessary conditions) is a problem. According to Thomson’s theorem, the actual shape optimisation we seek is for a monotone decreasing convex function that gives minimum energy. Hence, we shall optimize an already monotone decreasing and convex curve.

One of such shapes is the e^{-x} decaying-exponential function normalized to be within the (0, 1) range. The function to generate the g points is defined by equation 5.1. The function is shifted to the right, beginning at the end of the microstrip, and the action value gives a compression factor. This compression factor shall determine the shape of the g-points.

$$f(x) = e^{-(x-d)/\hat{A}}, \quad \text{for } d < x < a \quad (5.1)$$

Since we require only one value to give the compression factor, the action space will now be an array of size 1. With this, we have significantly reduced the model’s

complexity and can expect much faster and more meaningful learning.

Action	Type	Size	Value Range
compression factor	continuous-space array	1	(0,1)

Table 5.4: Action for Method 2

Unlike Method 1, the necessary conditions will be satisfied by default. Hence, the reward scheme only focuses on obtaining the minimum energy shape. The reward scheme is given by algorithm 7.

Algorithm 7 Reward Scheme for Method 2

```

1:  $W_{minimum} = \inf$  ▷ global minimum energy
2:  $x - coordinates - gpts_{prediction} \leftarrow$  generate points in the range( $d, a$ )
3:  $y - coordinates - gpts_{prediction} \leftarrow$  decay-exponential function ▷ Action from agent
4:  $rewards \leftarrow 0$ 
5: if  $\hat{A} \neq 0$  then
6:    $W_{prediction} \leftarrow energy(gpts_{prediction})$ 
7:    $reward \leftarrow reward + \frac{1}{W_{prediction}} \times 10^{-6}$ 
8:   if  $W_{prediction} < W_{minimum}$  then
9:      $W_{minimum} \leftarrow W_{prediction}$ 
10:     $reward \leftarrow reward + \frac{1}{W_{prediction}} \times 10^{-6}$ 
11:   end if
12: else
13:    $reward \leftarrow 0$ 
14: end if

```

Due to the usage of continuous action space with a model-less approach, we shall proceed with the PPO algorithm with an MLP policy. The training was done with environment parameters as shown in table 5.5 and training parameters as shown in table 5.6.

Parameters	Value
V_0	1 V
b	2×10^{-3} m
c	0.79×10^{-3} m
d	0.2×10^{-3} m
a	2×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	2.2

Table 5.5: Test Case 1: Method 2 Environment Parameters

Curve	Num Points	Num Fourier Coeffs
training	53	3000
prediction	53	3000
nominal	53	3000

Table 5.6: Training Parameters

In Figure 5.4, the final prediction curve looks promising, and from the energy value comparison from table 5.7, it can be observed that the prediction energy is quite close to that of the nominal curve. From Figure 5.5, we can see that the rewards have not converged even after more than fifty hours of training. This could be attributed to the slow training due to a large value for the number of fourier coefficients used in calculating the energy.

As the number of fourier coefficients increases, the calculation time increases, thereby increasing the total training time. From Figure 5.6, we can observe that the losses have also not been settled. The decaying exponential function method solves the problem of satisfying the necessary conditions, but it comes with the cost of having the flexibility to mould to the required shape. The flexibility is heavily limited since the exponential function always has a base fixed shape.

From the trial of method 2, we have learned two main things. First, using a higher number of Fourier coefficients will slow down the training, and second, another shape-drawing approach with more flexibility is required. Even though a considerably close value for minimum energy was obtained with this approach, it still does

not have the potential to be close to the nominal curve.

Curve	Energy
Prediction	$1.9623154541462161 \times 10^{-11}$ VAs
Nominal	$1.8531215737851701 \times 10^{-11}$ VAs

Table 5.7: Prediction Energy Values Method 2

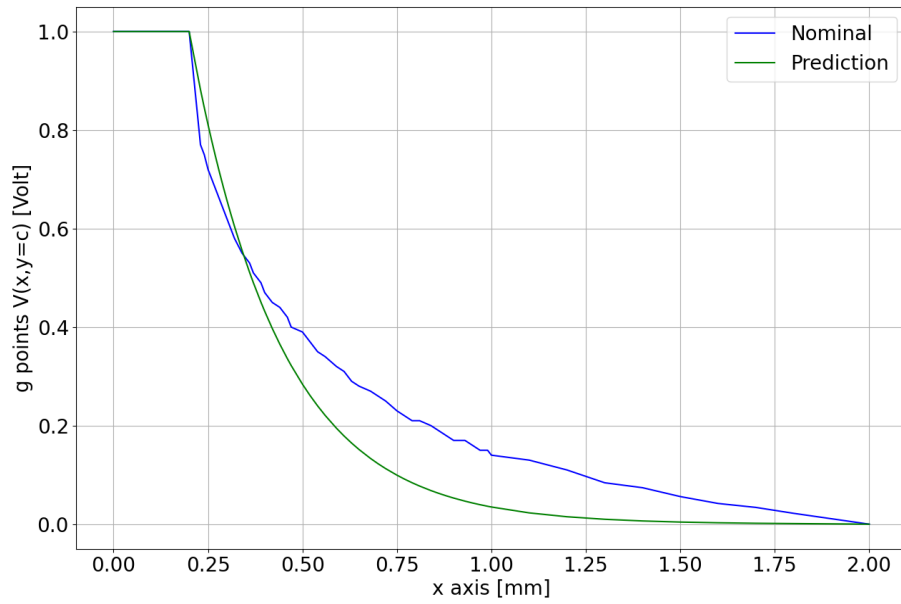


Figure 5.4: Method 2 G-Points

5.3 Method 3: Bézier Curves, Threshold Reward

We shall proceed with the Degenerate RL approach discussed in Method 2 and use the same observation space. The observation space will be as shown in table 5.3, which would take the values given in tables 5.8 and 5.9 for the respective training runs.

As mentioned in Chapter 2, we need to find the g-points to find the piece-wise linear functions that approximate the shape of the potential distribution. Since the

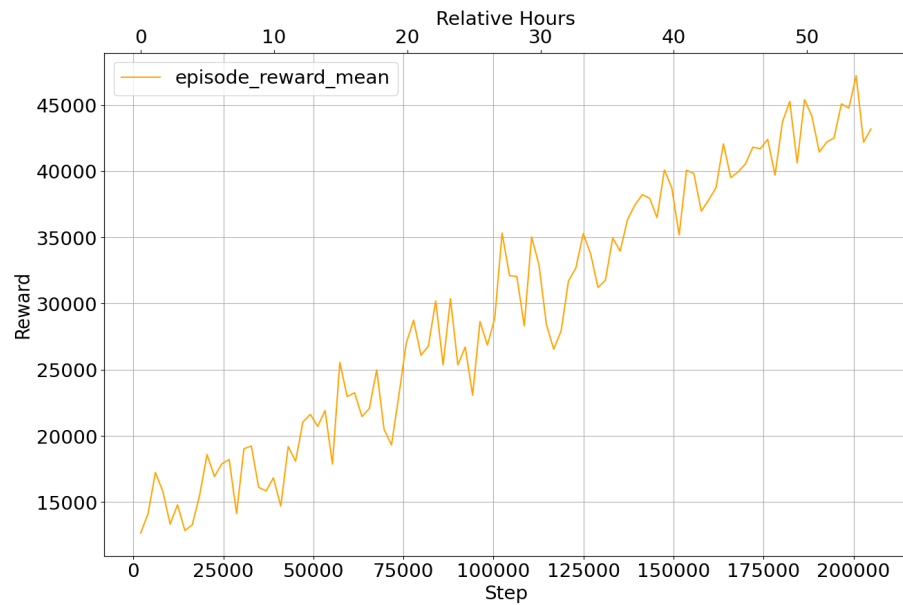


Figure 5.5: Episode Reward Mean for Method 2 [5]

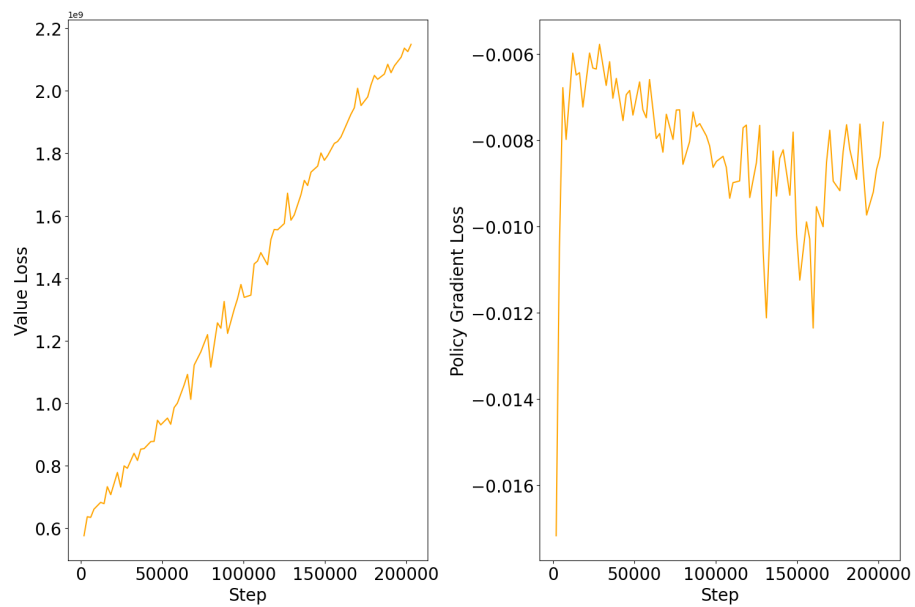


Figure 5.6: Value Loss(Left), Policy Gradient Loss(Right) for Method 2 [5]

Parameters	Value
V_0	1 V
b	2.76×10^{-3} m
c	0.1382×10^{-3} m
d	0.05×10^{-3} m
a	1.38×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 5.8: Test Case 2.1 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	2.76×10^{-3} m
c	0.1382×10^{-3} m
d	0.05×10^{-3} m
a	1.38×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 5.9: Test Case 2.1 Case L Environment Parameters

g-points could be any value between the range $[0, 1]$, we require continuous action space for the DRL algorithm. From [11], we discover that PPO, A2C, DDPG, TD3 and SAC are DRL algorithms that can work with continuous-space actions.

If we define the action space as a 1-D array of size n between the range $[0, 1]$, then the only option for shape drawing is to randomly scatter points within that range. This seems intuitive at first glance, as we do not have any hard control over how the shape is drawn, and we require the RL agent to Figure everything out. As discussed in Method 1, this approach leads to multiple complications, namely the explosion of training time, since this approach is heavily random. Even the necessary conditions for the shape, monotone decreasing, and convexity are not easily satisfied. The agent would have to learn to get the g-points to satisfy the necessary conditions first and then check for minimum energy shape within the constraints of satisfying those conditions. This increases the environment's complexity, leading to the agent not learning anything with a simple reward system.

The next option is a deterministic approach to shape drawing, which is given by some control points. There are various approaches to shape drawing by control points; one such approach that was discussed in [29] is the Bézier curve.

5.3.1 Bézier Curve

Bézier curves are parametric curves in which the parameter t ranges from 0 to 1. A collection of control points determines them, and the positions of these points relative to each other dictate the overall form of the curve [6]. The more control points, the greater the control over the shape of the curve, as shown in Figure 5.7 and 5.8.

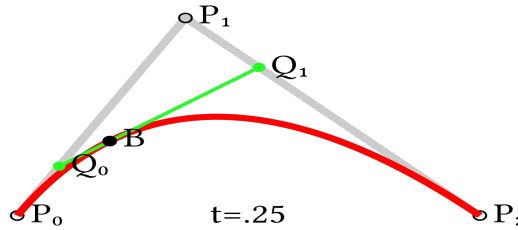


Figure 5.7: Quadratic Bézier: Three control points[6]

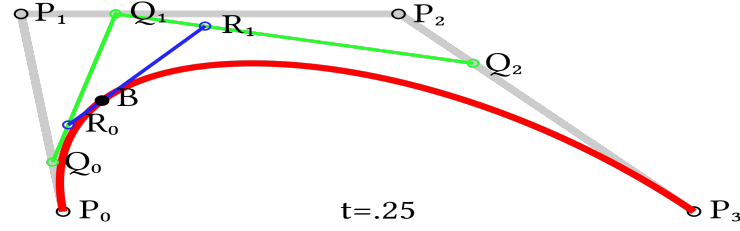


Figure 5.8: Cubic Bézier: Four control points[6]

5.3.2 Control Points

Since the potential curve is not necessarily quadratic, moving forward with the cubic Bézier curve is better. Higher-order Bézier curves will give more control over the shape but would also introduce more complexity to the system. The cubic Bézier curve has four control points P_0 , P_1 , P_2 and P_3 , along with parameter t in the range $[0, 1]$. It then follows equation 5.2 to get the curve's points.

$$B(t) = (1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3 \quad (5.2)$$

Even though we have four control points for the cubic Bézier curve, only two are flexible, and the other two are anchor points. Hence, the action that the agent should predict is a 1-D array of size four within $(0, 1)$ as shown in table 5.10, which would then give the control points as shown in the table 5.11.

Action	Type	Size	Value Range
control points	continuous-space array	4	$(0,1)$

Table 5.10: Action for Method 3

The algorithm 8 represents the Bézier curve drawing approach.

5.3.3 Reward Scheme

With the Bézier Curve, we have the flexibility to draw a shape with minimum control points. Since the set range for the control points is $(0, 1)$, the resulting Bézier Curve

Control Points	Value
P_0	$(d, 1)$
P_1	$(d + action[0](a - d), action[1])$
P_2	$(d + action[2](a - d), action[3])$
P_3	$(a, 0)$

Table 5.11: Bézier Control Points

Algorithm 8 Cubic Bézier curve

```

1:  $P_0 \leftarrow (d, 1)$ 
2:  $P_1 \leftarrow (d + action[0](a - d), action[1])$ 
3:  $P_2 \leftarrow (d + action[2](a - d), action[3])$ 
4:  $P_3 \leftarrow (a, 0)$ 
5:  $t \leftarrow range(0, 1, n)$  ▷ n: number of points in range 0 to 1
6:  $B(t) \leftarrow (1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3$ 

```

is not necessarily a monotone decreasing curve. We shall introduce a threshold parameter into the reward scheme to reward curves that decrease instead of increase. Monotonicity should be checked and rewarded afterwards, which would guarantee convexity. Being within the threshold area would help the training as guiding the g-points towards a monotone decrease condition is simpler. The threshold line can be a very simple line segment connecting the points $(d, 1)$ and $(a, 0)$ as shown in Figure 5.9.

The idea is to take the mean of the difference between the generated g-points and the threshold line. The reward value shall gradually grow as more g-points get below the threshold line. The reward scheme is as shown in algorithm 9.

5.3.4 Training Parameters

In previous methods, we observed that when the number of Fourier coefficients and g-points increases, the training time increases since these parameters directly affect the calculation. Hence, it is important for us to reduce this amount as much as possible. Experiments conducted by M.Sc. Emre Ecik found that the energy value for a non-nominal curve was less than that of the nominal in certain combinations

Algorithm 9 Reward Scheme Method 3

```

1:  $W_{minimum} = \inf$  ▷ global minimum energy
2: while train do
3:    $reward \leftarrow 0$ 
4:   if  $gpts_{prediction} < threshold$  then
5:     if  $gpts_{prediction}$  is monotone decreasing then
6:        $W_{prediction} \leftarrow energy(gpts_{prediction})$ 
7:        $reward \leftarrow reward + \frac{1}{W_{prediction}} \times 10^{-5}$ 
8:       if  $W_{prediction} < W_{minimum}$  then
9:          $W_{minimum} \leftarrow W_{prediction}$ 
10:         $reward \leftarrow reward + \frac{1}{W_{prediction}} \times 10^{-5}$ 
11:      end if
12:    else
13:       $reward \leftarrow reward + 0$ 
14:    end if
15:  else
16:     $reward \leftarrow reward + mean(threshold - gpts_{prediction})$ 
17:  end if
18: end while

```

Curve	Num Points	Num Fourier Coeffs
training	10	20
prediction	53	1500
nominal	53	1500

Table 5.12: Training Parameters

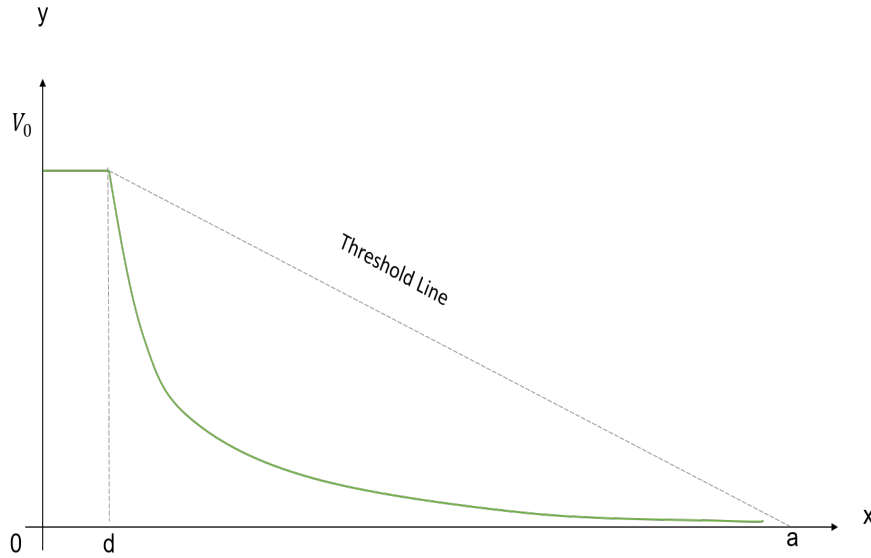


Figure 5.9: Threshold Line For Rewards

of Fourier coefficients and the number of g-points [7].

The experiment was conducted with environment parameters as in table 5.18, and the results are in Figure 5.10. In this Figure, the above-mentioned problem can be seen around fifty-three g-points and five hundred Fourier coefficients. Based on these results, we can use a minimum number of points during training to speed up the process; this is fine as we do not consider the nominal values during training. We shall use more points and Fourier coefficients for better accuracy for testing. The parameters used for training and testing are given in table 5.12.

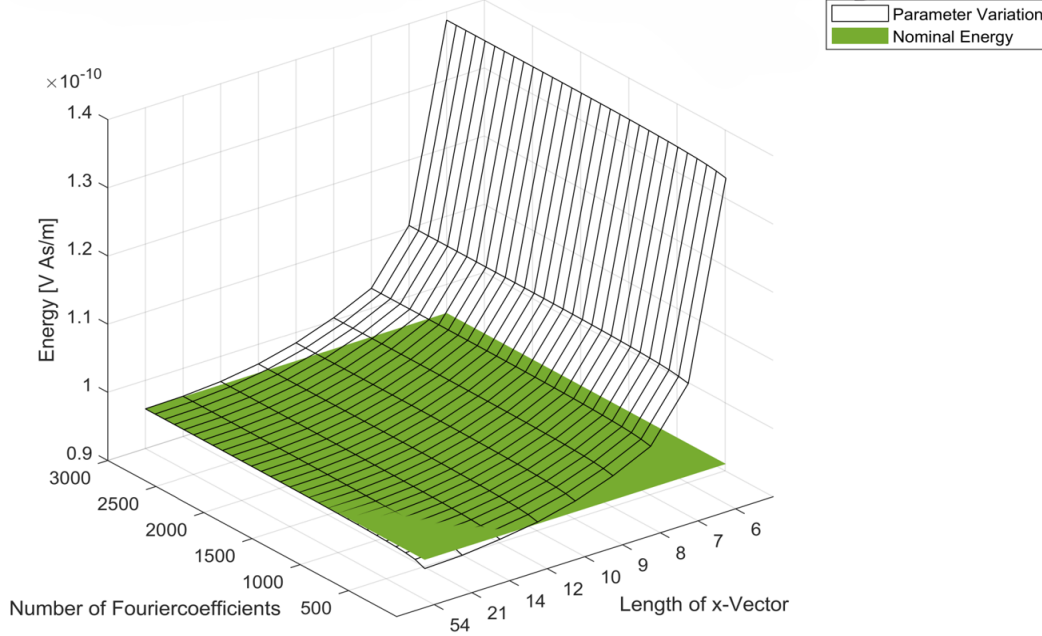


Figure 5.10: Experiment Results from M.Sc. Emre Ecik [7]

5.3.5 Learning Policy and Training

Unlike the previous methods, where we use a PPO agent, we shall use an SAC agent. As outlined in Chapter 2, numerous RL algorithms offer the capability to handle continuous space actions, each with its own advantages and drawbacks. SAC stands out notably for its sample efficiency and enhanced flexibility in exploration[18].

Since SAC is a stochastic policy, it necessitates a symmetric action space. This ensures that the model predicts positive and negative output values. Consequently, when providing inputs to the Cubic Bézier curve function, it's essential to consider the absolute value of the predicted actions.

The model was trained for a hundred thousand timesteps with training parameters as shown in table 5.12 and environment parameters in table 5.18.

The result after training is shown in Figure 5.11. This resulted from the predicted action value as in table 5.13, which would then make up the control points according to table 5.11. The energy values for the predicted curve and nominal curve are also given in table 5.13.

From Figure 5.11, we can see that the model converged super fast. The same can be seen from the Figure 5.13. This can be credited to the low entropy value used during SAC hyperparameter tuning. A low entropy value is used for stability, which will help the model converge faster. Along with this, the reward scheme gives very high rewards as it takes the inverse of the calculated energy values, and the chances for the model to have proper gradients when taking batches from the replay buffer, where most are alike, is very low.

Action Value	Case D Prediction Energy	Case D Nominal Energy
[0,0,0,0]	$1.025799249366519 \times 10^{-10}$ VAs	$9.572519409401707 \times 10^{-11}$ VAs

Table 5.13: Predicted Action for Method 3 and Resulting Energy

Using the same action value [0, 0, 0, 0] as shown in table 5.13, we shall predict for Case L (table 5.19). The results are shown in tables 5.14 - 5.17.

Case	Nominal	Prediction
Case D	$9.572519409401707 \times 10^{-11}$ VAs	$1.025799249366519 \times 10^{-10}$ VAs
Case L	$1.1549945548531716 \times 10^{-11}$ VAs	$1.3102336648788825 \times 10^{-11}$ VAs

Table 5.14: Prediction Energy Values for Case D and Case L with Method 3

Case	Nominal	Prediction
Case D	$1.9145038818803414 \times 10^{-10}$ F	$2.051598498733038 \times 10^{-10}$ F
Case L	$2.3099891097063433 \times 10^{-11}$ F	$2.620467329757765 \times 10^{-11}$ F

Table 5.15: Capacitance for Case D and Case L with Method 3

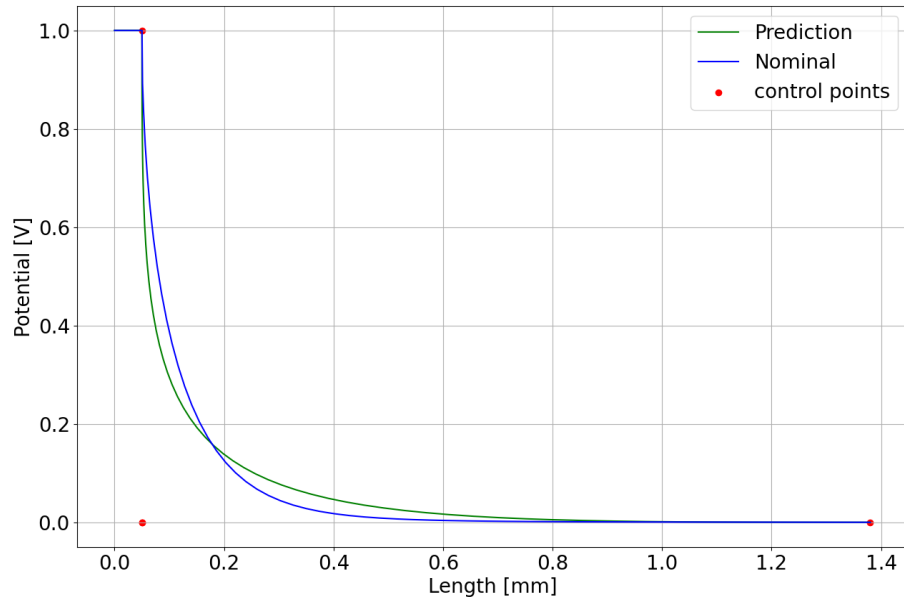


Figure 5.11: Method 3 Case D G-Points

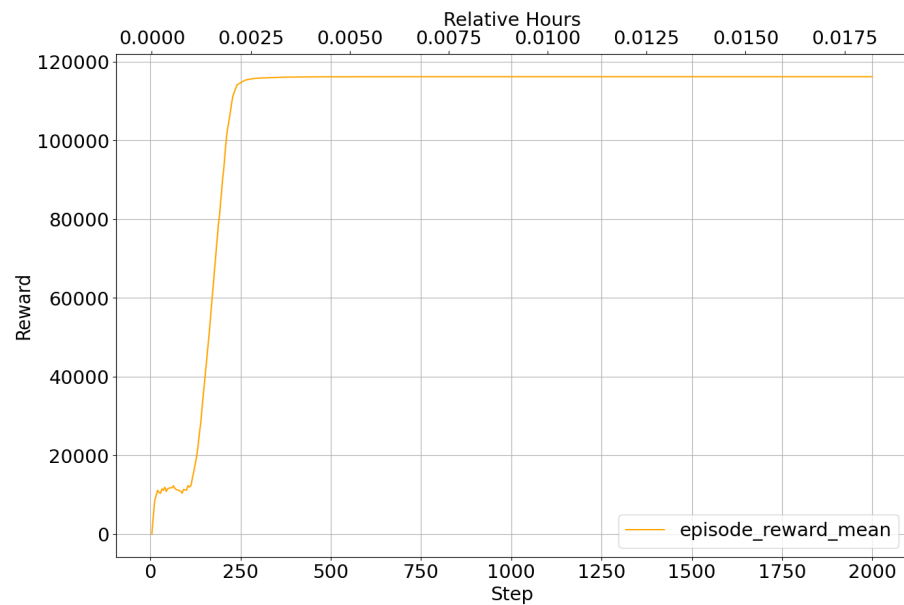


Figure 5.12: Episode Reward Mean for Method 3 Case D [5]

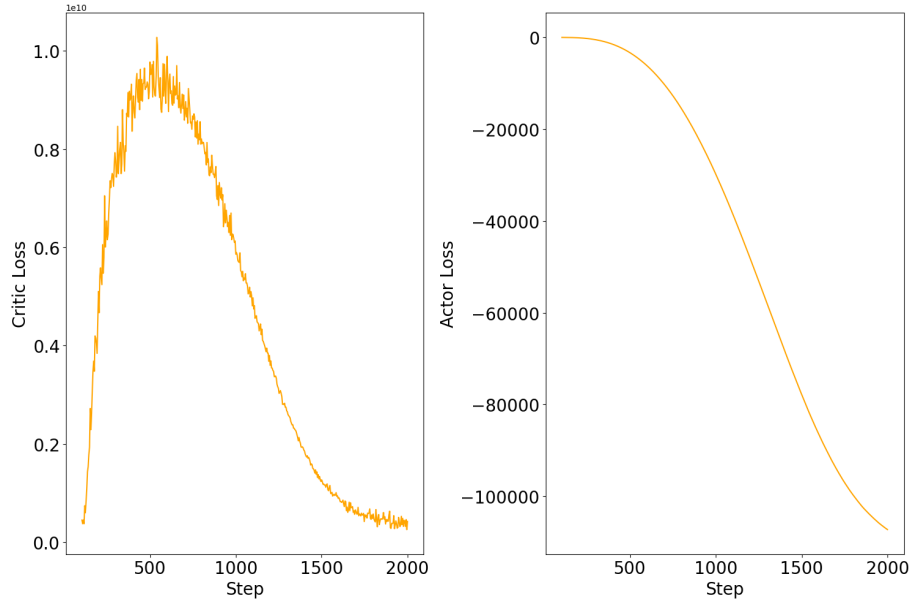


Figure 5.13: Critic Loss(Left), Actor Loss(Right) for Method 3 Case D [5]

Case	Nominal	Prediction
Case D	50.15869228243089 Ω	45.4929119044289 Ω
Case L	144.40072197593878 Ω	127.29183508995955 Ω

Table 5.16: Characteristic Impedance for Case D and Case L with Method 3

Attribute	Nominal	Prediction
ϵ_{eff}	8.287934665301181	7.8291321377499745
v_{ph}	104135197.135103 m/s	107143021.95241089 m/s

Table 5.17: Phase Velocity and Effective Dielectric Constant with Method 3

5.4 Method 4: Bézier Curves, Scaled Reward

Although the results of Method 3 look promising, it still can be tuned further. For this, we shall make changes according to the inferences gathered from all the previous methods. The training environment parameters are the same as Method 3, as shown in tables 5.18 and 5.19.

From Method 3, we understood three main things. First, to achieve the necessary conditions by default, we need to limit the search space for the action. Through experimentation, it was determined that limiting the search space to the limits $(0, 0.2)$ had the best results. Therefore, the action space is a 1-D array of size four within the range $(0, 0.2)$ that gives the (x, y) coordinates of the control points as shown in table 5.11. Second, the hyperparameter entropy coefficient should be varied during training rather than a fixed value to promote more exploration. Third, the reward scheme should be scaled to avoid getting stuck at local minima.

Parameters	Value
V_0	1 V
b	2.76×10^{-3} m
c	0.1382×10^{-3} m
d	0.05×10^{-3} m
a	1.38×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 5.18: Test Case 2.1 Case D Environment Parameters

5.4.1 Reward

Since reinforcement learning does not have labels to compare the outputs, rewards determine the model's accuracy. However, there are no direct approaches to how a reward function should be, making it one of the biggest challenges in any RL problem. We shall look at some general guidelines for reward shaping.

Parameters	Value
V_0	1 V
b	2.76×10^{-3} m
c	0.1382×10^{-3} m
d	0.05×10^{-3} m
a	1.38×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 5.19: Test Case 2.1 Case L Environment Parameters

Guidelines

One main idea for the reward is that it should represent the goal of the task. This aids the agent in directing towards the desired actions. Reward functions should also strike a balance between readability and simplicity.

Scaling rewards decreases the magnitude of the obtained benefits. This technique can be particularly advantageous when dealing with extensive state or action spaces. By amplifying rewards, the learning process becomes more stable, enabling the agent to focus on environmental elements relevant to its objectives. Hence, it is desirable to scale the rewards; for one, it becomes more readable and stable with slight changes in the action.

The reward scheme should have levels, like a curriculum. This is done so that the agent can begin with smaller tasks and then move on to complex ones. This helps the agent progressively learn instead of overloading with difficult tasks from the beginning [34].

There are many more guidelines and approaches to reward shaping, notably Inverse Reinforcement Learning and Reinforcement Learning with Human Feedback. However, these techniques are not suited to our problem as the optimal or final state is unknown. Based on these guidelines, we shall shape our reward scheme.

Reward Scheme

In our problem, the optimal state is unknown, and Thomson’s minimum energy theorem defines its only attribute. Since we know from the guidelines that the reward should represent the goal of the task, which is minimum energy, taking the inverse of the energy value as the reward is a good choice. Therefore, maximizing the reward would mean minimizing the energy.

Since energy values are of the order 10^{-11} (refer table 5.20), the inverse of these values would be extremely high and small changes would also result in high differences in the reward. The scalability of the reward function is an issue here. A baseline value to compare our predicted energies could help scale the reward and achieve stability during training.

Case	Nominal	Baseline
Case D	$9.572519409401707 \times 10^{-11}$ VAs	$1.025799249366519 \times 10^{-10}$ VAs
Case L	$1.1549945548531716 \times 10^{-11}$ VAs	$1.3102336648788825 \times 10^{-11}$ VAs

Table 5.20: Energy Values

For this, we shall adopt a baseline predicted curve where the action for the cubic Bézier curve is an array like $[0, 0, 0, 0]$, and this would result in control points like in table 5.21. This is the result of Method 3 as shown in table 5.13.

Control Points	Value
P_0	$(d, 1)$
P_1	$(d, 0)$
P_2	$(d, 0)$
P_3	$(a, 0)$

Table 5.21: Control Points: Baseline Potential Curve

The resulting curves from these control points are given in figures 5.14 and 5.15, where the environment parameters for those curves are given in tables 5.18 and 5.19.

These images show that the predicted green colour curve is very close to the nominal blue colour curve, calculated using the formulas from [21]. The energy for each curve was calculated using the equation 2.22. Since the predicted curve is close to the nominal curve, this can be considered as a baseline reference during training. The rewards might be assigned as a ratio between the inverse of the predicted and baseline curves' energies like

$$reward \leftarrow \frac{1/W_{prediction}}{1/W_{baseline}} \times 100 \quad (5.3)$$

As the energy of the predicted curve decreases and approaches that of the baseline energy, the value will rise to 100. If the energy falls below the baseline, the value will exceed 100.

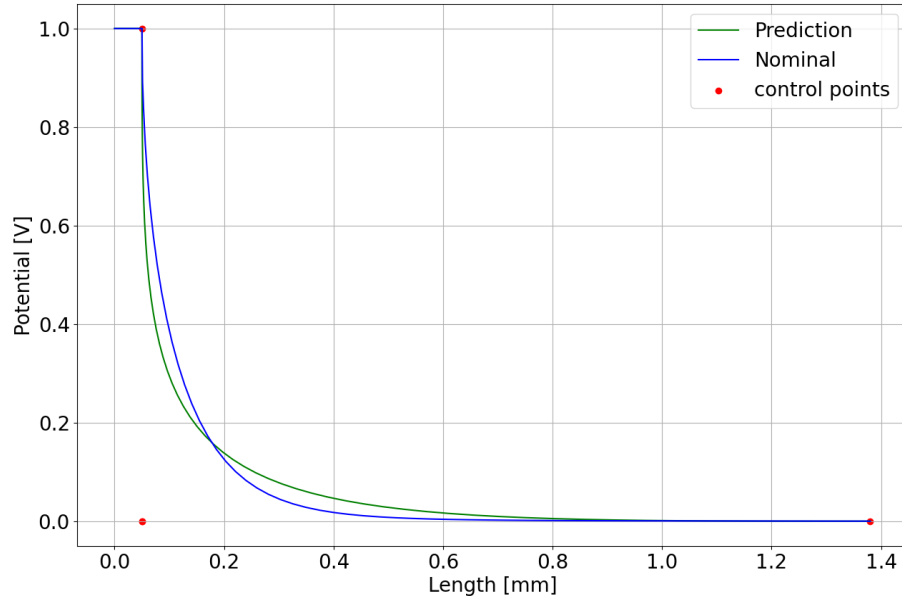


Figure 5.14: Case D Baseline Curve

In addition to the above reward function, we shall add a check for the necessary conditions. This is achieved by counting the number of g-points that satisfy the necessary conditions: the number of monotone-decreasing g-points plus the number of convex g-points. During training, ten points are used for the eight g-points curve,

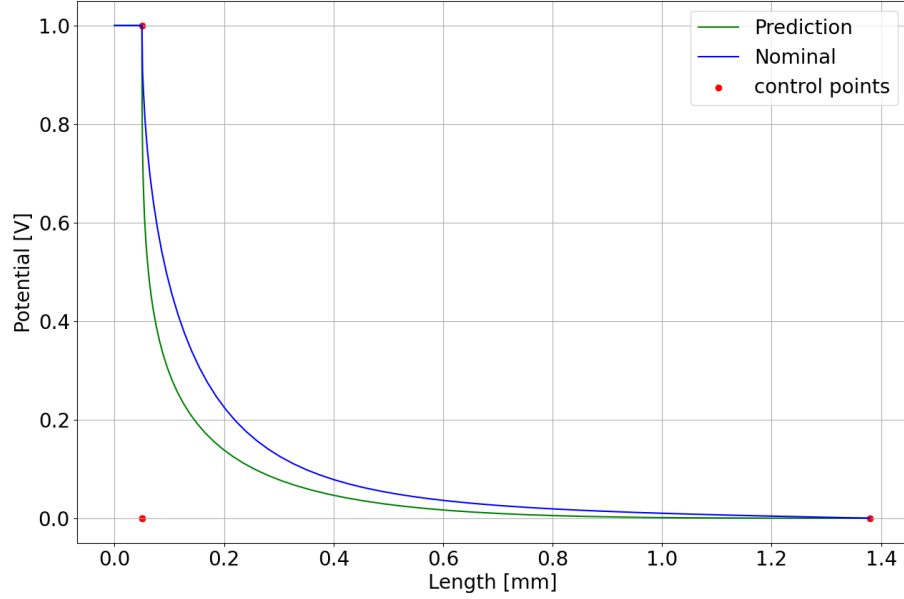


Figure 5.15: Case L Baseline Curve

excluding the anchor points $(d, 1)$ and $(a, 0)$. Hence, the maximum value for the necessary condition check rewards is sixteen. Once both conditions are satisfied, we shall look for the minimum energy state. We can give rewards like a curriculum, as discussed in the guidelines section, with this approach. The reward scheme for training can then be summarized as in the algorithm 10.

5.4.2 Learning Policy and Training

We shall reuse the degenerate RL approach from Method 2 with SAC Policy, as in Method 3. We shall reuse the same Bézier Curve approach from Method 3 with reduced search space for the action space. The action space is given in table 5.22.

According to [11], the SAC implementation features an 'auto' option for the hyperparameter entropy coefficient. This simplifies hyperparameter tuning significantly, as the algorithm dynamically adjusts this parameter to promote exploration. This solves the second concern we had regarding exploration. With the new reward scheme, we shall solve the third concern of being stuck in a local minima.

Algorithm 10 Reward Scheme

```

1:  $gpts_{baseline} \leftarrow CubicBezier([0, 0, 0, 0])$ 
2:  $W_{baseline} \leftarrow energy(gpts_{baseline})$ 
3: if all  $\hat{A} = 0$  then
4:    $reward \leftarrow -100$  ▷ Penalize zero action
5:   break
6: end if
7: if any  $\hat{A} = 0.2$  then
8:    $reward \leftarrow -100$  ▷ Penalize boundary actions
9:   break
10: end if
11: if  $gpts$  is monotone decreasing then
12:   if  $gpts$  is convex then
13:      $W_{prediction} \leftarrow energy(gpts_{prediction})$ 
14:      $reward \leftarrow 20 + \frac{1/W_{prediction}}{1/W_{baseline}} \times 100$ 
15:   else
16:      $reward \leftarrow degree_{monotone} + degree_{convex}$  ▷ degree is the number of points
    satisfying that condition
17:   end if
18: else
19:    $reward \leftarrow degree_{monotone}$ 
20: end if

```

Action	Type	Size	Value Range
control points	continuous-space array	4	(0,0.2)

Table 5.22: Action for Method 4

During training, the energy is calculated per equation 2.22. Twenty Fourier coefficients were used, with ten Bézier curve points, including the anchor points. The intermediate predictions used fifty-three Bézier curve points, including the anchor points and one thousand five hundred Fourier coefficients. The parameters are as shown

The model is then trained with SAC policy with environment training parameters as shown in the table 5.12 for fifty thousand timesteps. The parameters for the two environment cases, Case D and Case L are shown in the tables 5.18 and 5.19. The energy values for the nominal and baseline curves are also in the table 5.20.

The training scheme is represented in the algorithm 11.

Algorithm 11 DRL Learning with SAC Policy

```

1:  $gpts_{baseline} \leftarrow CubicBezier([0, 0, 0, 0])$ 
2:  $W_{baseline} \leftarrow energy(gpts_{baseline})$ 
3: for  $timesteps = 1$  to  $timestep_{max}$  do
4:    $observation \leftarrow reset(env)$ 
5:    $\hat{A} \leftarrow model_{SAC}$ 
6:    $gpts \leftarrow CubicBezier(\hat{A})$ 
7:    $reward \leftarrow RewardScheme(W_{baseline}, gpts, \hat{A})$ 
8:    $model_{SAC} \leftarrow updatemodel(observation, reward, \hat{A})$ 
9: end for

```

5.4.3 Training Results

Case D Training Results

After training with the SAC agent in the Case D environment (Table 5.18), the rewards obtained are shown in Figure 5.16. From this, it was observed that the agent was fairly stable and converged around twenty thousand timesteps.

Since the hyperparameter entropy coefficient was set to 'auto', the agent could explore freely, as shown in Figure 5.18. From Figure 5.14, we can observe that the initial baseline curve is very close to the nominal curve; slight deviations from it could get the agent closer to the desired nominal curve. This can also be seen in

how the entropy coefficients did not have to shoot up high for exploration, as the rewards had a fairly upward trend, as shown in Figure 5.16. We can observe the rewards (Figure 5.16), entropy coefficient (Figure 5.18) and policy losses (Figure 5.17) stabilizing around twenty thousand timesteps.

The absolute (since SAC uses symmetric action space $[-1,1]$, absolute of the predicted value is taken) of the predicted action values, which are then fed to the Bézier Curve control as in table 5.11, is given in the table 5.23.

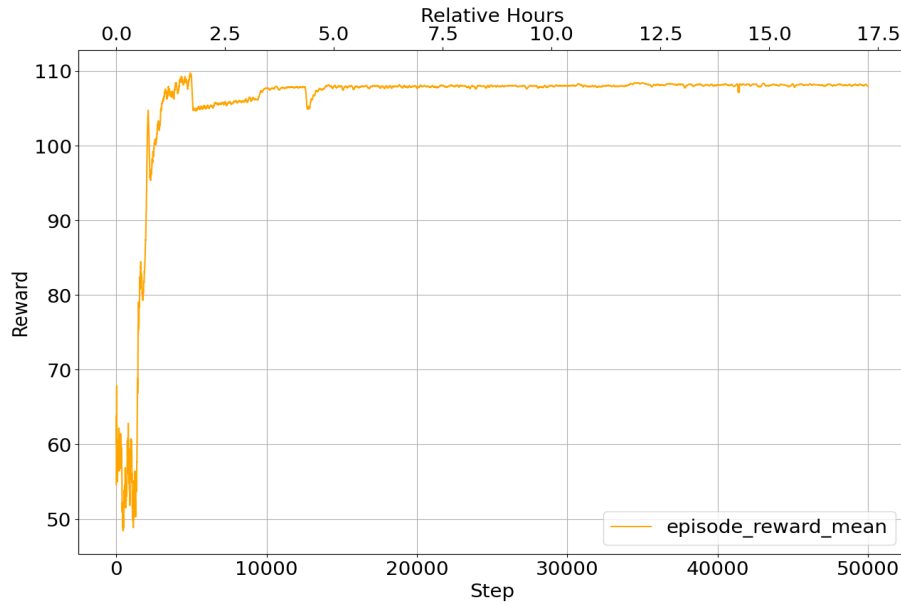


Figure 5.16: Episode Reward Mean for Method 4 Case D [5]

Timestep	Action
20K	[0.00516614, 0.00443451, 0.10257914, 0.00134882]
50K	[0.02586469, 0.00258739, 0.01518556, 0.00065063]

Table 5.23: Action Values for Case D

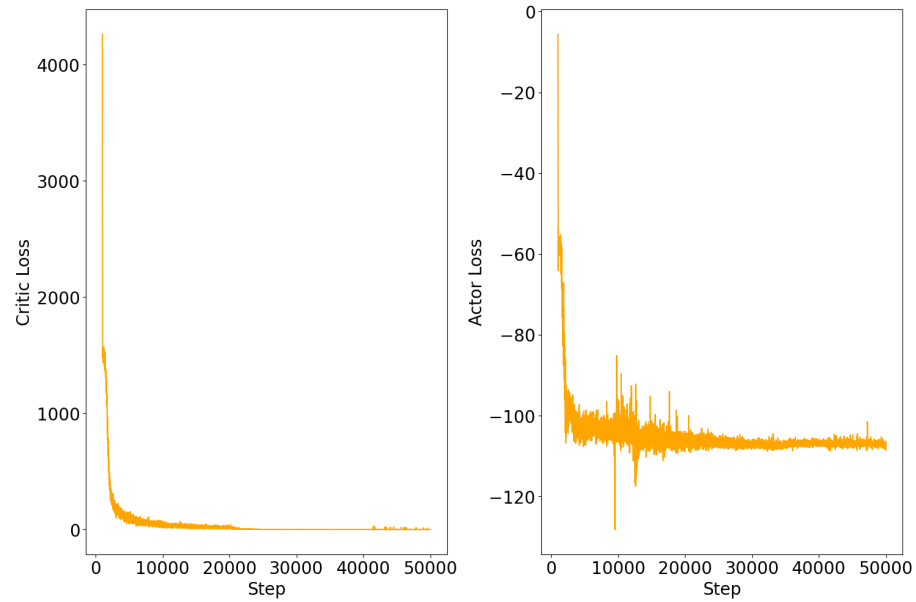


Figure 5.17: Critic Loss(Left), Actor Loss(Right) for Method 4 Case D [5]

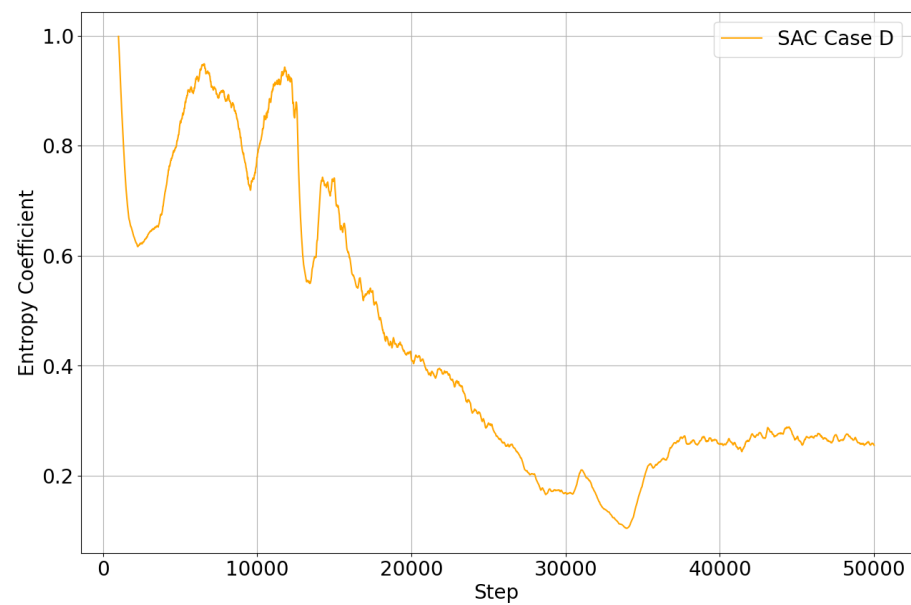


Figure 5.18: Entropy Coefficient for Method 4 Case D [5]

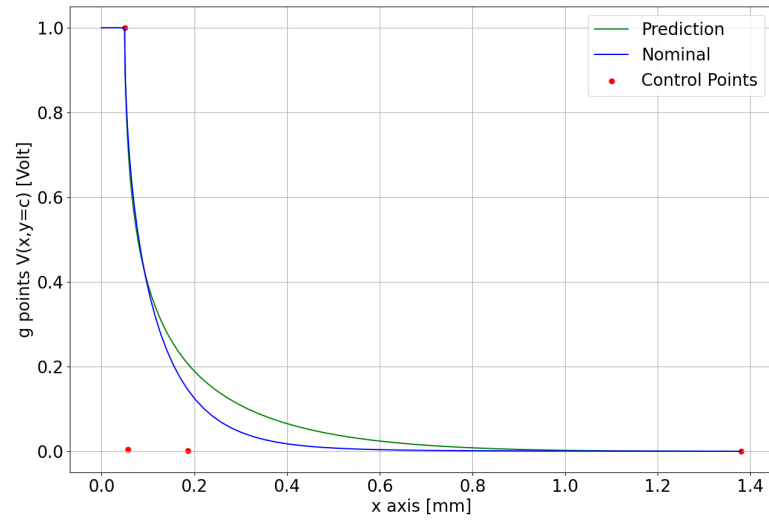


Figure 5.19: G-Points at 20K Timesteps for Case D with SAC Agent

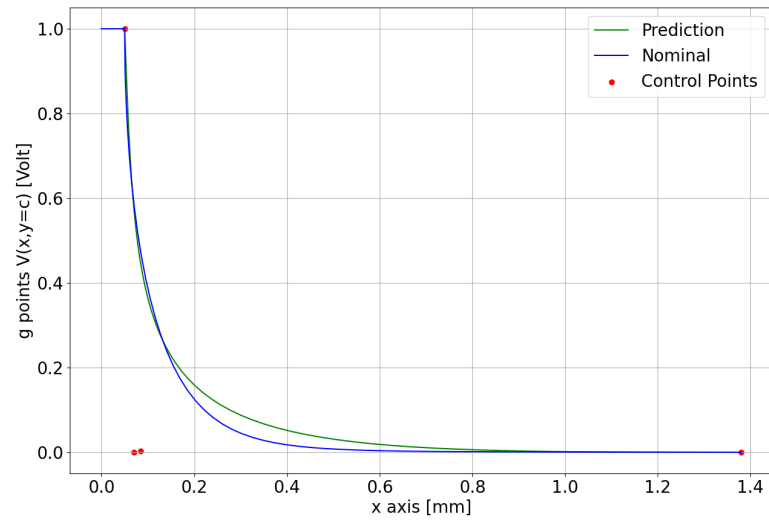


Figure 5.20: G-Points at 50K Timesteps for Case D with SAC Agent

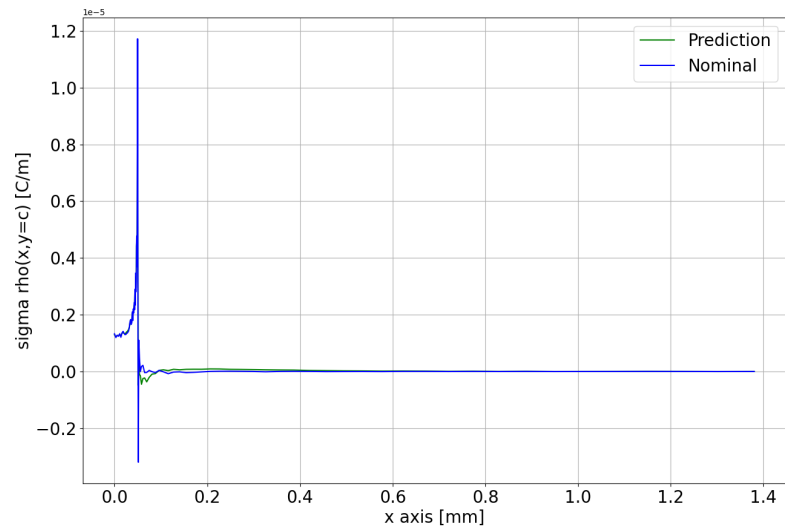


Figure 5.21: Surface Charge Density at 20K Timesteps for Case D with SAC Agent

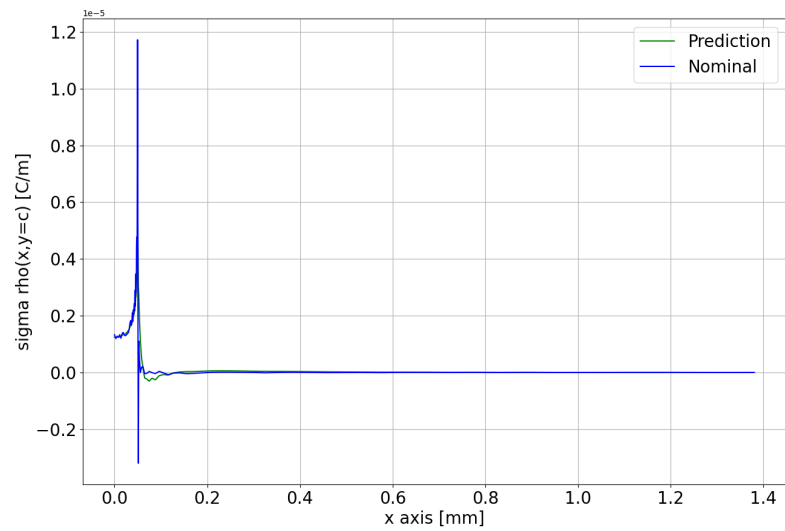


Figure 5.22: Surface Charge Density at 50K Timesteps for Case D with SAC Agent

Case L Training Results

After training with the SAC agent in the Case L environment (Table 5.19), the rewards obtained are shown in Figure 5.23. From this, it was observed that the agent was not fairly stable and converged around forty-five thousand timesteps.

Since the hyperparameter entropy coefficient was set to 'auto', the agent could explore freely, as shown in Figure 5.25. From Figure 5.15, we can observe that the initial baseline curve is not very close to the nominal curve; slight deviations from it could not get the agent closer to the desired nominal curve. This can also be seen in how the entropy coefficients did have to shoot up high for exploration, as the rewards did not have a fairly upward trend, as shown in Figure 5.23. We can observe the rewards (Figure 5.23), entropy coefficient (Figure 5.25) and policy losses (Figure 5.24) stabilizing only around forty-five thousand timesteps.

The absolute (since SAC uses symmetric action space $[-1,1]$, absolute of the predicted value is taken) of the predicted action values, which are then fed to the Bézier Curve control as in table 5.11, is given in the table 5.24.

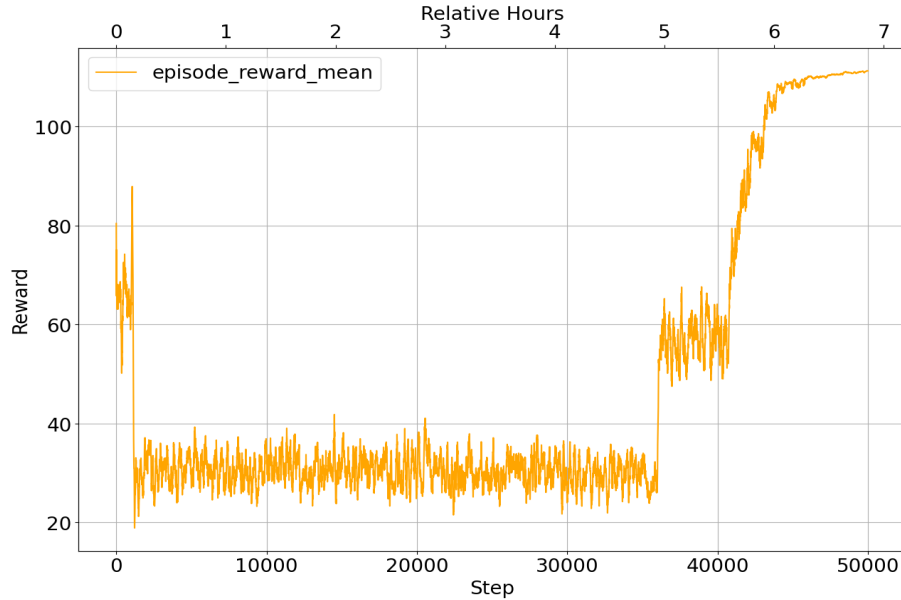


Figure 5.23: Episode Reward Mean for Method 4 Case L [5]

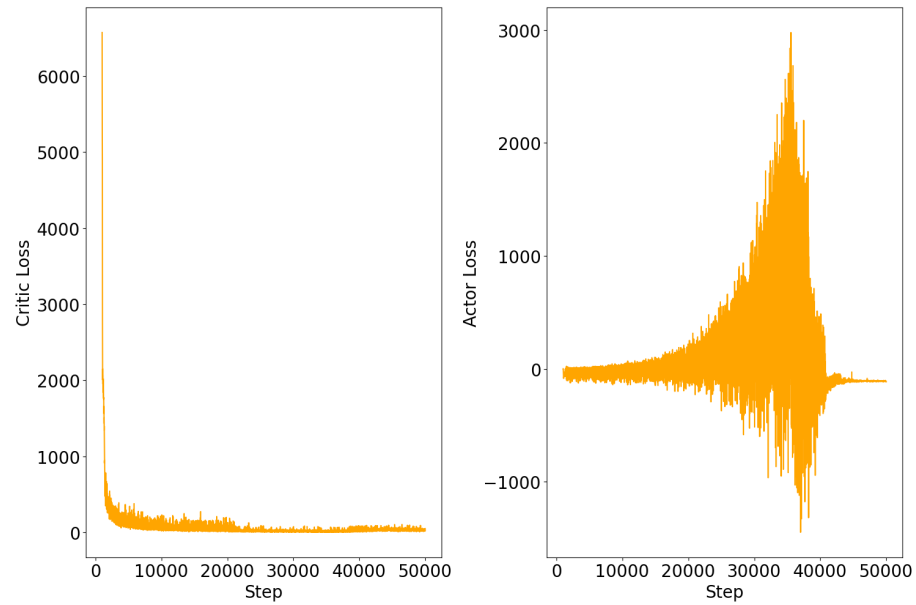


Figure 5.24: Critic Loss(Left), Actor Loss(Right) for Method 4 Case L [5]

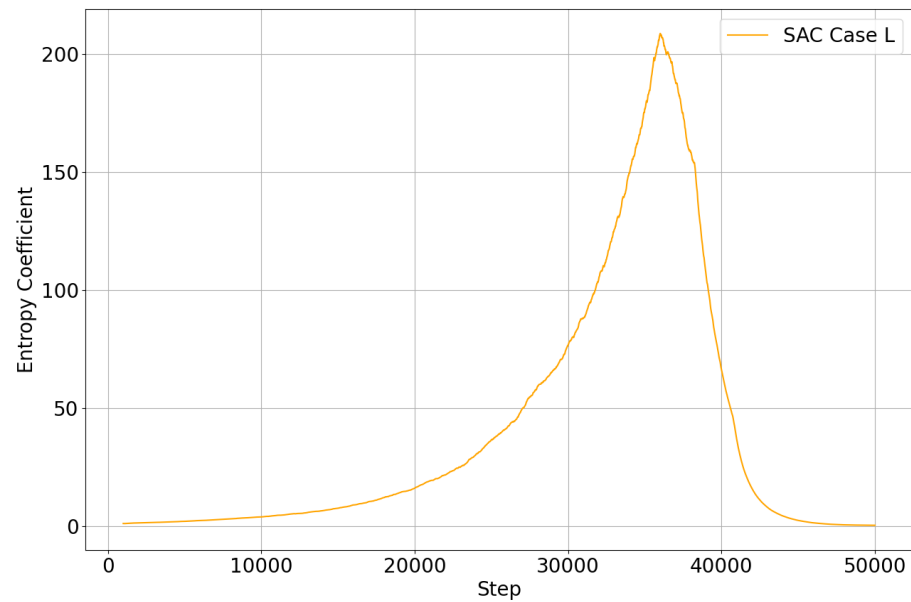


Figure 5.25: Entropy Coefficient for Method 4 Case l [5]

Timestep	Action
45K	[0.02970897, 0.0822546, 0.06288461, 0.04898012]
50K	[0.00155696, 0.01231095, 0.1131704, 0.0128462]

Table 5.24: Action Values for Case L

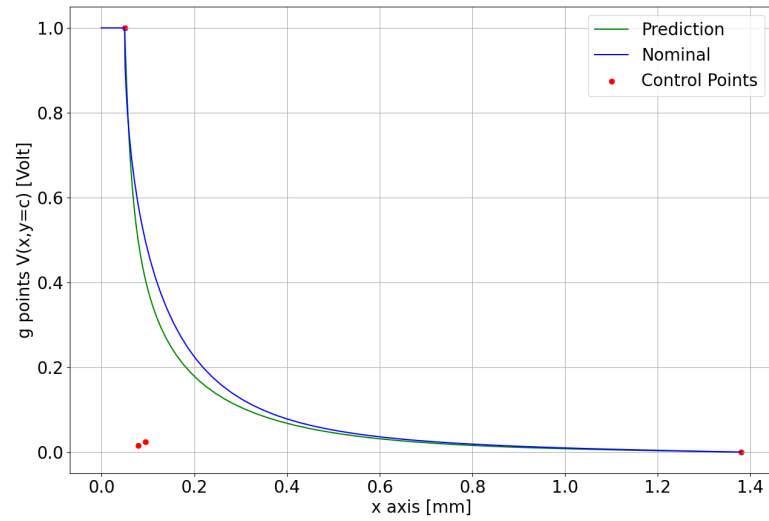


Figure 5.26: G-Points at 45K Timesteps for Case L with SAC Agent

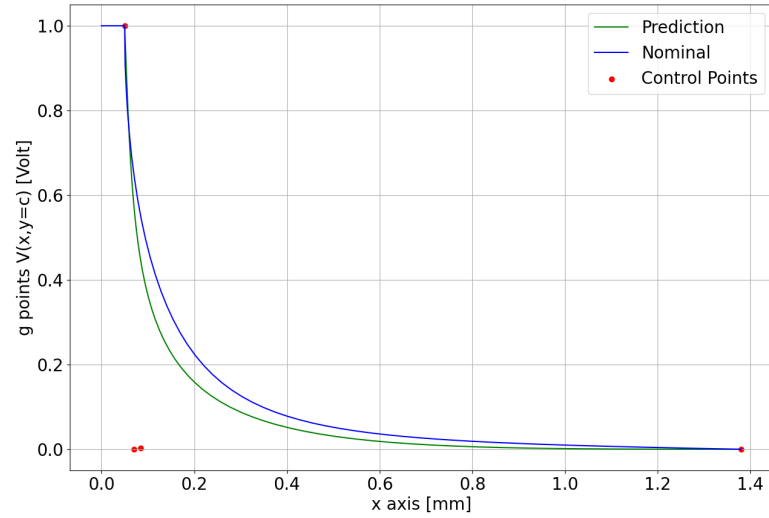


Figure 5.27: G-Points at 50K Timesteps for Case L with SAC Agent

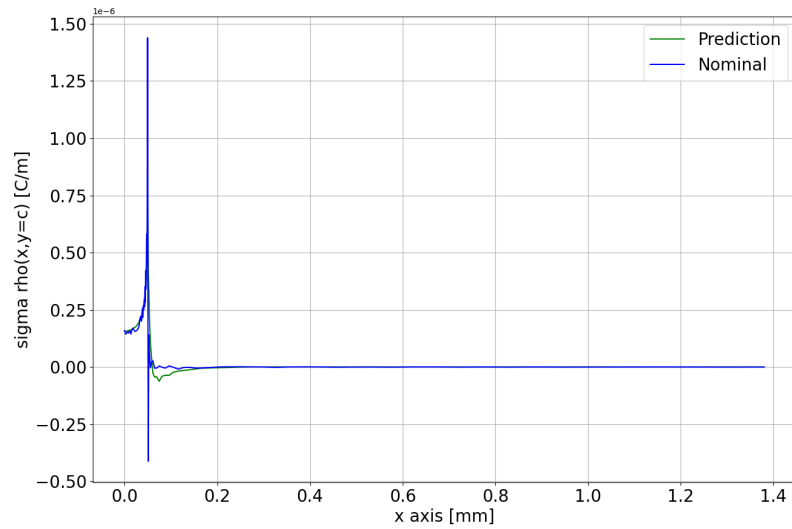


Figure 5.28: Surface Charge Density at 45K Timesteps for Case L with SAC Agent

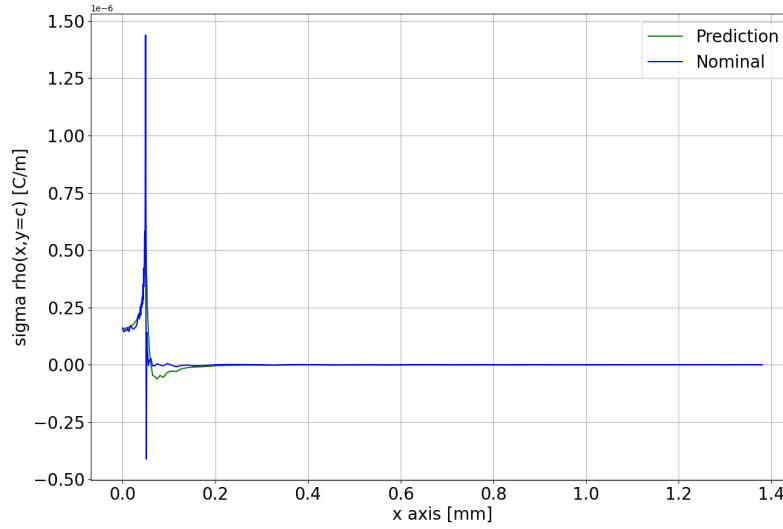


Figure 5.29: Surface Charge Density at 50K Timesteps for Case L with SAC Agent

Observations

Interestingly, the rewards did not exceed the one hundred and twenty for both cases D and L. Since our reward scheme, as in Algorithm 10, is given like a curriculum, where the value up to twenty is to check for the necessary conditions and then from there, it is the percentage of closeness to the initial curve, we expect the reward value to go beyond one hundred and twenty when we obtain an energy value less than the initial curve. From Table 5.25, we can see that the energy values go below that of the initial curve as given in Table 5.20. This seems counterintuitive to the rewards obtained. There are multiple reasons to explain this behaviour, which is common in stochastic policies like SAC.

Stochastic policies like SAC strike a balance between exploration and exploitation by incorporating entropy regularization into their objective function (Figure 5.18 and Figure 5.25). This encourages the agent to explore more of the state space, potentially resulting in lower immediate rewards but promising improved long-term performance. The decrease in mean episode rewards may reflect the consequence of this exploration strategy.

Research suggests that stochastic reinforcement learning techniques like SAC can converge to an optimal policy, even amidst noise and non-Markovian rewards. The temporary decrease in mean episode rewards observed during training could be transient, with the final learned policy potentially achieving higher rewards [35]. These are some possible explanations for the observed behaviour.

In Figure 5.22 and Figure 5.29, we observe the surface charge density of the predicted curves quickly dropping to zero after d (5.18 and 5.19). This is also an indication of good performance, as the conducting surface of the microstrip ends at d ; the charge should also fall to zero from there. Hence, the Surface Charge Density figures show that the predicted potential curve is close to the nominal curve.

With equation 5.4, we can calculate the relative error of the predicted energy compared to the nominal energy. The results are given in Table 5.29. The Characteristic Impedance, Effective Dielectric Constant, and Phase Velocity are all calculated according to equations 2.30 - 2.33. Results of the calculations are given in Table 5.25 - 5.28.

$$RelativeError = \frac{|predictionenergy - nominalenergy|}{|nominalenergy|} \quad (5.4)$$

We are within two percent of error for Case D and four percent for Case L. Case D works much better than Case L; this can be inferred from the rewards obtained for each case. Since the rewards stabilized only after forty-five thousand timesteps due to the initial curve being far from the nominal curve, further training or a shift in the initial curve could potentially result in a better result. The relative error calculated using equation 5.4, of attributes of the predicted curve to that of the nominal curve is given in table 5.29. For comparison of the results, the relative error of all the attributes for Method is also calculated and shown in table 5.30.

The relative error calculations and results show that the predicted curve is very close to the nominal curve. This is the best result from all the previous methods.

Case	Times	Nominal	Prediction
Case D	20K	$9.572519409401707 \times 10^{-11}$ VAs	$9.715020457610047 \times 10^{-11}$ VAs
Case D	50K	$9.572519409401707 \times 10^{-11}$ VAs	$9.750790196290308 \times 10^{-11}$ VAs
Case L	45K	$1.1549945548531716 \times 10^{-11}$ VAs	$1.1834838493990639 \times 10^{-11}$ VAs
Case L	50K	$1.1549945548531716 \times 10^{-11}$ VAs	$1.1971589414856301 \times 10^{-11}$ VAs

Table 5.25: Prediction Energy Values for Case D and Case L with Method 4

Case	Times	Nominal	Prediction
Case D	20K	$1.9145038818803414 \times 10^{-10}$ F	$1.9430040915220094 \times 10^{-10}$ F
Case D	50K	$1.9145038818803414 \times 10^{-10}$ F	$1.9501580392580616 \times 10^{-10}$ F
Case L	45K	$2.3099891097063433 \times 10^{-11}$ F	$2.3669676987981277 \times 10^{-11}$ F
Case L	50K	$2.3099891097063433 \times 10^{-11}$ F	$2.3943178829712602 \times 10^{-11}$ F

Table 5.26: Capacitance for Case D and Case L with Method 4

Case	Timestep	Nominal	Prediction
Case D	20K/45K	50.15869228243089 Ω	49.186540048781225 Ω
Case D	50K	50.15869228243089 Ω	48.815021914217894 Ω
Case L	20K/45K	144.40072197593878 Ω	140.92465028885923 Ω
Case L	50K	144.40072197593878 Ω	139.3148744243648 Ω

Table 5.27: Characteristic Impedance for Case D and Case L with Method 4

Attribute	Timestep	Nominal	Prediction
ϵ_{eff}	20K/45K	8.287934665301181	8.2088323068735
ϵ_{eff}	50K	8.287934665301181	8.144942044362077
v_{ph}	20K/45K	104135197.135103 m/s	104635730.59443161 m/s
v_{ph}	50K	104135197.135103 m/s	105045318.86817566 m/s

Table 5.28: Phase Velocity and Effective Dielectric Constant with Method 4

Attribute	Case	Timestep	Relative Error
Energy	Case D	20K	1.4886472632103678%
Energy	Case D	50K	1.8623183643118164%
Energy	Case L	45K	2.4666172170408132%
Energy	Case L	50K	3.650613455733443%
Capacitance	Case D	20K	1.4886472632103678%
Capacitance	Case D	50K	1.8623183643118164%
Capacitance	Case L	45K	2.466617217040813%
Capacitance	Case L	50K	3.650613455733443%
Impedance	Case D	20K/45K	0.954427871624716%
Impedance	Case D	50K	2.678838516457187%
Impedance	Case L	20K/45K	2.4072398250604072%
Impedance	Case L	50K	3.522037481517178%
ϵ_{eff}	Case D/L	20K/45K	0.954427871624716%
ϵ_{eff}	Case D/L	50K	1.725310668021631%
v_{ph}	Case D/L	20K/45K	0.48065733114157766%
v_{ph}	Case D/L	50K	0.8739808999371067%

Table 5.29: Attribute Relative Error for Case D and Case L with Method 4

Attribute	Case	Relative Error
Energy	Case D	7.160842981318388%
Energy	Case L	13.44067895155104%
Capacitance	Case D	7.160842981318388%
Capacitance	Case L	13.44067895155104%
Impedance	Case D	9.302037524682987%
Impedance	Case L	11.848200377301474%
ϵ_{eff}	Case D/L	5.5357884211136446%
v_{ph}	Case D/L	2.8883844272226185%

Table 5.30: Attribute Relative Error for Case D and Case L with Method 3

Agent Comparison

We shall compare the training rewards of multiple agents(SAC, TD3, PPO) with the same method 4. The rewards are shown in figures 5.30 and 5.31.

SAC is the clear winner in case D, but all agents perform relatively well in case L. It can also be seen that TD3 converges faster than SAC. However, it is important to remember that during hyperparameter tuning, the entropy coefficient for SAC was set as auto, which causes the agent to explore more instead of converging faster. Hence, it cannot be attributed to the fact that TD3 performed better than SAC simply by looking at the convergence rate. Lack of exploration is also why TD3 performed poorly in case D.

To further explain why lack of exploration greatly impacts performance, we should analyse the reward scheme for Method 4 (algorithm 10). Since the reward scheme is based on the percentage of the current curve's energy compared to the base curve, without ample exploration, the agent will be stuck at a local solution instead of a global one. In addition, the reward scheme penalizes action values that are simply zero (no change from that of the base curve). The lack of exploration combined with the penalisation, which is again included to promote exploration, will result in the agent not learning anything.

This is precisely why we can see a poor performance for PPO and TD3 in case D but a decent performance in case L. This happens as the case D base curve is already very close to that of the nominal curve, as shown in Figure 5.14 when compared to the base curve for case L, as shown in Figure 5.15. Hence, without exploration, the case D agent is expected to be stuck at a local solution. For case L, the 'auto' parameter for exploration made it explore heavily, which could be attributed to why the SAC converged slower than TD3.

This could be optimised by further hyperparameter tuning to obtain the right value for the entropy coefficient parameter. However, such tuning does not guarantee that the same values will work for other environment parameters applied during training. Hence, a general approach for training which works for different cases is preferred.

Hence, by comparing the different agents for the same method (Method 4), we can confirm that SAC is the more stable and reliable RL algorithm for this particular problem.

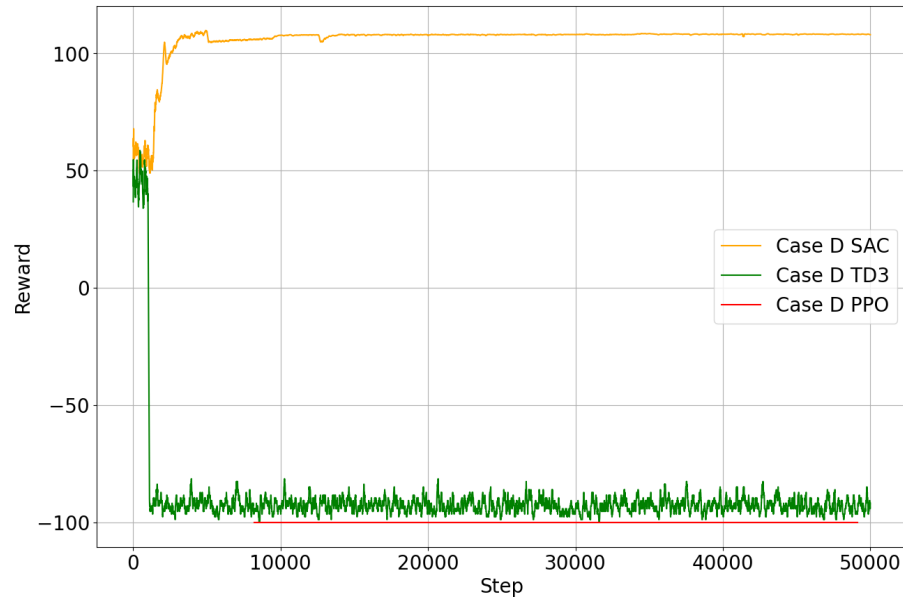


Figure 5.30: Multiple Agents Episode Reward Mean for Method 4 Case D [5]

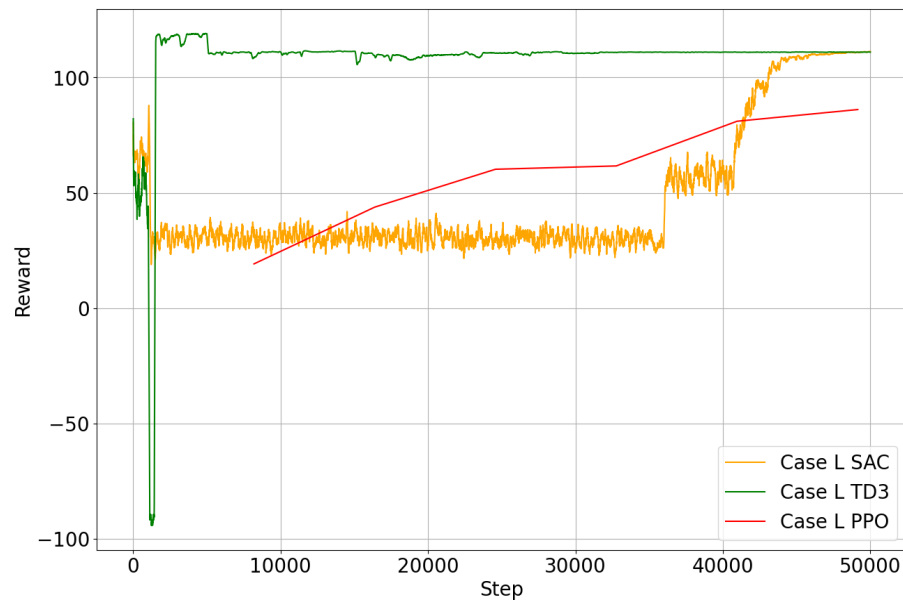


Figure 5.31: Multiple Agents Episode Reward Mean for Method 4 Case L [5]

Model Comparison

We have successfully trained two separate RL models, each for Case L and Case D, with environment parameters as in tables 5.18 and 5.19. To observe the models' capacity, we shall alternate the models with the cases. The results obtained after predicting Case L using the Case D model and Case D using the Case L model are given in table 5.31 and figures 5.32 and 5.33.

It is observed that the Case D model performs relatively well for Case L while the Case L model for Case D is poor. This can be attributed to the more flexibility the Case D curve has over the Case L curve. The Case L curve is relatively flatter than the Case D curve. This behaviour is expected since the RL agent was trained under these constraints individually. With this observation, we can conclude that training for Case D is enough as it works relatively well for Case L. The further optimisation step with the Genetic Algorithm mentioned in the later chapters allows this flexibility.

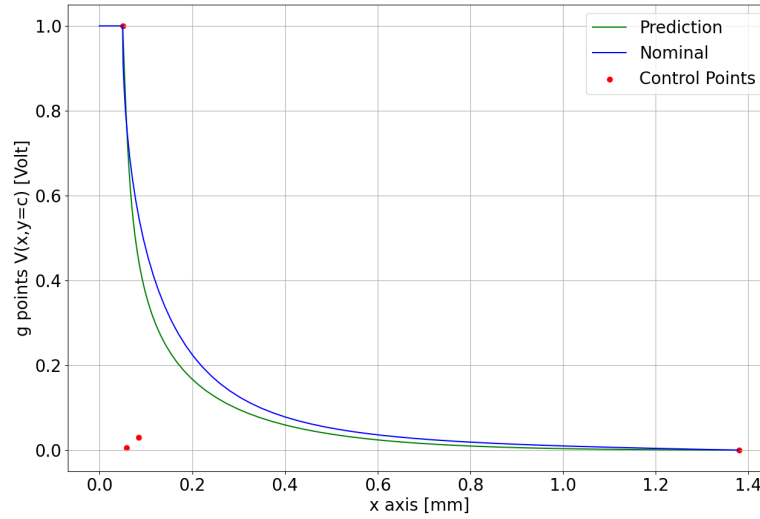


Figure 5.32: Test Case 2.1 Case L Prediction with Case D Model

Case	Model	Prediction	Nominal
Case L	Case D	$1.1947096779352938 \times 10^{-11}$ VAs	$1.1549945548531716 \times 10^{-11}$ VAs
Case D	Case L	$1.553741121139916 \times 10^{-10}$ VAs	$9.572519409401707 \times 10^{-11}$ VAs

Table 5.31: Energy after Prediction

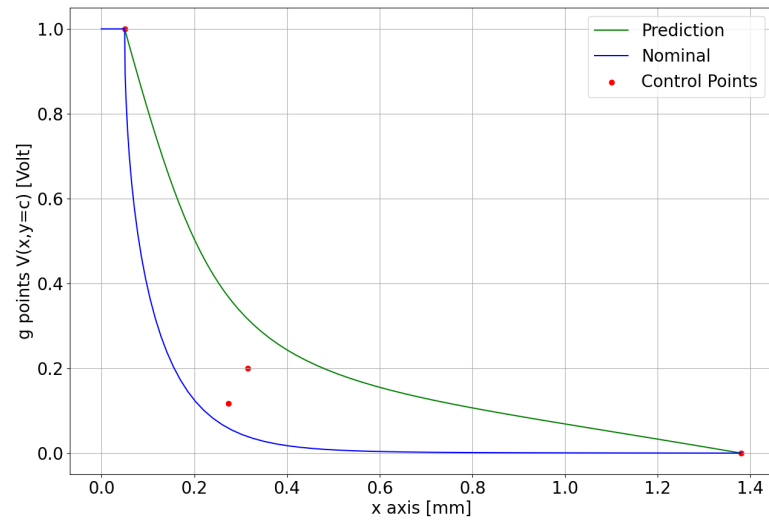


Figure 5.33: Test Case 2.1 Case D Prediction with Case L Model

Chapter 6

Test Cases

We shall add small perturbations to the original microstrip arrangement geometries and environment parameters and see how close the trained agent can predict. For the energy calculations of the curves, we shall use the parameters shown in table 6.1.

In the test case 2 series, we shall slightly vary the microstrip arrangement geometries and keep the dielectric constant the same as the parameters upon which the agent was trained. The figures and tables in each section show the agent’s performance with respect to the nominal curve to obtain a near-global solution.

The relative errors are shown in tables 6.8, 6.15, and 6.22. It can be observed that the relative error compared to the attributes of the nominal curve is very low, indicating that we are obtaining a near-global solution.

Curve	Num Points	Num Fourier Coeffs
prediction	47	2000
nominal	47	2000

Table 6.1: Test Parameters

6.1 Test Case 2.2

The microstrip arrangement geometry and environment parameters for this test case are given in table 6.2 and 6.3.

Parameters	Value
V_0	1 V
b	2.77×10^{-3} m
c	0.1382×10^{-3} m
d	0.06×10^{-3} m
a	1.39×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 6.2: Test Case 2.2 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	2.77×10^{-3} m
c	0.1382×10^{-3} m
d	0.06×10^{-3} m
a	1.39×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 6.3: Test Case 2.2 Case L Environment Parameters

6.2 Test Case 2.3

The microstrip arrangement geometry and environment parameters for this test case are given in table 6.9 and 6.10.

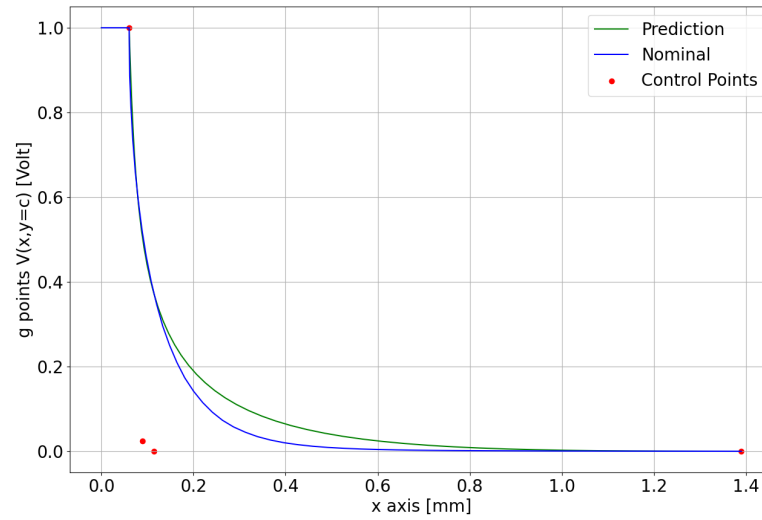


Figure 6.1: Test Case 2.2 G-Points Case D with SAC Agent

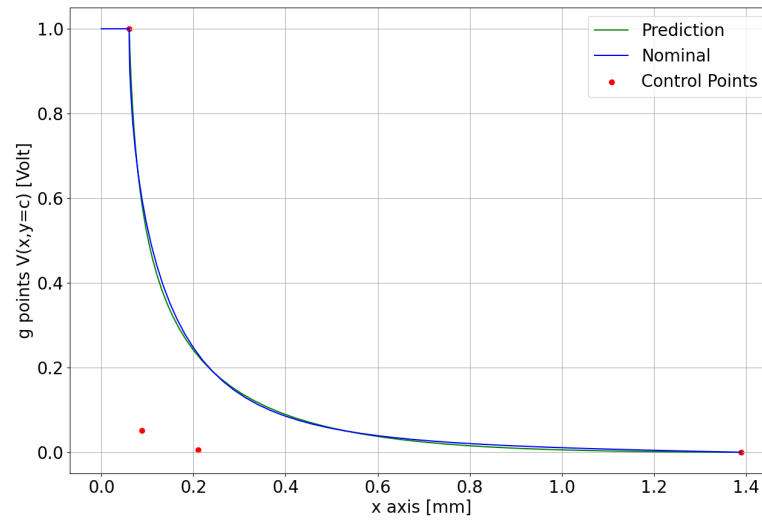


Figure 6.2: Test Case 2.2 G-Points Case L with SAC Agent

Case	Nominal	Prediction
Case D	$1.0468095074348635 \times 10^{-10}$ VAs	$1.0639441467102435 \times 10^{-10}$ VAs
Case L	$1.2461810048713603 \times 10^{-11}$ VAs	$1.254026481360012 \times 10^{-11}$ VAs

Table 6.4: Test Case 2.2: Prediction Energy Values for Case D and Case L

Case	Nominal	Prediction
Case D	$2.093619014869727 \times 10^{-10}$ F	$2.127888293420487 \times 10^{-10}$ F
Case L	$2.4923620097427205 \times 10^{-11}$ F	$2.508052962720024 \times 10^{-11}$ F

Table 6.5: Test Case 2.2: Capacitance for Case D and Case L

Case	Nominal	Prediction
Case D	46.17691431458123 Ω	45.66006577586866 Ω
Case L	133.83452881011652 Ω	132.99722938721214 Ω

Table 6.6: Test Case 2.2: Characteristic Impedance for Case D and Case L

Attribute	Nominal	Prediction
ϵ_{eff}	8.400140134882916	8.484223918113587
v_{ph}	103437362.30330169 m/s	102923522.65125842 m/s

Table 6.7: Test Case 2.2: Phase Velocity and Effective Dielectric Constant

Attribute	Case	Relative Error
Energy	Case D	1.6368440631922931%
Energy	Case L	0.6295615530956963%
Capacitance	Case D	1.6368440631922931%
Capacitance	Case L	0.6295615530956963%
Impedance	Case D	1.1192790735031164%
Impedance	Case L	0.625622872026048%
ϵ_{eff}	Case D/L	1.000980720327508%
v_{ph}	Case D/L	0.496764071126035%

Table 6.8: Test Case 2.2: Attribute Relative Error for Case D and Case L

Parameters	Value
V_0	1 V
b	2.80×10^{-3} m
c	0.1382×10^{-3} m
d	0.09×10^{-3} m
a	1.42×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 6.9: Test Case 2.3 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	2.80×10^{-3} m
c	0.1382×10^{-3} m
d	0.09×10^{-3} m
a	1.42×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 6.10: Test Case 2.3 Case L Environment Parameters

Case	Nominal	Prediction
Case D	$1.3062368224761704 \times 10^{-10}$ VAs	$1.3224351204870717 \times 10^{-10}$ VAs
Case L	$1.501359025253815 \times 10^{-11}$ VAs	$1.5103788493992156 \times 10^{-11}$ VAs

Table 6.11: Test Case 2.3: Prediction Energy Values for Case D and Case L

Case	Nominal	Prediction
Case D	$2.6124736449523407 \times 10^{-10}$ F	$2.6448702409741434 \times 10^{-10}$ F
Case L	$3.00271805050763 \times 10^{-11}$ F	$3.020757698798431 \times 10^{-11}$ F

Table 6.12: Test Case 2.3: Capacitance for Case D and Case L

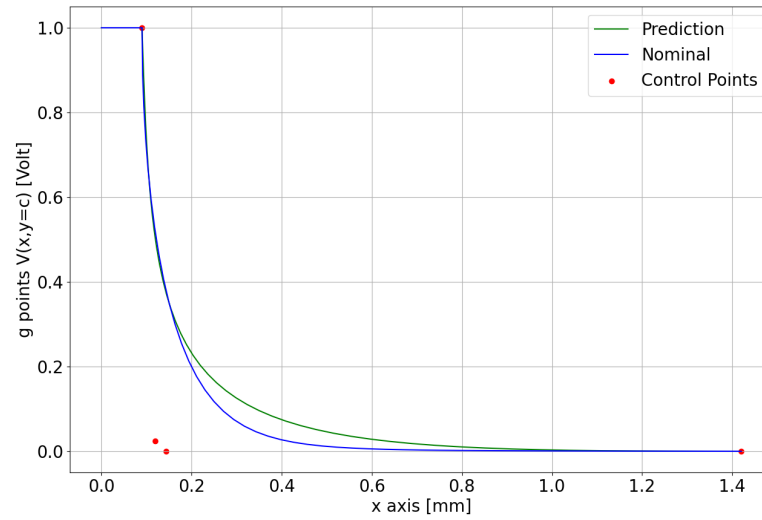


Figure 6.3: Test Case 2.3 G-Points Case D with SAC Agent

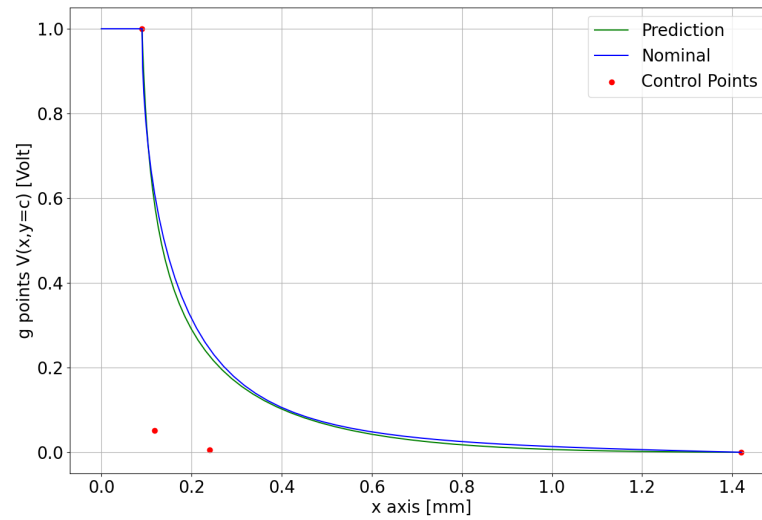


Figure 6.4: Test Case 2.3 G-Points Case L with SAC Agent

Case	Nominal	Prediction
Case D	37.66136536591817 Ω	37.318069269743546 Ω
Case L	111.08738469193295 Ω	110.42398247659322 Ω

Table 6.13: Test Case 2.3: Characteristic Impedance for Case D and Case L

Attribute	Nominal	Prediction
ϵ_{eff}	8.700362807992192	8.75565174269421
v_{ph}	101637042.9999383 m/s	101315633.28249057 m/s

Table 6.14: Test Case 2.3: Phase Velocity and Effective Dielectric Constant

Attribute	Case	Relative Error
Energy	Case D	1.2400736016762275%
Energy	Case L	0.6007772953491723%
Capacitance	Case D	1.2400736016762275%
Capacitance	Case L	0.6007772953491723%
Impedance	Case D	0.9115338566171409%
Impedance	Case L	0.5971895163248948%
ϵ_{eff}	Case D/L	0.6354784958074317%
v_{ph}	Case D/L	0.3162328497179154%

Table 6.15: Test Case 2.3: Attribute Relative Error for Case D and Case L

6.3 Test Case 2.4

The microstrip arrangement geometry and environment parameters for this test case are given in table 6.16 and 6.17.

Parameters	Value
V_0	1 V
b	3.10×10^{-3} m
c	0.1382×10^{-3} m
d	1.20×10^{-3} m
a	1.45×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 6.16: Test Case 2.4 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	3.10×10^{-3} m
c	0.1382×10^{-3} m
d	1.20×10^{-3} m
a	1.45×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 6.17: Test Case 2.4 Case L Environment Parameters

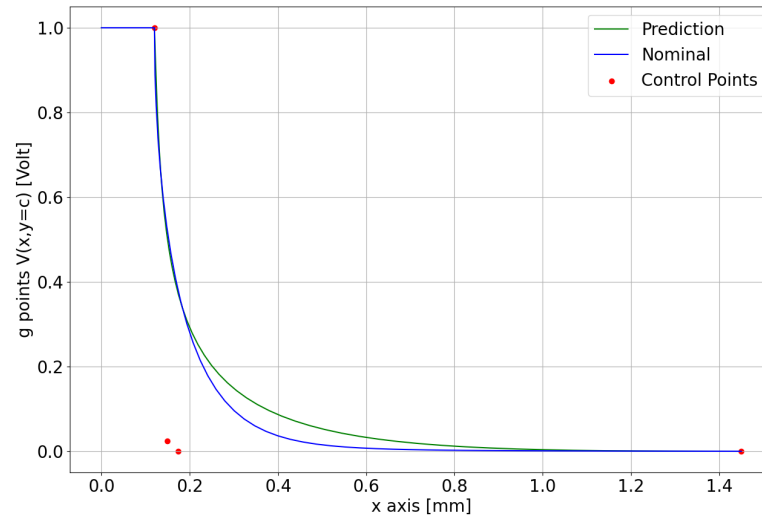


Figure 6.5: Test Case 2.4 G-Points Case D with SAC Agent

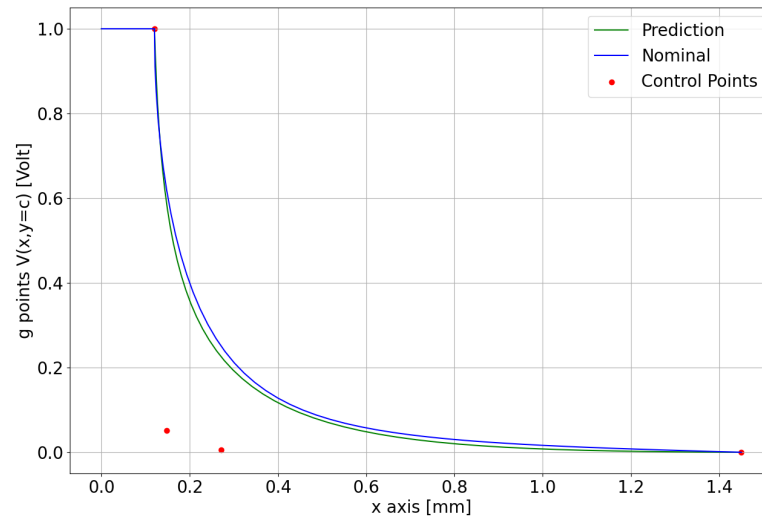


Figure 6.6: Test Case 2.4 G-Points Case L with SAC Agent

Case	Nominal	Prediction
Case D	$1.5608399996953995 \times 10^{-10}$ VAs	$1.5767531208929557 \times 10^{-10}$ VAs
Case L	$1.742083135386507 \times 10^{-11}$ VAs	$1.752662165483713 \times 10^{-11}$ VAs

Table 6.18: Test Case 2.4: Prediction Energy Values for Case D and Case L

Case	Nominal	Prediction
Case D	$3.121679999390799 \times 10^{-10}$ F	$3.1535062417859114 \times 10^{-10}$ F
Case L	$3.484166270773014 \times 10^{-11}$ F	$3.505324330967426 \times 10^{-11}$ F

Table 6.19: Test Case 2.4: Capacitance for Case D and Case L

Case	Nominal	Prediction
Case D	31.98421307920982 Ω	31.726221057501704 Ω
Case L	95.73713458977545 Ω	95.1592673611724 Ω

Table 6.20: Test Case 2.4: Characteristic Impedance for Case D and Case L

Attribute	Nominal	Prediction
ϵ_{eff}	8.959618332733037	8.996332276379068
v_{ph}	100155763.98122224 m/s	99951187.70492615 m/s

Table 6.21: Test Case 2.4: Phase Velocity and Effective Dielectric Constant

Attribute	Case	Relative Error
Energy	Case D	1.0195228979691462%
Energy	Case L	0.6072632173698687%
Capacitance	Case D	1.0195228979691462%
Capacitance	Case L	0.6072632173698687%
Impedance	Case D	0.8066230082609547%
Impedance	Case L	0.6035977900102822%
ϵ_{eff}	Case D/L	0.4097712902780772%
v_{ph}	Case D/L	0.20425811572307426%

Table 6.22: Test Case 2.4: Attribute Relative Error for Case D and Case L

Chapter 7

Reinforcement Learning + Genetic Algorithm

7.1 Genetic Algorithm

Inspired by natural selection and evolution, genetic algorithms represent an optimisation algorithm aimed at discovering optimal or near-optimal solutions to intricate problems by iteratively refining a population of candidate solutions. The workflow of genetic algorithms encompasses several key steps, as shown in Figure 7.1.

First, Initialization: The process begins with generating an initial population comprising randomly created individuals, each representing a potential solution encoded as a string or chromosome. Second, Fitness Evaluation: Every individual within the population undergoes assessment using a fitness function, gauging its effectiveness in problem-solving. Individuals with superior fitness scores are more likely to be chosen for reproduction.

Third, Selection: Through various methods like roulette wheel selection or tournament selection, individuals from the current population serve as parents for the subsequent generation, favouring those with higher fitness scores in line with the "survival of the fittest" concept. Fourth, Crossover: Selected parent individuals undergo a crossover operation, wherein genetic material (chromosome segments) is exchanged, producing offspring for the next generation. This introduces novel solutions to the population, exploring diverse regions of the search space. Fifth, Mutation: Offspring individuals undergo random alterations or mutations with a

low probability, sustaining diversity within the population and preventing premature convergence to local optima by exploring new areas of the search space.

Sixth, Replacement: The offspring individuals, possibly combined with some of the fitter individuals from the previous generation, constitute the new population for the subsequent generation. Finally, Termination: The algorithm repeats the evaluation, selection, crossover, and mutation steps until a termination condition is met, such as reaching a satisfactory fitness level, a fixed number of generations, or a predefined computational time limit. [36] [37]

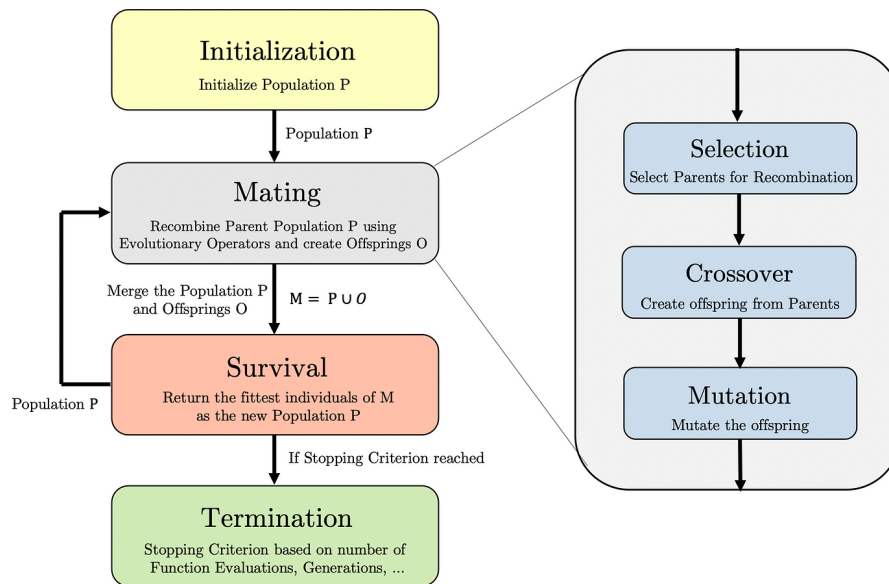


Figure 7.1: Genetic Algorithm Flow Chart [8]

The fundamental principles guiding genetic algorithms include the survival of the fittest individuals, genetic recombination via crossover, and the introduction of random variations through mutation. These principles facilitate effective search space exploration, leading to convergence towards optimal or near-optimal solutions across successive generations.

Genetic algorithms find applications across various domains, such as scheduling, engineering design, machine learning model optimisation, routing, finance portfolio optimisation, bioinformatics, and drug design. They handle problems characterized

by large, complex search spaces, non-linear or discontinuous objective functions, and multiple constraints or objectives. However, their computational cost can be significant, particularly for high-dimensional or complex problems, and their performance hinges on appropriately tuning parameters and designing fitness functions.

7.2 Limitations Of RL

Reinforcement Learning is extremely susceptible to noise. Slight variations in random initialization can significantly impact training outcomes, posing challenges for reproducibility in experimental results. The inherent noise complicates the comparison of algorithms and hyperparameter configurations, as improved performance may stem from alterations or mere chance occurrences. Training sessions must be conducted under precisely identical conditions to ensure consistency and robustness. This requirement adds complexity to the iterative refinement of algorithms.

Soft Actor-Critic (SAC) stands out as one of the prevailing algorithms in the current market, boasting nearly 20 hyperparameters requiring fine-tuning [11]. In deep reinforcement learning (RL), you encounter an additional layer of complexity alongside the customary deep learning parameters concerning network architecture, such as layer count, nodes per layer, activation function, max pool, dropout, batch normalization, and learning rate. This complexity stems from over 10 hyperparameters specific to RL, including buffer size, entropy coefficient, gamma, and action noise, among others.

Moreover, "Hyperparameters" are included in reward shaping (RewardArt) to steer the agent's behaviour towards desired outcomes. Tuning even a single one of these parameters can prove to be exceedingly challenging. Like many hyperparameter tuning endeavours, there's often no straightforward setting for each parameter or a fail-safe method to pinpoint the optimal hyperparameters efficiently. Instead, it often feels like navigating in the dark until a semblance of effectiveness is achieved.

With numerous components in play, the likelihood of introducing bugs is high and pinpointing them becomes a formidable task. In reinforcement learning (RL), multiple learning networks add complexity. Moreover, the inherent randomness in the learning process means that outcomes may vary from one run to the next. Determining whether a discrepancy is due to an introduced bug or a chance occurrence in

the random seed is challenging. This uncertainty necessitates conducting numerous additional experiments to conclude, which demands a significant investment of time.

In model-free learning, there's no attempt to construct or learn a model of the environment. Therefore, the sole method of policy acquisition is through direct interaction with the environment. With an on-policy approach, policy learning or enhancement relies solely on samples obtained while acting according to the current policy. Consequently, each time a backward gradient update is performed, previous samples must be discarded, and fresh ones collected. Notably, algorithms like Proximal Policy optimisation (PPO) exemplify this state-of-the-art model-free, on-policy paradigm. As a result, extensive interaction with the environment, often amounting to millions of steps, is necessary before policy learning can occur.[38]

7.3 GA optimisation of RL Results

As mentioned in the previous section, RL algorithms and their training are usually like searching in the darkness. However, it is the best method for the same. This would enable us to train and reach a near-global solution for our problem.

The results in the Chapter Test Cases show that we are reaching near-global solutions for the test cases slightly perturbed from the training case. Thus, the trained model could then be used to predict the near-global solutions, which can then further be optimized faster using a Genetic Algorithm.

GA can handle combinatorial problems well and does not require training. It does not necessarily need to be GA; any non-linear optimisation would eventually work. However, non-linear optimisation usually takes a lot of time for a large search space problem. [39]

7.4 Software Implementation for GA

The pymoo framework stands out as a robust Python library tailored for multi-objective optimisation employing evolutionary algorithms, including genetic algorithms. Highlighted below are pivotal aspects of pymoo's genetic algorithm imple-

mentation:

Modular Design: pymoo offers a modular and adaptable setup for genetic algorithms. Users benefit from the flexibility to fine-tune various elements, such as selection, crossover, mutation operators, and survival strategies, to suit their needs.

Single and Multi-Objective: pymoo supports both single-objective and multi-objective genetic algorithms to accommodate diverse optimisation requirements. It provides a fundamental $(\mu + \lambda)$ GA implementation named GA for single-objective tasks.

Multi-Objective Algorithms: In the realm of multi-objective optimisation, pymoo boasts a rich selection of algorithms, including NSGA-II, NSGA-III, MOEA/D, RVEA, and SMS-EMOA, among others, empowering users with a breadth of choices.

Customization: pymoo empowers users with extensive customization options for fine-tuning the genetic algorithm's behaviour. This includes tailoring initial population sampling strategies, parent selection methods (e.g., tournament, rank-based), crossover operators (e.g., SBX, uniform), mutation operators (e.g., polynomial, uniform), and survival strategies for parent and offspring selection.

Duplicate Elimination: To streamline the optimisation process, Pymoo integrates a built-in feature to weed out duplicate individuals after parent-offspring merging, ensuring population diversity is maintained throughout the evolutionary process.[13]

The code shown in Listing 4 sets up a basic GA for the single-objective 'g1' problem, with a population size of 100 and duplicate elimination enabled. The program for Genetic Algorithm optimisation of the RL predicted curve was developed along with M.Sc. Julian Withöft [40].

Overall, pymoo provides a comprehensive and flexible framework for implementing and customizing genetic algorithms, making it a valuable tool for researchers and practitioners in evolutionary optimisation.

7.5 GA optimisation Results

GA is applied to the predicted curves after training on the Test Case 2.1 environment parameters in tables 5.18 and 5.19. The initial population of the GA will

Listing 4 Genetic Algorithm Implementation Example [13]

```
from pymoo.algorithms.soo.nonconvex.ga import GA
from pymoo.problems import get_problem
from pymoo.optimize import minimize

problem = get_problem("g1")

algorithm = GA(
    pop_size=100,
    eliminate_duplicates=True)

res = minimize(problem,
                algorithm,
                seed=1,
                verbose=False)

print("Best solution found: \nX = %s\nF = %s" % (res.X, res.F))
```

be our predicted curve, which will be further optimized by minimizing energy. To reduce the search space, we shall provide a threshold region with the same concept from Figure 5.9.

In figures 7.2 and 7.3, the RL predicted curve is taken as the initial population, which will be further optimized using GA to reach the Nominal curve. GA is run with a population size of one hundred and one thousand evaluation steps as given in table 7.1.

After running for one thousand steps, we observe that the evaluation parameter (energy) converges. The convergence is shown in the figures 7.4 and 7.5. The resulting curves obtained from GA optimisation are shown in figures 7.12 and 7.13.

The GA optimisation ran for one thousand evaluation steps to converge finally, and the time taken for the optimisation is given in table 7.2. The energy values obtained in comparison to the nominal curve are shown in table 7.3.

Parameter	Value
population size	100
evaluation steps	1000

Table 7.1: Genetic Algorithm Parameters

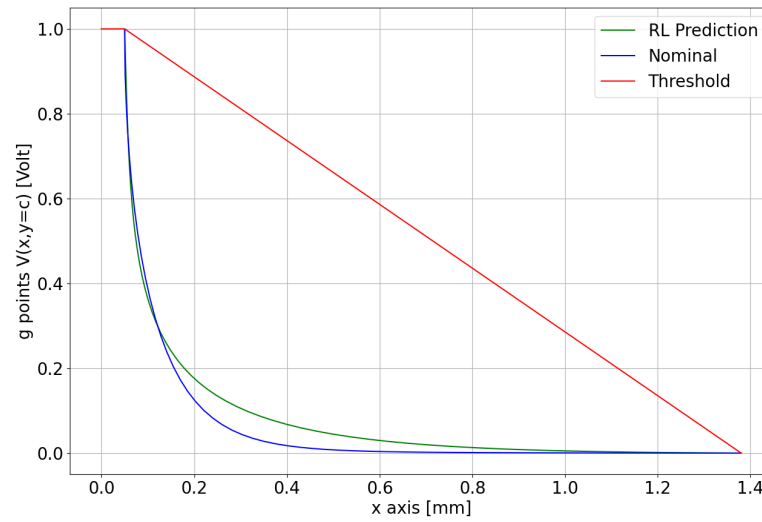


Figure 7.2: Case D Training Environment: Initial Population for GA

Case	Evaluation Time
Case D	323.55 s
Case L	243.4 s

Table 7.2: Genetic Algorithm Evaluation Time

Case	Nominal	GA Result
Case D	$9.572519409401707 \times 10^{-11}$ VAs	$9.57754469908439 \times 10^{-11}$ VAs
Case L	$1.1549945548531716 \times 10^{-11}$ VAs	$1.15683880719816 \times 10^{-11}$ VAs

Table 7.3: Energy Values for Case D and Case L after GA optimisation

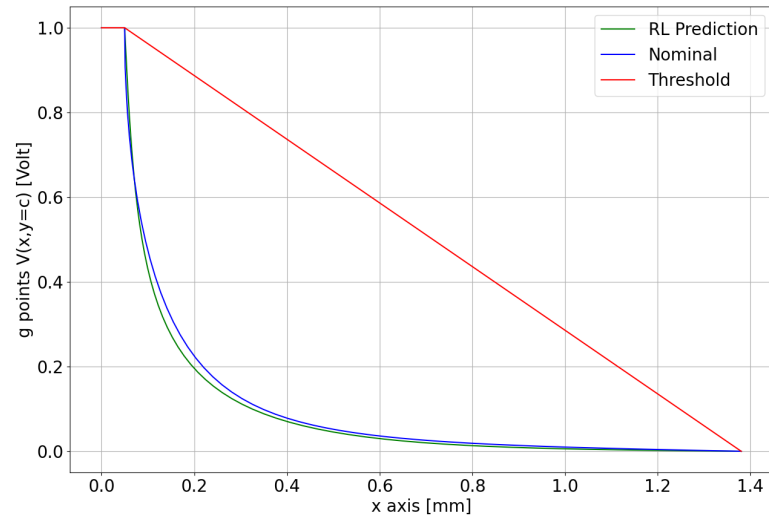


Figure 7.3: Case L Training Environment: Initial Population for GA

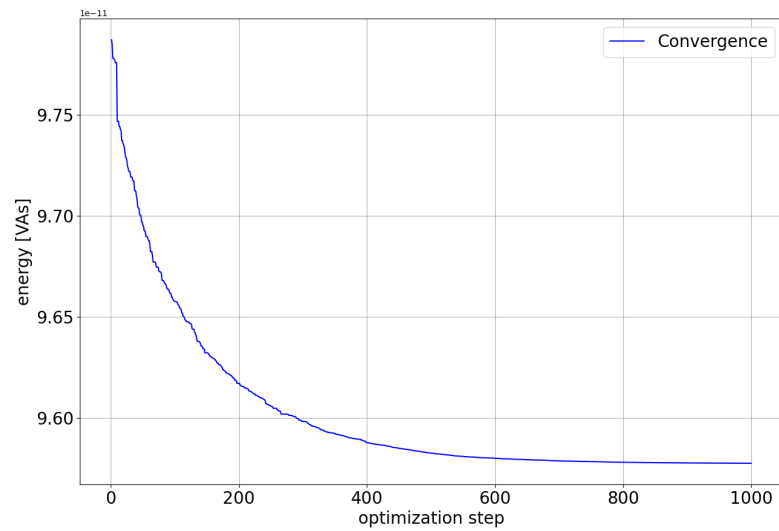


Figure 7.4: Case D Training Environment: GA Convergence

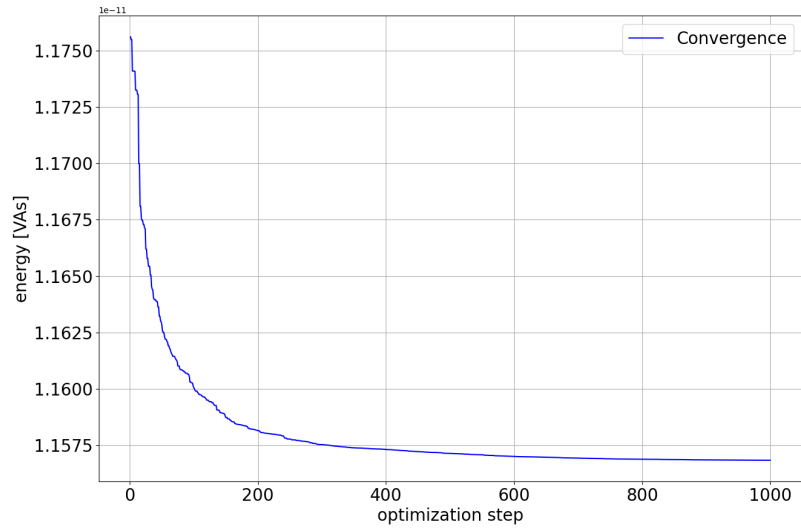


Figure 7.5: Case L Training Environment: GA Convergence

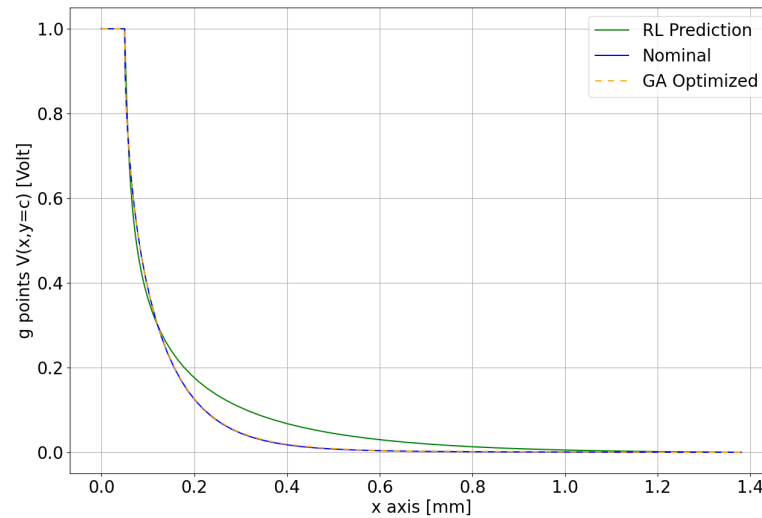


Figure 7.6: Case D Training Environment: GA Result

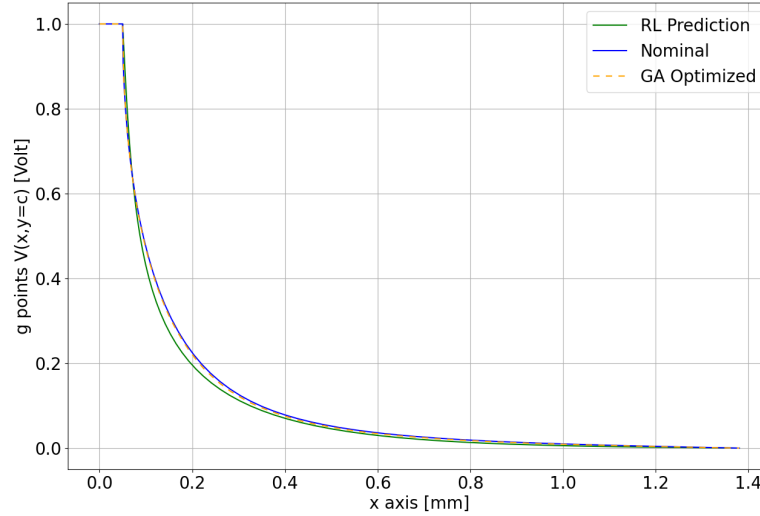


Figure 7.7: Case L Training Environment: GA Result

7.6 Observations

From the Chapter Test Cases, it can be observed that our RL algorithm has successfully predicted the near global curve. This prediction, on average, takes around $1.75 \text{ s} \pm 30.7 \text{ ms}$. When combined with GA optimisation, which takes very little time, this could work as a suitable solution for our problem. The time to run both is in the table 7.4.

Case	Time RL	Time GA
Case D	$1.75 \text{ s} \pm 30.7 \text{ ms}$	323.55 s
Case L	$1.75 \text{ s} \pm 30.7 \text{ ms}$	243.4 s

Table 7.4: Run Time for RL Prediction and GA optimisation

7.7 Test Case 2 Series

We have a proof of concept in the above sections where GA was deployed on the RL predictions of Test Case 2.1, and further optimisation was achieved. The GA optimisation is done for the rest of the test case 2 series mentioned in the chapter Test Cases.

7.7.1 Test Case 2.2

Parameters	Value
V_0	1 V
b	2.77×10^{-3} m
c	0.1382×10^{-3} m
d	0.06×10^{-3} m
a	1.39×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 7.5: Test Case 2.2 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	2.77×10^{-3} m
c	0.1382×10^{-3} m
d	0.06×10^{-3} m
a	1.39×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 7.6: Test Case 2.2 Case L Environment Parameters

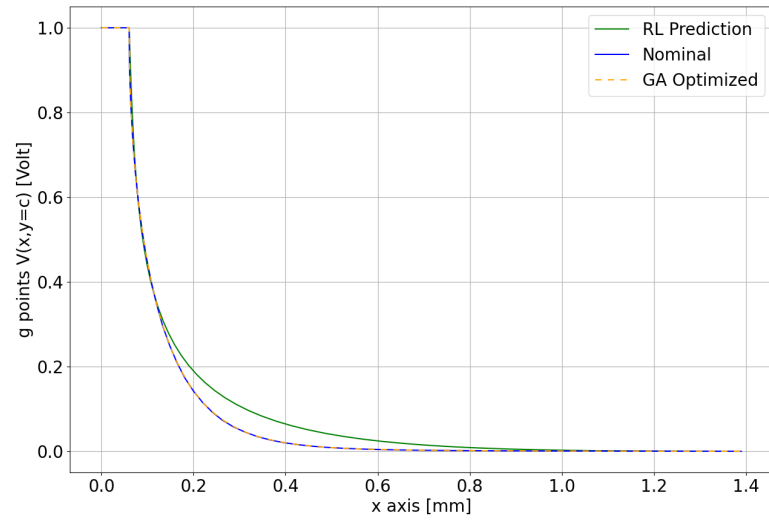


Figure 7.8: Case D Test Case 2.2: GA Result

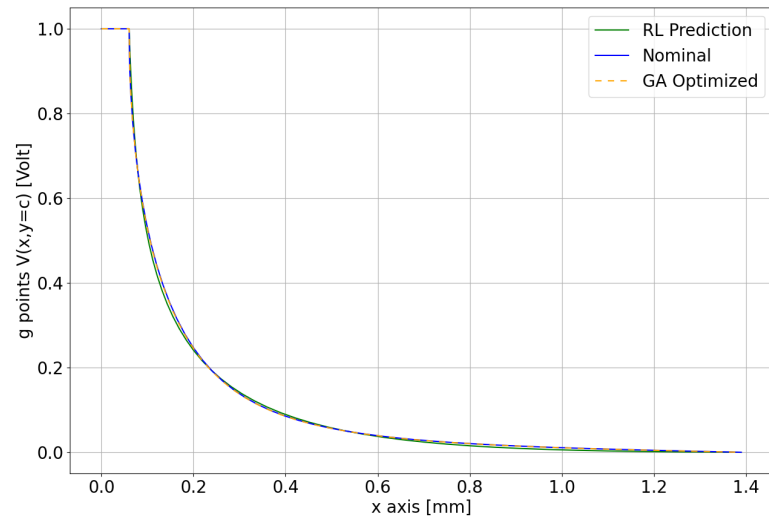


Figure 7.9: Case L Test Case 2.2: GA Result

7.7.2 Test Case 2.3

Parameters	Value
V_0	1 V
b	2.80×10^{-3} m
c	0.1382×10^{-3} m
d	0.09×10^{-3} m
a	1.42×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 7.7: Test Case 2.3 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	2.80×10^{-3} m
c	0.1382×10^{-3} m
d	0.09×10^{-3} m
a	1.42×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 7.8: Test Case 2.3 Case L Environment Parameters

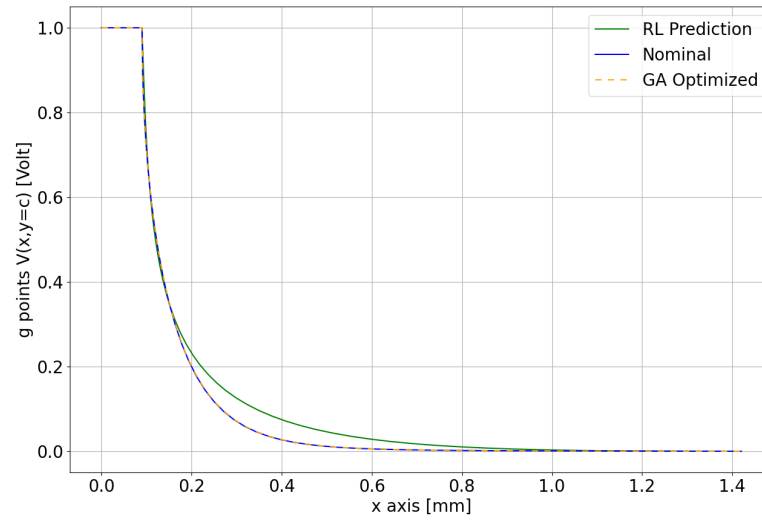


Figure 7.10: Case D Test Case 2.3: GA Result

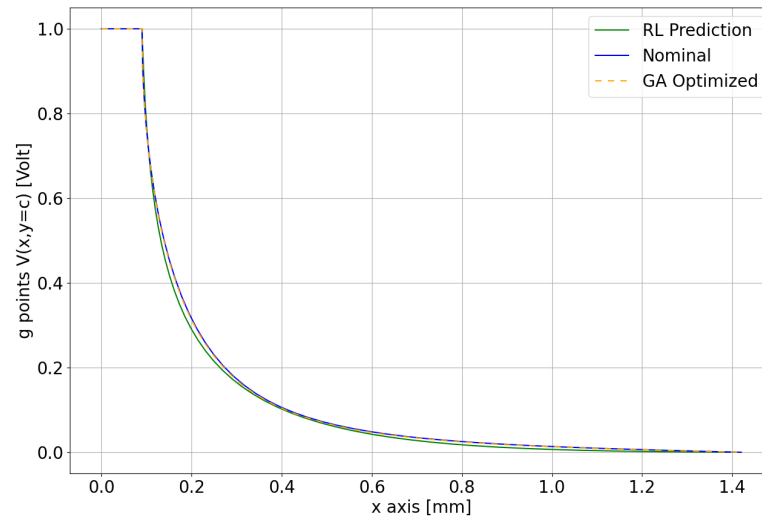


Figure 7.11: Case L Test Case 2.3: GA Result

7.7.3 Test Case 2.4

Parameters	Value
V_0	1 V
b	3.10×10^{-3} m
c	0.1382×10^{-3} m
d	1.20×10^{-3} m
a	1.45×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	12.9

Table 7.9: Test Case 2.4 Case D Environment Parameters

Parameters	Value
V_0	1 V
b	3.10×10^{-3} m
c	0.1382×10^{-3} m
d	1.20×10^{-3} m
a	1.45×10^{-3} m
t	0 m
ϵ_{r1}	1.0
ϵ_{r2}	1.0

Table 7.10: Test Case 2.4 Case L Environment Parameters

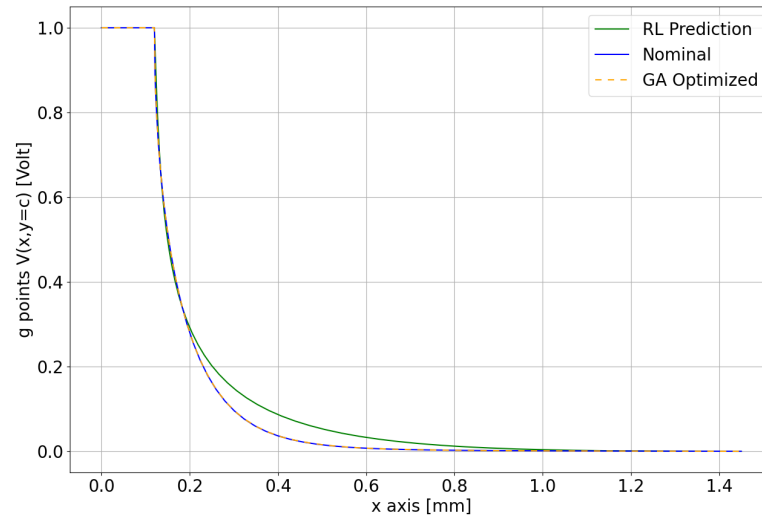


Figure 7.12: Case D Test Case 2.4: GA Result

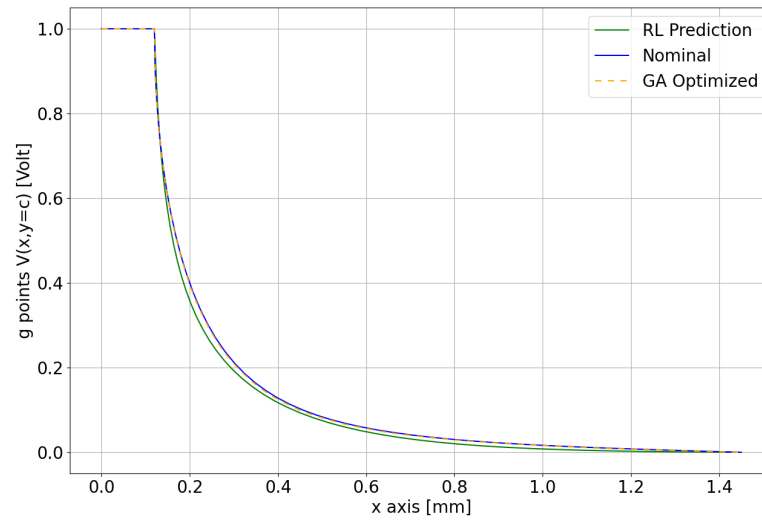


Figure 7.13: Case L Test Case 2.4: GA Result

7.8 Impact of RL

It is natural to ask whether the RL prediction to give a starting curve is necessary. GA performs really well in optimising, as the various test cases prove, and hence, we should test whether it can perform well on its own without the RL starting curve.

GA optimisation was done without the RL curve for the environment parameters as given in table 5.18. Instead of giving a starting shape, a line segment was given. In addition, all the RL methods were also compared, as shown in Figure 7.14.

It can be seen from Figure 7.15 that only the GA optimisation starting from the curves predicted by RL methods 3 and 4 converge to the nominal curve; the rest do not. From table 7.11, we can see that for almost the same time, GA starting from RL prediction curves perform much better than the rest. Hence, it can be concluded that the near-global solution provided by the RL prediction is essential for the GA optimisation to perform well.

Another interesting factor is the convergence for Method 3 and Method 4, as shown in Figure 7.16. This figure shows that Method 4 converges around 400 steps, and Method 3 converges around 600 steps. As given in table 7.11, the final energy value obtained from optimising the Method 3 curve is slightly lesser than that for the Method 4 curve. This shows that for GA to optimise a curve, it need not be a near-global solution like the curve given by Method 4, and Method 3 curve is enough. This shows the power of GA for optimisation tasks. However, the Method 4 curve is still useful on its own as it is a near-global solution.

With the above inferences, it can be concluded that having a good starting curve is beneficial, and hence, the RL search for a near-global solution is essential for the GA optimisation step's functioning.

Method	Energy Pred	Energy Nom	Converged	Completion Time
no RL	1.18×10^{-10} VAs	9.57×10^{-11} VAs	No	238.96 s
Method 2	9.97×10^{-11} VAs	9.57×10^{-11} VAs	No	237.26 s
Method 3	9.56×10^{-11} VAs	9.57×10^{-11} VAs	Yes	251.26 s
Method 4	9.57×10^{-11} VAs	9.57×10^{-11} VAs	Yes	244.82 s

Table 7.11: Test Case 2.1 Case D: GA Optimisation Results

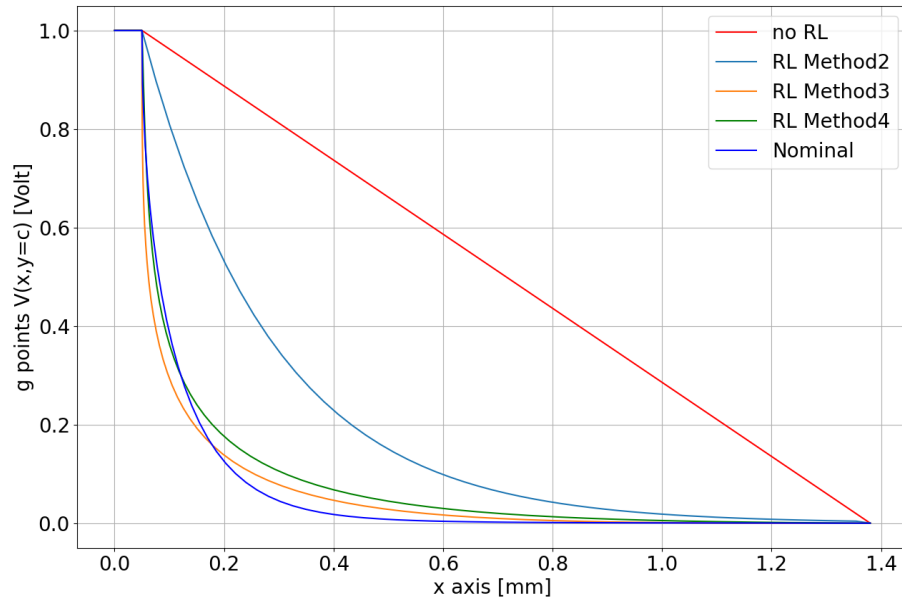


Figure 7.14: Test Case 2.1 Case D: GA Optimisation Starting Curves

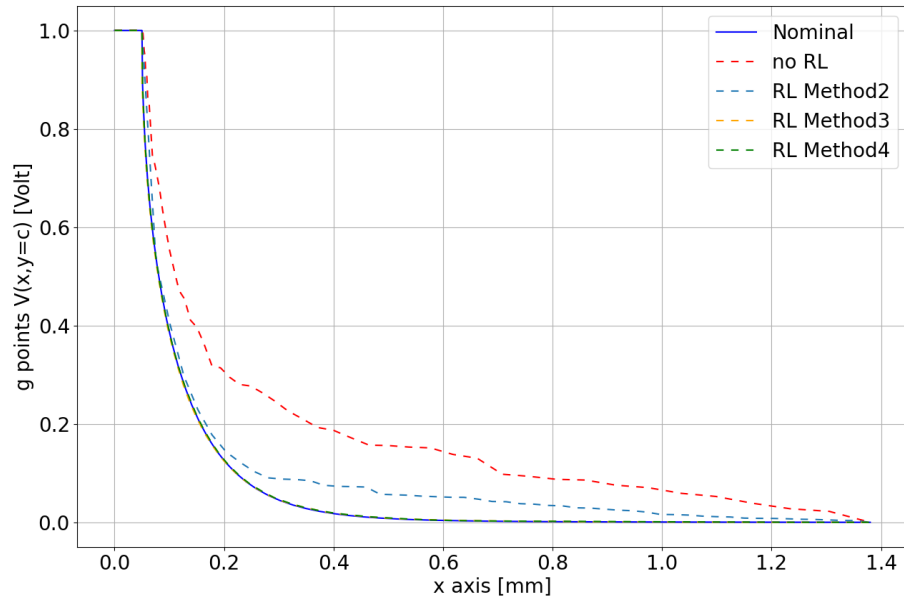


Figure 7.15: Test Case 2.1 Case D: GA optimisation Results

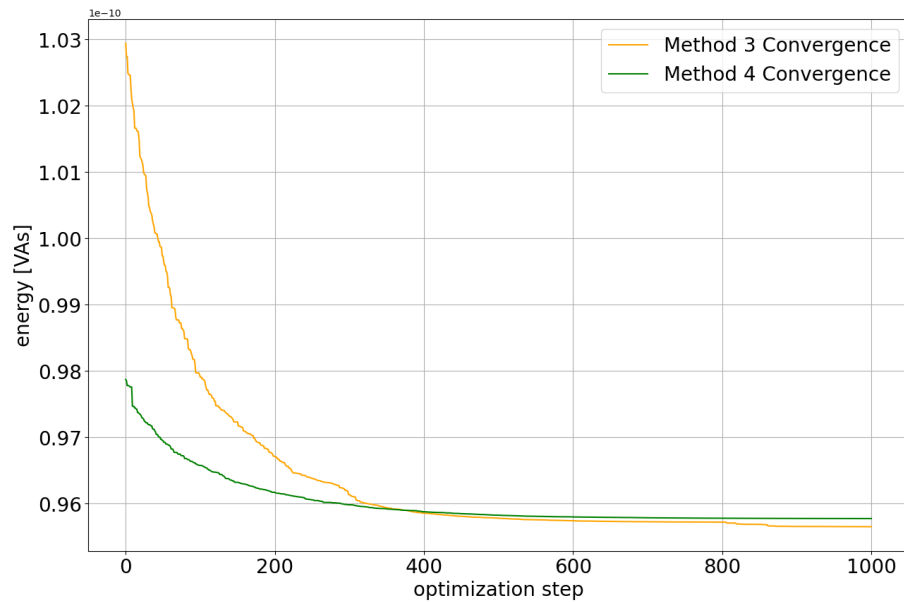


Figure 7.16: Method 3 and Method 4 GA Convergence

Chapter 8

Conclusion and Future Scope

8.1 Future Scope

Figure 5.16 and 5.23 show that cases D and L stabilize around seven hours of training. Considering the accuracy we achieved (even though it is near-global), seven hours of training is quite long. We could try out faster implementations of the DRL algorithm like JAX and SB3-JAX.

JAX (Just Another eXperimental) is a library developed by Google that provides a powerful and composable framework for numerical computing, particularly designed for machine learning and scientific computing tasks. JAX is built on top of the XLA (Accelerated Linear Algebra) compiler and is primarily focused on providing high-performance and automatic differentiation capabilities for array-based computations. Since the NVIDIA GPU support is unavailable for Windows OS, it is not implemented here.[41] [11]

The pure Python [12] implementation is generally slow as it is an interpreted language with dynamic typing. An alternative approach for improving the speed with little deviation from Python code is Numba [42], a JIT (Just In Time) compiler approach that turns frequently reused code snippets to byte code, thereby achieving faster run.

With a detailed development phase, we could achieve an extreme speedup from normal Python using Cython. Cython is a programming language and compiler that allows developers to write C extensions for Python, combining the ease of Python

with the speed of C. It is primarily used to enhance the performance of Python code by compiling Python-like code into optimized C or C++ code, which can then be compiled into native machine code. This requires an additional development phase since we must rewrite the entire code in a different language setting. Doing so would result in a production-ready version of the learning algorithm that could replace the optimisation algorithm. [43]

In Figure 8.1, the quicksort algorithm is implemented with various computing programs and the execution time for each is compared. We can see that compiled languages like C++ and Cython have a significant performance boost over the rest.

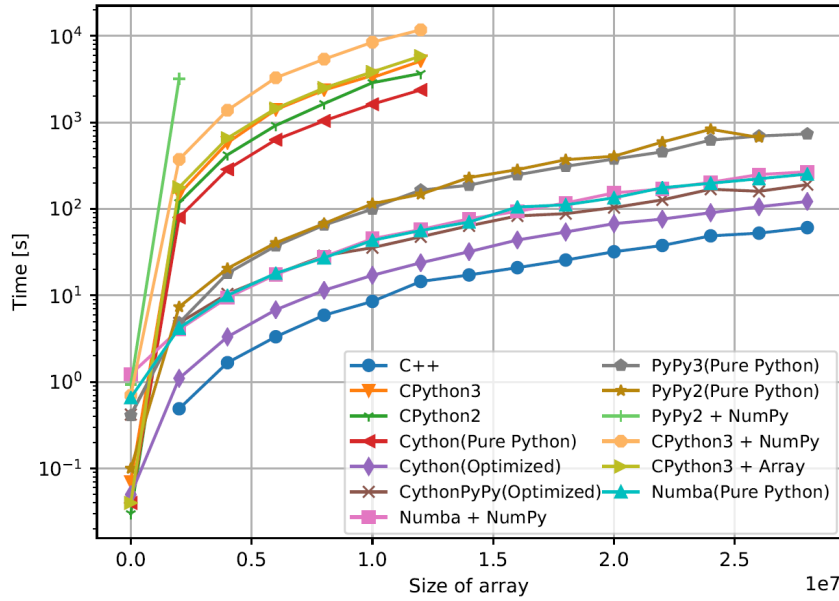


Figure 8.1: Comparison of the Execution Times for Quicksort Algorithm [9]

8.2 Conclusion

In conclusion, a Reinforcement Learning algorithm was developed to predict near-global solutions optimised using a Genetic Algorithm. The combination of RL + GA is proposed as the final solution. Future work on software optimisation is proposed to improve the RL training speed. The combination of Case D RL model +

GA is implemented and hosted as a web application with Graphical User Interface based on streamlit [44] as shown in Figure 8.2. This application allows users to interact easily, feed in the environment parameters, see the convergence's live plot, and download the resulting curve coordinates.

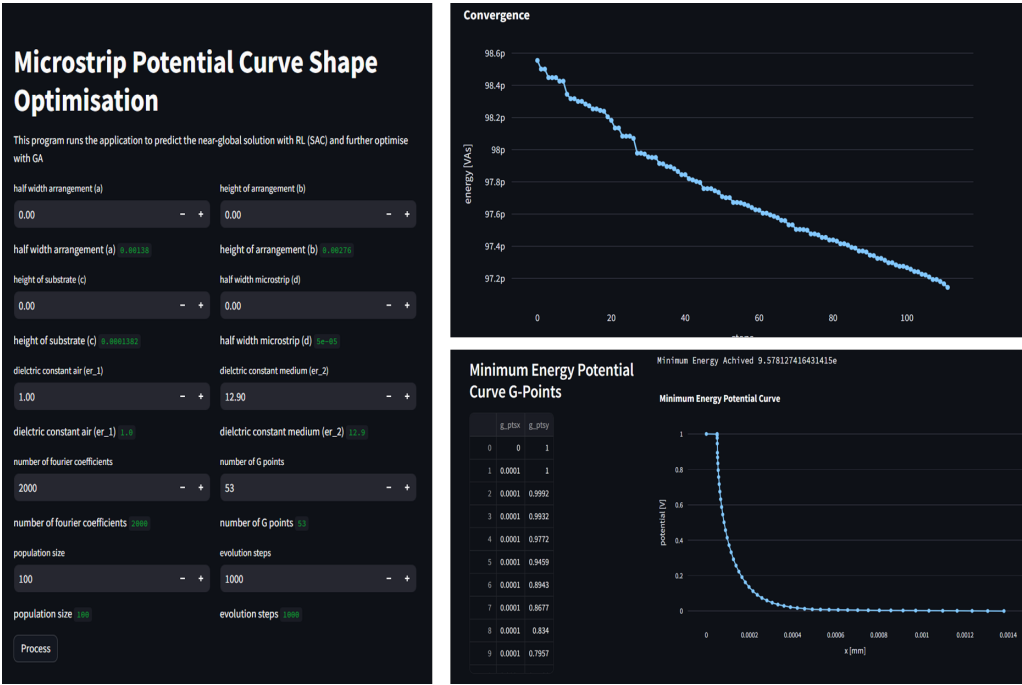


Figure 8.2: GUI Application for Potential Curve Shape Optimisation

Bibliography

- [1] J. Peng, E. C. Jury, P. Dönnies, and C. Ciurtin, “Machine learning techniques for personalised medicine approaches in immune-mediated chronic inflammatory diseases: Applications and challenges,” *Frontiers in Pharmacology*, vol. 12, p. 720694, 2021.
- [2] V. Vijayan, “Deep reinforcement learning: Artificial intelligence, machine learning, and deep learning,” *Medium*, 2024.
- [3] OpenAI, “Spinning up in deep reinforcement learning,” <https://spinningup.openai.com>, 2024.
- [4] V. Saxena, B. Guldogan, and D. D. Nimara, “Online and offline reinforcement learning: What are they and how do they compare,” *Ericsson Blog*, 2023.
- [5] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015, software available from [tensorflow.org](https://www.tensorflow.org). [Online]. Available: <https://www.tensorflow.org/>
- [6] MMRNDEV. (2024) Understanding bézier curves. [Online]. Available: <https://mmrndev.medium.com/understanding-b%C3%A9zier-curves-f6eaa0fa6c7d>
- [7] M. S. Emre Ecik, “Relation of fourier coefficients and number of g points for energy calculation,” 2024, internal Communication.

- [8] A. Fawzy, “An introduction to genetic algorithms: The concept of biological evolution in optimization,” *Towards Data Science*, 2019.
- [9] L. Razik, “High-performance computing methods in large-scale power system simulation,” Ph.D. dissertation, Dissertation, RWTH Aachen University, 2020, 2020.
- [10] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “Openai gym,” 2016.
- [11] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [12] P. S. Foundation. (2022) Python 3 documentation. Accessed: March 2022. [Online]. Available: <https://docs.python.org/3/reference/index.html>
- [13] J. Blank and K. Deb, “pymoo: Multi-objective optimization in python,” *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.
- [14] OpenAI. (2024) Proximal policy optimization (ppo). [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>
- [15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International conference on machine learning*. PMLR, 2016, pp. 1928–1937.
- [16] OpenAI, “Deep deterministic policy gradient (ddpg),” <https://spinningup.openai.com/en/latest/algorithms/ddpg.html>, 2024.
- [17] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.
- [18] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.
- [19] R. Shyam and R. Singh, “A taxonomy of machine learning techniques,” vol. 8, pp. 18–25, 12 2021.

- [20] W. John, “Charakteristische parameter von leitungsstrukturen auf quasi-planaren substraten,” Dissertation, Brandenburgische Technische Universität Cottbus, 2008.
- [21] E. Yamashita, H. Ohashi, and K. Atsuki, “Characterization of microstrip lines near a substrate edge and design formulas of edge-compensated microstrip lines (mmics),” *IEEE transactions on microwave theory and techniques*, vol. 37, no. 5, pp. 890–896, 1989.
- [22] H. A. Wheeler, “Transmission-line properties of parallel wide strips by a conformal-mapping approximation,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 12, no. 3, pp. 280–289, 1964.
- [23] —, “Transmission-line properties of parallel strips separated by a dielectric sheet,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 13, no. 2, pp. 172–185, 1965.
- [24] H. Lee and V. Tripathi, “New perspectives on the green’s function for quasi-tem planar structures,” in *1983 IEEE MTT-S International Microwave Symposium Digest*. IEEE, 1983, pp. 571–573.
- [25] J. G. Fikioris, J. L. Tsalamengas, and G. J. Fikioris, “Strongly convergent green’s function expansions for rectangularly shielded microstrip lines,” *IEEE transactions on microwave theory and techniques*, vol. 36, no. 10, pp. 1386–1396, 1988.
- [26] W. John, “Prerequisites for the calculation of the characteristic parameters of a single microstrip by solving a mixed boundary value problem,” Internal Report – TU Dortmund AG DAT, 2024.
- [27] E. G. Bakhoun, “Proof of thomson’s theorem of electrostatics,” *Journal of electrostatics*, vol. 66, no. 11-12, pp. 561–563, 2008.
- [28] H. Ghraieb, J. Viquerat, A. Larcher, P. Meliga, and E. Hachem, “Single-step deep reinforcement learning for two-and three-dimensional optimal shape design,” *AIP Advances*, vol. 12, no. 8, p. 110080, 2022.
- [29] J. Viquerat, J. Rabault, A. Kuhnle, H. Ghraieb, A. Larcher, and E. Hachem, “Direct shape optimization through deep reinforcement learning,” *Journal of Computational Physics*, vol. 428, p. 110080, 2021.

- [30] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.
- [31] C. E. Shannon, “A mathematical theory of communication,” *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [32] W. John and G. Mrozynski, *Calculation of the parameters of microstrip lines by solving a mixed boundary value problem*. Cadlab, 1990.
- [33] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, “Automatic differentiation in pytorch,” in *NIPS-W*, 2017.
- [34] A. B. Tech. (2024) Top 8 reward shaping techniques in reinforcement learning. [Online]. Available: <https://medium.com/@aiblogtech/top-8-reward-shaping-techniques-in-a-reinforcement-learning-a0721d59deda>
- [35] J. Corazza, I. Gavran, and D. Neider, “Reinforcement learning with stochastic reward machines,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 6, 2022, pp. 6429–6436.
- [36] K. Man, K. Tang, and S. Kwong, “Genetic algorithms: concepts and applications [in engineering design],” *IEEE Transactions on Industrial Electronics*, vol. 43, no. 5, pp. 519–534, 1996.
- [37] MathWorks, “What Is the Genetic Algorithm?” <https://www.mathworks.com/help/gads/what-is-the-genetic-algorithm.html>, Accessed: April 2024.
- [38] M. Tabrez. (2018) Why you shouldn’t use reinforcement learning. [Online]. Available: <https://towardsdatascience.com/why-you-shouldnt-use-reinforcement-learning-163bae193da8>
- [39] C. Hwang, “Machine learning applications in hardware design: Sipi perspective,” PowerPoint presentation, 2024, eMC Laboratory, Missouri S&T, Rolla MO.
- [40] M. S. Withöft, Julian, “Ga optimization program,” internal Communication.
- [41] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang,

- “JAX: composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <http://github.com/google/jax>
- [42] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A llvm-based python jit compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6.
- [43] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. Seljebotn, and K. Smith, “Cython: The best of both worlds,” *Computing in Science Engineering*, vol. 13, no. 2, pp. 31–39, 2011.
- [44] S. Inc., “Streamlit,” <https://streamlit.io/>, 2019, python library for building interactive data apps.

Appendix A

Derivations

In this section, we shall look in detail at the derivations for the equations used in this thesis. [26]. As mentioned earlier, the following analysis can be limited to the area to the right of the y-axis (see Figure A.1) since the problem is symmetrical to the y-axis (mirror axis).

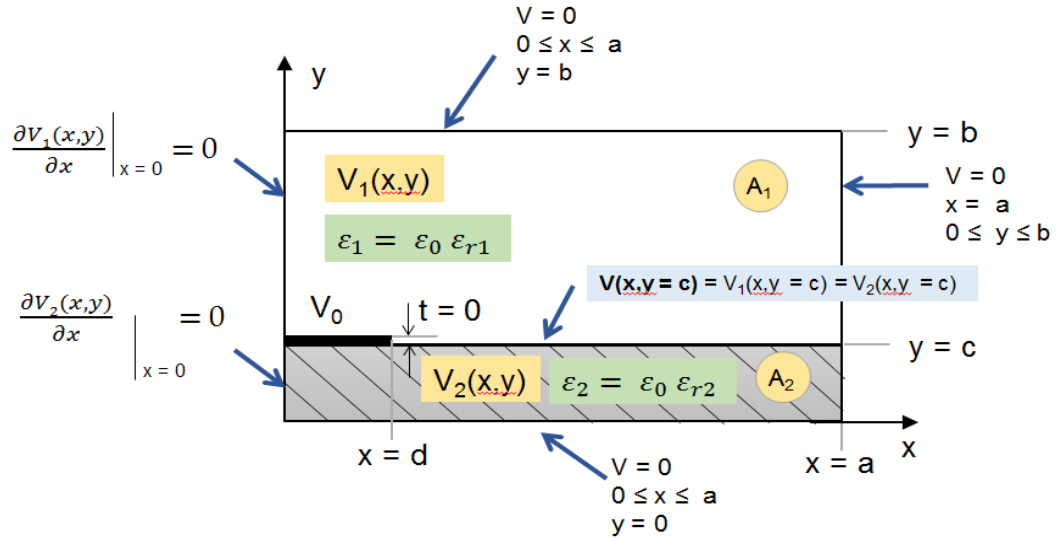


Figure A.1: Reduced Microstrip Arrangement under Study (use of symmetry characteristics of the arrangement)

To solve the two ordinary differential equations in x and y mentioned above, the following boundary conditions for the potential $V(x,y)$ must be fulfilled:

$$\begin{aligned}
 &\text{Area } A_1(0 \leq x \leq a);(c \leq y \leq b) \\
 &V_1(0 \leq x \leq a, y = b) = 0 \\
 &V_1(x = a, c \leq y \leq b) = 0 \\
 &\left. \frac{\partial V_1(x, y)}{\partial x} \right|_{x=0} = 0
 \end{aligned} \tag{A.1}$$

$$\begin{aligned}
 &\text{Area } A_2(0 \leq x \leq a);(0 \leq y \leq c) \\
 &V_2(0 \leq x \leq a, y = 0) = 0 \\
 &\left. \frac{\partial V_2(x, y)}{\partial x} \right|_{x=0} = 0
 \end{aligned} \tag{A.2}$$

At the surface layer $y = c$, the following applies:

$$V(x, y = c) = V_1(x, y = c) = V_2(x, y = c) \tag{A.3}$$

This results in the solution function for the required potentials of areas A_1 and A_2 :

$$V_1(x, y) = \sum_{n=0}^{\infty} v_n \cos \left[(2n+1) \frac{\pi}{2a} x \right] \left[\frac{\sinh \left[(2n+1) \frac{\pi}{2a} (b-y) \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]} \right] \tag{A.4}$$

$$V_2(x, y) = \sum_{n=0}^{\infty} v_n \cos \left[(2n+1) \frac{\pi}{2a} x \right] \left[\frac{\sinh \left[(2n+1) \frac{\pi}{2a} y \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} c \right]} \right] \tag{A.5}$$

A.1 Calculation of the Transmission Line Parameters Z_D and ϵ_{eff}

To calculate the required line parameters Z_D and ϵ_{eff} , the length-related capacity parameters (plane problem) C'_D ($A_1: \epsilon_{r1}; A_2: \epsilon_{r2}$) and C'_L ($A_1: \epsilon_{r1}; A_2: \epsilon_{r2}$) must be determined; two separate calculation steps are therefore necessary.

The following equations apply:

$$\begin{aligned}\epsilon_{eff} &= \frac{C'_D}{C'_L} \\ Z'_D &= \frac{1'}{c_0 \sqrt{C'_D C'_L}} \\ Z'_L &= \frac{1'}{c_0 C'_L} \\ v_{ph} &= \frac{c_0}{\sqrt{\epsilon_{eff}}} \text{ for } (\mu_r = 1)\end{aligned}\tag{A.6}$$

where, c_0 : speed of light in vacuum

The required capacitance values C'_D ($A_1: \epsilon_{r1}; A_2: \epsilon_{r2}$) and C'_L ($A_1: \epsilon_{r1}; A_2: \epsilon_{r2}$) can be determined by integrating the surface charge density $\sigma(x)$ over $(-d \leq x \leq d)$ or by calculating the electrical energy W' in the area $A = A_1 + A_2$. At this point, it is time to once again recall the requirements of the approach chosen for Q-TEM.

The following equations apply:

$$\begin{aligned}Q' &= 2 \int_0^d \sigma(x) dx \\ C'_D &= \frac{Q'_D}{V_0} \\ C'_L &= \frac{Q'_L}{V_0}\end{aligned}\tag{A.7}$$

and respectively;

$$\begin{aligned} C'_D &= 2 \frac{W'_D}{V_0^2} \\ C'_L &= 2 \frac{W'_L}{V_0^2} \end{aligned} \tag{A.8}$$

A.2 Equations for Calculating the Potential Distribution using First Order Spline Functions

As explained earlier, the problem is symmetrical with respect to the y-axis (see Figure A.1). The following considerations can be applied to the area to the right of the y-axis. For the $V(x, y = c)$ according to the above approach from the potential distributions $V_1(x, y)$ and $V_2(x, y)$ with $y = c$, the following results:

$$V(x, y = c) = \sum_{n=0}^{\infty} v_n \cos \left[(2n + 1) \frac{\pi}{2a} x \right] \tag{A.9}$$

We shall approximate the potential distribution with first-order spline functions as shown in Figure A.2.

The following approach is chosen:

$$V(x, y = c) = g(x)V_0 \tag{A.10}$$

Without the restriction of the generality, $V_0 = 1$ V will be used in the future. The following three approximates $g(x)$:

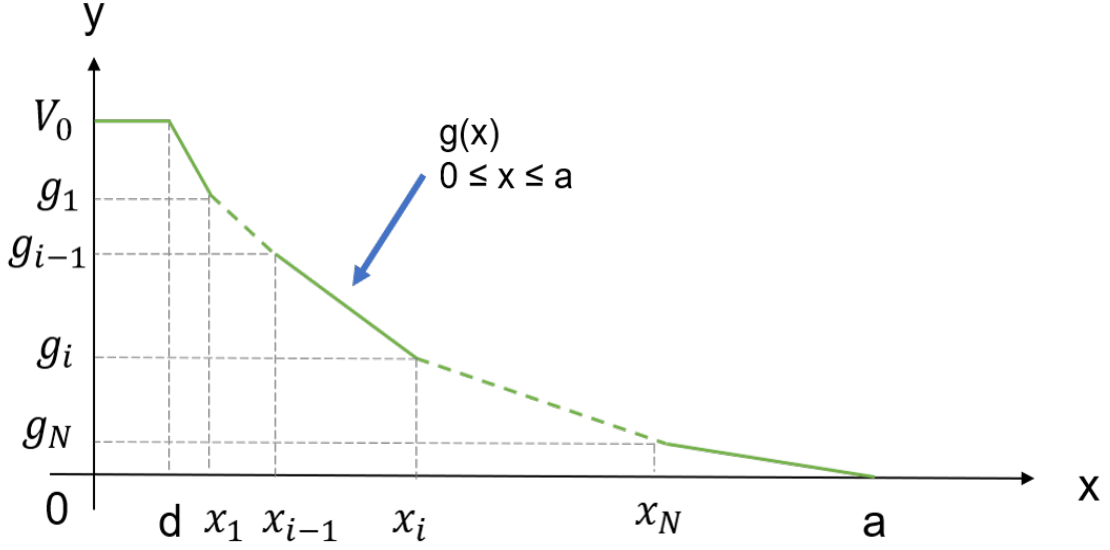


Figure A.2: Approximation of the Potential Distribution by Spline Functions of the First Order (symmetrical problem)

$$\begin{aligned}
 g_a(x) &= \frac{g_1 - 1}{x_1 - d}(x - d) + 1 \quad \text{where, } d \leq |x| \leq x_1 \\
 g_i(x) &= \frac{g_i - g_{i-1}}{x_i - x_{i-1}}(x - x_{i-1}) + g_{i-1} \quad \text{where, } x_i \leq |x| \leq x_N \\
 g_e(x) &= \frac{g_N}{x_N - a}(x - x_N) + g_N \quad \text{where, } x_N \leq |x| \leq a
 \end{aligned} \tag{A.11}$$

The following boundary conditions must be fulfilled by $g(x)$ for $0 \leq x \leq a$:

$$\begin{aligned}
 g_a(x)|_{x=d} &= 1 & g_i(x)|_{x=x_i} &= g_i \\
 g_a(x)|_{x=x_1} &= g_1 & g_i(x)|_{x=x_{i-1}} &= g_{i-1} \\
 g_e(x)|_{x=x_N} &= g_N & g_e(x)|_{x=a} &= 0
 \end{aligned} \tag{A.12}$$

For $V(x, y=c)$ the following results:

$$V(x, y = c) = \begin{cases} V_0, & \text{for } 0 \leq x \leq d \\ V_0 \left[\frac{g_1 - 1}{x_1 - d} \right] (x - d) + 1, & \text{for } d \leq x \leq x_i \\ V_0 \left[\frac{g_i - g_{i-1}}{x_i - x_{i-1}} \right] (x - x_{i-1}) + g_{i-1}, & \text{for } x_i \leq x \leq x_N \\ V_0 \left[\frac{g_N}{x_N - a} \right] (x - x_N) + g_N, & \text{for } x_N \leq x \leq a \end{cases} \quad (\text{A.13})$$

This can describe the required potential coefficients v_n approximately by using the orthogonal relationship of the trigonometric functions as a function of the selected first-order spline functions.

$$\begin{aligned} \int_0^a V_0 g(x) \cos \left[(2k+1) \frac{\pi}{2a} x \right] dx = \\ \sum_{n=0}^{\infty} \int_0^a v_n \cos \left[(2n+1) \frac{\pi}{2a} x \right] \cos \left[(2k+1) \frac{\pi}{2a} x \right] dx \end{aligned} \quad (\text{A.14})$$

The following equations apply:

$$\int_0^a \cos \left[(2n+1) \frac{\pi}{2a} x \right] \cos \left[(2k+1) \frac{\pi}{2a} x \right] dx = \begin{cases} 0, & \text{if } n \neq k \\ \frac{a}{2}, & \text{if } n = k \end{cases} \quad (\text{A.15})$$

$$\int_0^a \sin \left[(2n+1) \frac{\pi}{2a} x \right] \sin \left[(2k+1) \frac{\pi}{2a} x \right] dx = \begin{cases} 0, & \text{if } n \neq k \\ \frac{a}{2}, & \text{if } n = k \end{cases} \quad (\text{A.16})$$

$$\begin{aligned} V_0 \int_0^a g(x) \cos \left[(2n+1) \frac{\pi}{2a} x \right] dx &= v_n \frac{a}{2} \\ v_n &= \frac{2}{a} V_0 \int_0^a g(x) \cos \left[(2n+1) \frac{\pi}{2a} x \right] dx \end{aligned} \quad (\text{A.17})$$

$$\begin{aligned}
v_n = & \frac{2}{a} V_0 \left[\int_0^d \cos \left[(2n+1) \frac{\pi}{2a} x \right] dx \right. \\
& + \int_d^{x_1} \left(\left[\frac{g_1-1}{x_1-d} \right] (x-d) + 1 \right) \cos \left[(2n+1) \frac{\pi}{2a} x \right] dx \\
& + \int_{x_{i-1}}^{x_i} \left(\left[\frac{g_i-g_{i-1}}{x_i-x_{i-1}} \right] (x-x_{i-1}) + g_{i-1} \right) \cos \left[(2n+1) \frac{\pi}{2a} x \right] dx \\
& \left. + \int_{x_N}^a \left(\left[\frac{g_N}{x_N-a} \right] (x-x_N) + g_N \right) \cos \left[(2n+1) \frac{\pi}{2a} x \right] dx \right]
\end{aligned} \tag{A.18}$$

After solving the above integrals, the potential coefficients are obtained:

$$\begin{aligned}
v_n = & \frac{2}{a} V_0 \left(\frac{1}{[(2n+1) \frac{\pi}{2a}]^2} \right) \left\{ \frac{g_1-1}{x_1^G-d} \left[\cos \left((2n+1) \frac{\pi}{2a} x_1^G \right) - \cos \left((2n+1) \frac{\pi}{2a} d \right) \right] \right. \\
& + \sum_{i=2}^N \frac{g_i-g_{i-1}}{x_i^G-x_{i-1}^G} \left[\cos \left((2n+1) \frac{\pi}{2a} x_i^G \right) - \cos \left((2n+1) \frac{\pi}{2a} x_{i-1}^G \right) \right] \\
& \left. + \frac{g_N}{a-x_N^G} \cos \left((2n+1) \frac{\pi}{2a} x_N^G \right) \right\}
\end{aligned} \tag{A.19}$$

A.3 Calculation of Surface Charge Density using the Potential Coefficients

The required surface charge density $\sigma(x)$ is calculated from the normal derivative of the potentials $V_1(x, y)$ and $V_2(x, y)$ on the boundary surface $y = c$ of the two areas A_1 and A_2 :

$$\sigma(x) = -\epsilon_1 \frac{\partial V_1(x, y)}{\partial y} \Big|_{y=c} + \epsilon_2 \frac{\partial V_2(x, y)}{\partial y} \Big|_{y=c} \tag{A.20}$$

Since the arrangement is symmetrical to the y-axis, the surface charge density $\sigma(x)$ is also described using an even function in the form of a representation by a Fourier series:

$$\sigma(x, y = c) = \sum_{n=0}^{\infty} \sigma_n \cos \left[(2n+1) \frac{\pi}{2a} x \right] \quad (\text{A.21})$$

By using the equation A.20, we get σ_n as:

$$\sigma_n = v_n \left[(2n+1) \frac{\pi}{2a} \right] \left\{ \epsilon_2 \coth \left((2n+1) \frac{\pi}{2a} c \right) + \epsilon_1 \coth \left((2n+1) \frac{\pi}{2a} (b-c) \right) \right\} \quad (\text{A.22})$$

with:

$$\sigma_n = \psi_n v_n \quad (\text{A.23})$$

where,

$$\psi_n = \left[(2n+1) \frac{\pi}{2a} \right] \left\{ \epsilon_2 \coth \left((2n+1) \frac{\pi}{2a} c \right) + \epsilon_1 \coth \left((2n+1) \frac{\pi}{2a} (b-c) \right) \right\} \quad (\text{A.24})$$

$$\psi_n = \left[(2n+1) \frac{\pi}{2a} \right] \left\{ \epsilon_2 \frac{1 + e^{-(2n+1) \frac{\pi}{a} c}}{1 - e^{-(2n+1) \frac{\pi}{a} c}} + \epsilon_1 \frac{1 + e^{-(2n+1) \frac{\pi}{a} (b-c)}}{1 - e^{-(2n+1) \frac{\pi}{a} (b-c)}} \right\} \quad (\text{A.25})$$

The dielectric constants are given by:

$$\begin{aligned}\epsilon_2 &= \epsilon_0 \epsilon_{r2} \\ \epsilon_1 &= \epsilon_0 \epsilon_{r1}\end{aligned}\tag{A.26}$$

With this we can calculate the surface charge density with the known potential coefficients v_n .

A.4 Calculation of the Energy of the Electric Fields for the areas A_1 and A_2 using the Potential Coefficients

The field energy of the total area of the arrangement to be considered according to Figure A.1 .

$$W_2'^1 = 2 \left[\iint_{A_1} W_1' dA + \iint_{A_2} W_2' dA \right] \tag{A.27}$$

is calculated by integrating the energy densities W_1' and W_2' of areas A_1 and A_2 . As the energy of the electric field W_1' and W_2' is calculated for the selected symmetrical arrangement (see Figure A.1), factor 2 must be introduced into the above equation. The following applies here:

$$W_2'^1 = \frac{1}{2} \epsilon E^2 \tag{A.28}$$

where,

$$\vec{E} = -gradV(x, y) \tag{A.29}$$

expanding equation A.27

$$W_2'^1 = 2 \left\{ \frac{1}{2} \left[\epsilon_1 \iint_{A_1} (E_{x1}^2 + E_{y1}^2) \partial x \partial y + \epsilon_2 \iint_{A_2} (E_{x2}^2 + E_{y2}^2) \partial x \partial y \right] \right\} \quad (\text{A.30})$$

The electric field's energy calculation for area A_1 is carried out below. The same procedure is used without any restriction of its generality to calculate the energy of the electric field for area A_2 . The following applies:

$$\begin{aligned} \vec{E}_{x1} &= -\frac{\partial V_1(x, y)}{\partial x} = \\ -\sum_{n=0}^{\infty} v_n &\left\{ -(2n+1) \frac{\pi}{2a} \sin \left[(2n+1) \frac{\pi}{2a} x \right] \left[\frac{\sinh \left[(2n+1) \frac{\pi}{2a} (b-y) \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]} \right] \right\} \end{aligned} \quad (\text{A.31})$$

$$\begin{aligned} \vec{E}_{y1} &= -\frac{\partial V_1(x, y)}{\partial y} = \\ -\sum_{n=0}^{\infty} v_n &\cos \left[(2n+1) \frac{\pi}{2a} x \right] \left\{ -(2n+1) \frac{\pi}{2a} \left[\frac{\cosh \left[(2n+1) \frac{\pi}{2a} (b-y) \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]} \right] \right\} \end{aligned} \quad (\text{A.32})$$

$$\text{let } \bar{v}_n = v_n \frac{(2n+1) \frac{\pi}{2a}}{\sinh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]} \quad (\text{A.33})$$

$$\begin{aligned} E_{x1}^2 &= \sum_{n=0}^{\infty} \sum_{k=0}^{\infty} \bar{v}_n \bar{v}_k \left\{ \sin \left[(2n+1) \frac{\pi}{2a} x \right] \sin \left[(2k+1) \frac{\pi}{2a} x \right] \right. \\ &\quad \left. \sinh \left[(2n+1) \frac{\pi}{2a} (b-y) \right] \sinh \left[(2k+1) \frac{\pi}{2a} (b-y) \right] \right\} \end{aligned} \quad (\text{A.34})$$

$$\begin{aligned} E_{y1}^2 &= \sum_{n=0}^{\infty} \sum_{k=0}^{\infty} \bar{v}_n \bar{v}_k \left\{ \cos \left[(2n+1) \frac{\pi}{2a} x \right] \cos \left[(2k+1) \frac{\pi}{2a} x \right] \right. \\ &\quad \left. \cosh \left[(2n+1) \frac{\pi}{2a} (b-y) \right] \cosh \left[(2k+1) \frac{\pi}{2a} (b-y) \right] \right\} \end{aligned} \quad (\text{A.35})$$

This also allows the calculation of the energy W' of the electric field to be specified by using the orthogonal properties of the trigonometric functions as a function of the potential coefficients v_n .

$$W'_1 = \frac{\epsilon_1}{2} \left[2 \int_0^a \int_c^b (E_{x1}^2 + E_{y1}^2) \partial x \partial y \right] \quad (\text{A.36})$$

substituting equations A.34 and A.35 in equation A.36

$$\begin{aligned} W'_1 = \frac{\epsilon_1}{2} \left\{ 2 \int_0^a \int_c^b \left[\sum_{n=0}^{\infty} \sum_{k=0}^{\infty} \bar{v}_n \bar{v}_k \left(\sin \left[(2n+1) \frac{\pi}{2a} x \right] \sin \left[(2k+1) \frac{\pi}{2a} x \right] \right. \right. \right. \\ \cdot \sinh \left[(2n+1) \frac{\pi}{2a} (b-y) \right] \sinh \left[(2k+1) \frac{\pi}{2a} (b-y) \right] \Big) \\ + \sum_{n=0}^{\infty} \sum_{k=0}^{\infty} \bar{v}_n \bar{v}_k \left(\cos \left[(2n+1) \frac{\pi}{2a} x \right] \cos \left[(2k+1) \frac{\pi}{2a} x \right] \right. \\ \cdot \cosh \left[(2n+1) \frac{\pi}{2a} (b-y) \right] \cosh \left[(2k+1) \frac{\pi}{2a} (b-y) \right] \Big) \Big] \partial x \partial y \Big\} \end{aligned} \quad (\text{A.37})$$

$$\int_0^a \cos \left[(2n+1) \frac{\pi}{2a} x \right] \cos \left[(2k+1) \frac{\pi}{2a} x \right] \partial x = \begin{cases} 0, & \text{if } n \neq k \\ \frac{a}{2}, & \text{if } n = k \end{cases} \quad (\text{A.38})$$

$$\int_0^a \sin \left[(2n+1) \frac{\pi}{2a} x \right] \sin \left[(2k+1) \frac{\pi}{2a} x \right] \partial x = \begin{cases} 0, & \text{if } n \neq k \\ \frac{a}{2}, & \text{if } n = k \end{cases} \quad (\text{A.39})$$

considering the symmetrical $n = k$ condition,

$$W'_1 = \epsilon_1 \int_0^a \int_c^b \left[\sum_{n=0}^{\infty} v_n^2 \left(\sin^2 \left[(2n+1) \frac{\pi}{2a} x \right] \cdot \sinh^2 \left[(2n+1) \frac{\pi}{2a} (b-y) \right] \right) + v_n^2 \left(\cos^2 \left[(2n+1) \frac{\pi}{2a} x \right] \cdot \cosh^2 \left[(2n+1) \frac{\pi}{2a} (b-y) \right] \right) \right] \partial x \partial y \quad (\text{A.40})$$

$$W'_1 = \epsilon_1 \sum_{n=0}^{\infty} v_n^2 (2n+1) \frac{\pi}{4} \frac{\cosh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]} \quad (\text{A.41})$$

$$W_2'^1 = W_1' + W_2' \quad (\text{A.42})$$

$$W_2'^1 = \epsilon_1 \frac{\pi}{4} \sum_{n=0}^{\infty} v_n^2 (2n+1) \frac{\cosh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} (b-c) \right]} + \epsilon_2 \frac{\pi}{4} \sum_{n=0}^{\infty} v_n^2 (2n+1) \frac{\cosh \left[(2n+1) \frac{\pi}{2a} (c) \right]}{\sinh \left[(2n+1) \frac{\pi}{2a} (c) \right]} \quad (\text{A.43})$$

If the parameters for the first-order spline functions are selected appropriately, it is now possible to minimize the energy W' of the electric field in the area of the selected arrangement if the potential coefficients v_n are known. This leads to adjusting the shape (shape optimization) of the potential function $V(x, y=c)$ so that the energy W' of the electric field in the area under consideration is at a minimum.

However, it is also possible to select the potential distribution $V(x, y=c)$ in the region $d \leq x \leq a$ to minimise the surface charge density $\sigma(x)$ in this region.

The mathematical derivations for the equations are referenced from [26].

Appendix B

Software Documentation

B.1 Microstrip Arrangement

The general class defining microstrip arrangement parameters and corresponding functions

```
# importing libraries
import numpy as np
import mpmath as mp
from typing import *
import multiprocessing

# class to hold the microstrip arrangement functions
class MicroStripArrangement:
    """
    This class initializes a micro strip arrangement and encapsulates
    the functions to calculate its properties
    """
    # constructor function
    def __init__(self
        ,V0:float
        ,hw_arra:float
        ,ht_arra:float
        ,ht_subs:float
        ,hw_micrstr:float
```

```
,ht_micrstr:float
,er_1:float
,er_2:float
,num_fs:float) -> None:

# Set up the microstrip
self.V0 = V0 # potential of the microstrip
self.hw_arra = hw_arra # half width of the arrangement
self.ht_arra = ht_arra # height of the arrangement
self.ht_subs = ht_subs # height of the substrate
self.hw_micrstr = hw_micrstr # half width of the microstrip
self.ht_micrstr = ht_micrstr # height of the microstrip
self.er_1 = er_1 # relative permitivitty of medium 1(usually
    air)
self.er_2 = er_2 # relative permitivitty of medium 2
self.e0 = 8.85418781762e-12 # dielectric constant
self.c = 299792458 # speed of light in vacuum (m/s)
self.w_arra = 2*self.hw_arra # width of the arrangement
self.w_micrstr = 2*self.hw_micrstr # width of the microstrip
self.e1 = self.e0*self.er_1 # permitivitty of medium 1
self.e2 = self.e0*self.er_2 # permitivitty of medium 2
self.num_fs = num_fs # number of fourier coefficients(vn)

# function to check monotonicity
def monotonically_decreasing(self,g:np.ndarray) -> bool:
    """
    Function to check whether the suggested g values are
    monotonically decreasing
    Input
    -----
    g: support points generated by the agent
    numpy array

    Output
    -----
    boolean truth value
    """
    # take the numpy differentiation
```

```
dx = np.diff(g)

# check if all first difference are negative
# return the boolean value
return np.all(dx < 0)

def degree_of_monotonicity(self,g:np.ndarray) -> float:
    """
    Function to measure degree of monotonicity
    Input
    -----
    g: support points generated by the agent
        numpy array

    Output
    -----
    degree of monotonicity
    """
    # take the numpy differentiation
    dx = np.diff(g)

    # count number of negative values
    return np.sum(dx<0)

# convex or concave function
def is_convex(self,g:np.ndarray) -> bool:
    """
    Function to check whether the suggested g values is a convex
        function
    Input
    -----
    g: support points generated by the agent
        numpy array

    Output
    -----
    boolean truth value
    """
```

```
# Check if the array has at least 3 elements
if len(g) < 3:
    # print("The array should have at least 3 elements to
    perform convexity check.")
    return False

# Calculate the second differences
dx2 = g[2:] - 2 * g[1:-1] + g[:-2]

# Check if all second differences are non-negative
# return the boolean value
return np.all(dx2 >= 0)

# degree of convexity
def degree_of_convex(self,g:np.ndarray) -> float:
    """
    Function to measure degree of convexity
    Input
    -----
    g: support points generated by the agent
    numpy array

    Output
    -----
    degree of convexity
    """

    # Calculate the second differences
    dx2 = g[2:] - 2 * g[1:-1] + g[:-2]

    # count number of positive values
    return np.sum(dx2>0)

def potential_coeff(self,g:np.ndarray,x:np.ndarray) -> np.ndarray:
    :
    """
    Function to calculate the potential coefficients
```

*This function takes in g values for corresponding x values,
which are from $(hw_micrstr + delta$ to $hw_arra - delta)$*

Input

V0: potential of the microstrip
float

g: support points y-axis from agent
numpy array

x: support points x-axis
numpy array

hw_micrstr: half width of the microstrip
float

hw_arra: half width of the arrangement
float

num_fs: number of fourier coefficients
float

Output

vn = potential coefficients
np.ndarray
"""

vn = []

repeat for num_fs times

for n in range(self.num_fs):

vn_n = ((g[0] - 1)/(x[0] - self.hw_micrstr))(mp.cos((2*n
+1)*(mp.pi/(2*self.hw_arra))*x[0]) - mp.cos((2*n+1)*(mp
.pi/(2*self.hw_arra))*self.hw_micrstr))*

for i in range(1,len(x)):

vn_n += ((g[i] - g[i-1])/(x[i] - x[i-1]))(mp.cos((2*n
+1)*(mp.pi/(2*self.hw_arra))*x[i]) - mp.cos((2*n+1)
*(mp.pi/(2*self.hw_arra))*x[i-1]))*

*vn_n += (g[-1]/(self.hw_arra - x[-1]))*mp.cos((2*n+1)*(mp.
pi/(2*self.hw_arra))*x[-1])*

```
        vn_n *= 2/self.hw_arra*(self.V0/((2*n+1)*mp.pi/(2*self.
            hw_arra)**2))

        vn.append(float(vn_n))
    return np.array(vn)

def calculate_vn_n(self, args: Tuple[float, np.ndarray, np.ndarray])
-> float:
    """
    Function to calculate the potential coefficient
    This function takes in g values for corresponding x values,
    which are from (hw_micrstr + delta to hw_arra - delta)

    Input
    -----
    n: fourier coefficient
        float
    V0: potential of the microstrip
        float
    g: support points y-axis from agent
        numpy array
    x: support points x-axis
        numpy array
    hw_micrstr: half width of the microstrip
        float
    hw_arra: half width of the arrangement
        float

    Output
    -----
    vn_n = potential coefficient
        float
    """
    n, g, x = args
    vn_n = ((g[0] - 1)/(x[0] - self.hw_micrstr))*(mp.cos((2*n+1)
        *(mp.pi/(2*self.hw_arra))*x[0]) - mp.cos((2*n+1)*(mp.pi
        /(2*self.hw_arra))*self.hw_micrstr))
```

```
for i in range(1,len(x)):
    vn_n += ((g[i] - g[i-1])/(x[i] - x[i-1]))*(mp.cos((2*n+1)
        *(mp.pi/(2*self.hw_arra))*x[i]) - mp.cos((2*n+1)*(mp.pi
            /(2*self.hw_arra))*x[i-1]))

vn_n += (g[-1]/(self.hw_arra - x[-1]))*mp.cos((2*n+1)*(mp.pi
    /(2*self.hw_arra))*x[-1])
vn_n *= 2/self.hw_arra*(self.V0/((2*n+1)*mp.pi/(2*self.
    hw_arra))**2)

return float(vn_n)

# function to calculate potential coefficient parallely
def potential_coeff_parallel(self,g:np.ndarray,x:np.ndarray) ->
np.ndarray:
    """
    Function to calculate the potential coefficients parallely
    This function takes in g values for corresponding x values,
    which are from (hw_micrstr + delta to hw_arra - delta)

    Input
    -----
    V0: potential of the microstrip
        float
    g: support points y-axis from agent
        numpy array
    x: support points x-axis
        numpy array
    hw_micrstr: halw width of the microstrip
        float
    hw_arra: half width of the arrangement
        float
    num_fs: number of fourier coefficients
        float

    Output
    -----
    vn = potential coefficients
```

```
        np.ndarray
    """
    pool = multiprocessing.Pool()

    args_list = [(n, g, x) for n in range(self.num_fs)]
    vn = pool.map(self.calculate_vn_n, args_list)
    pool.close()
    pool.join()

    return np.array(vn)

# function to calculate the energy of the arrangement
def energy(self, vn: np.ndarray) -> float:
    """
    Function to calculate the total energy of the arrangement

    Input
    -----
    e1: permittivity of medium 1
        float
    e2: permittivity of medium 2
        float
    v: array of potential coefficients
        numpy array
    hw_arra: half width of the arrangement
        float
    ht_arra: height of the arrangement
        float
    ht_subs: height of the substrate
        float

    Output
    -----
    w12 = total energy of the arrangement
        float
    """
    # follow the formula
    w1 = 0
```



```
w2 = 0
for n in range(len(vn)):
    w1 += (vn[n]**2)*(2*n+1)*mp.pi*mp.coth((2*n+1)*mp.pi*(self
        .ht_arra - self.ht_subs)/(2*self.hw_arra))
    w2 += (vn[n]**2)*(2*n+1)*mp.pi*mp.coth((2*n+1)*mp.pi*(self
        .ht_subs)/(2*self.hw_arra))

w12 = float(self.e1*w1/4 + self.e2*w2/4)

return w12

def potential(self,vn:np.ndarray,x:np.ndarray) -> np.ndarray:
    """
    Function to calculate the potential distribution

    Input
    -----
    vn: fourier coefficients
        numpy array
    x: x-axis coordinates
        numpy array

    Output
    -----
    VF = potential distribution of the arrangement
        numpy array
    """

    for n in range(self.num_fs): # calculate sum of the fourier
        series for the potential
        if(n == 0):
            VF = vn[n] * np.cos((2*n+1)*(np.pi/(2*self.hw_arra))*x
                ) # first element of sum
        else:
            VF = VF + vn[n] * np.cos((2*n+1)*(np.pi/(2*self.
                hw_arra))*x) # add every other element of the sum

    return VF
```

```
def charge_density(self, vn: np.ndarray, x: np.ndarray) -> np.ndarray
:
    """
    Function to calculate the charge density

    Input
    -----
    vn: fourier coefficients
        numpy array
    x: x-axis coordinates
        numpy array

    Output
    -----
    sigma_F = charge density of the arrangement
        numpy array
    """
    sigma_n = []
    for n in range(self.num_fs):
        # Calculate approximation of sigma coefficient
        sigma_nn = vn[n]*(2*n+1)*(mp.pi/(2*self.hw_arra))*(self.e2
            *mp.coth((2*n+1)*(mp.pi/(2*self.hw_arra))*self.ht_subs)
            +self.e1*mp.coth((2*n+1)*(mp.pi/(2*self.hw_arra))*(self
                .ht_arra-self.ht_subs)))
        sigma_n.append(float(sigma_nn))

    # calculate Fourier approximation of charge density
    for n in range(self.num_fs):
        if(n == 0):
            sigma_F = sigma_n[n] * np.cos((2*n+1)*(np.pi/(2*self.
                hw_arra))*x) # first element of sum
        else:
            sigma_F = sigma_F + sigma_n[n] * np.cos((2*n+1)*(np.pi
                /(2*self.hw_arra))*x) # add every other element of
                the sum

    sigma_F = np.array([float(x) for x in sigma_F])
```

```
        return sigma_F

def calculate_sigma_n(self, args: Tuple[float, float]) -> float:

    n, vn_n = args
    sigma_nn = vn_n*(2*n+1)*(mp.pi/(2*self.hw_arra))\
        *(self.e2*mp.coth((2*n+1)*(mp.pi/(2*self.hw_arra))*self.
            ht_subs)\
            + self.e1*mp.coth((2*n+1)*(mp.pi/(2*self.hw_arra))*(\
                self.ht_arra-self.ht_subs))))

    return float(sigma_nn)

def charge_density_parallel(self, vn: np.ndarray, x: np.ndarray) ->
    np.ndarray:

    pool = multiprocessing.Pool()
    args_list = [(n, vn_n) for n, vn_n in enumerate(vn)]
    sigma_n = pool.map(self.calculate_sigma_n, args_list)
    pool.close()
    pool.join()
    sigma_n = np.array(sigma_n)

    # calculate fourier approximation of charge density
    for n in (range(self.num_fs)):
        if(n == 0):
            sigma_F = sigma_n[n] * np.cos((2*n+1)*(np.pi/(2*self.
                hw_arra))*x) # first element of sum
        else:
            sigma_F = sigma_F + sigma_n[n] * np.cos((2*n+1)*(np.pi
                /(2*self.hw_arra))*x) # add every other element of
                the sum

    sigma_F = np.array([float(x) for x in sigma_F])

    return sigma_F
```

```
def capacitance(self,w:float) -> float:
    """
    Function to calculate capacitance

    Input
    -----
    w: energy
        float

    Output
    -----
    C = capacitance
        float
    """

    C = (2*w)/(self.V0**2)

    return C
```

Listing B.1: Microstrip Arrangement

B.2 Method 3: Microstrip Arrangement RL Environment

Gymnasium environment for Method 3 RL training.

```
# import libraries
import numpy as np
from typing import *

# import the custom MicroStripArrangement library
from MicroStripArrangement import MicroStripArrangement as msa

# import gym
from gymnasium import Env
from gymnasium.spaces import Box
```

```
# Custom environment for scheduling process
class MicroStripArrangementEnv(Env,msa):
    """
    This class initializes a micro strip arrangement RL environment
    """

    """
    ### Methodology
    Cubic Bezier Curve, Threshold Reward
    """

    # initialization function
    def __init__(self,V0,hw_arra,ht_arra,ht_subs,hw_micrstr,
        ht_micrstr,er_1,er_2,num_fs,num_pts) -> None:

        # initialize the Micro Strip Arrangement class (measurements
        are in mm scale)
        msa.__init__(self,V0,hw_arra,ht_arra,ht_subs,hw_micrstr,
            ht_micrstr,er_1,er_2,num_fs)

        action_space_size = 4
        self.num_pts = num_pts

        # minimum energy in lifetime
        self.minimum_energy = np.inf
        self.action_minimum_energy = np.zeros(action_space_size)

        # x-axis values
        self.x = np.linspace(self.hw_micrstr ,self.hw_arra,num_pts)

        # line threshold y-axis values
        self.line_threshold = self.line_equ(self.x)

        # starting g points
        self.g_pts_start_x,self.g_pts_start_y,control_points = self.
            g_points_cubic_bezier(np.array([0,0,0,0]))
```

```
# current energy
self.current_energy = np.inf

# reward
self.reward = 0

# action space
#####
self.action_space = Box(low=0, high=1, shape=(
    action_space_size,), dtype=np.float32)

# observation space
#####
self.observation_space = Box(low=0, high=np.inf, shape=(5,),
    dtype=np.float32)

# function to return line equation parameters
def line_equ(self, x: np.ndarray) -> np.ndarray:
    # set end point, slope and intercept
    #####
    x_start = self.hw_micrstr
    x_intercept = self.hw_arra
    # the y-axis values are prior knowledge (x_start, 1) (
        x_intercept, 0)
    m = -1/(x_intercept-x_start)
    b = 0 - m*x_intercept

    # y = mx + b -> line equation
    #####
    y = m*x + b

    return y

def cubic_bezier_curve(self, t: np.ndarray, P0: np.ndarray, P1: np.
    ndarray, P2: np.ndarray, P3: np.ndarray) -> np.ndarray:
    # return the cubic bezier curve point for one (x,y) control
        points pair
```

```
        return (1 - t)**3 * P0 + 3 * (1 - t)**2 * t * P1 + 3 * (1 - t
            ) * t**2 * P2 + t**3 * P3

    # function to return g points
    def g_points_cubic_bezier(self, action: np.ndarray) -> Tuple[np.
        ndarray, np.ndarray, np.ndarray]:
        # control points
        #####
        P0 = np.array([self.hw_micrstr, 1])
        P10 = self.hw_micrstr + action[0]*(self.hw_arra-self.
            hw_micrstr)
        P1 = np.array([P10, action[1]])
        P20 = self.hw_micrstr + action[2]*(self.hw_arra-self.
            hw_micrstr)
        P2 = np.array([P20, action[3]])
        P3 = np.array([self.hw_arra, 0])

        # Generate exponential distribution
        #####
        # t_temp = np.exp(np.linspace(-5, 0, self.num_pts))
        # # Normalize to the range [0, 1]
        # t_values = (t_temp - np.min(t_temp)) / (np.max(t_temp) - np
            .min(t_temp))

        # Generate equidistant distribution
        #####
        t_values = np.linspace(0,1,self.num_pts)

        # bezier curve
        #####
        curve_points = np.array([self.cubic_bezier_curve(t, P0, P1,
            P2, P3) for t in t_values])
        # Calculate the x and y coordinates using the quadratic Bzier
            formula
        g_ptsx = curve_points[:, 0]
        g_ptsy = curve_points[:, 1]

        control_points = np.array([P0,P1,P2,P3])
```

```
        return g_ptsx,g_ptsy,control_points

    # environment reset function
    def reset(self, seed = None) -> Tuple[np.ndarray,dict]:

        # reset reward
        #####
        self.reward = 0

        # current energy
        #####
        self.current_energy = np.inf

        # observation space
        #####
        obs_space = self.hw_micrstr
        obs_space = np.append(obs_space,self.hw_arra)
        obs_space = np.append(obs_space,self.ht_arra)
        obs_space = np.append(obs_space,self.ht_subs)
        obs_space = np.append(obs_space,self.er_2)
        obs_space = obs_space.astype(np.float32)

        info = {}
        return obs_space, info

    def reward_calc(self,g_ptsx:np.ndarray,g_ptsy:np.ndarray,action:
np.ndarray) -> float:
        # initialize
        #####
        reward = 0
        line_threshold = self.line_equ(g_ptsx)

        # below threshold
        #####
        if np.all(g_ptsy <= line_threshold) and np.all(g_ptsy >= 0)
            and np.all(g_ptsx >= self.hw_micrstr):
                # is monotone decreasing and convex
                #####
```



```
if self.monotonically_decreasing(g_ptsy):
    # is convex
    #####
    # if self.is_convex(g_ptsy):
    # calculate potential coefficients
    #####
    vn = self.potential_coeff_parallel(g_ptsy[1:-1], g_ptsx
                                      [1:-1])

    # calculate energy
    #####
    self.current_energy = self.energy(vn)

    # update reward
    #####
    reward += (1/self.current_energy)*1e-5

    # new minimum energy achieved
    #####
    if self.current_energy < self.minimum_energy:
        # update minimum energy
        self.minimum_energy = self.current_energy

        # update reward
        reward += (1/self.current_energy)*1e-5

        # update action taken for minimum energy
        # self.action_minimum_energy = action
    # else:
    # convexity_degree = self.degree_of_convex(g_ptsy)
    # reward += convexity_degree/len(g_ptsy)

    # not monotone
    #####
else:
    reward += 0
```

```
# not below threshold
#####
else:
    # mean error from threshold
    reward += np.mean(self.line_threshold - g_pty)

# ensure the dtype of reward
#####
reward = float(reward)

return reward

# environment step function
def step(self, action) -> Tuple[np.ndarray, float, bool, bool, dict]:

    # set terminated as False
    #####
    terminated = False
    truncated = False

    # get the g points from RL agent
    #####
    g_ptsx, g_pty, control_points = self.g_points_cubic_bezier(
        action)

    # set rewards
    #####
    self.reward = self.reward_calc(g_ptsx, g_pty, action)

    # observation space
    #####
    obs_space = self.hw_micrstr
    obs_space = np.append(obs_space, self.hw_arra)
    obs_space = np.append(obs_space, self.ht_arra)
    obs_space = np.append(obs_space, self.ht_subs)
    obs_space = np.append(obs_space, self.er_2)
    obs_space = obs_space.astype(np.float32)
```

```
# set done to True
terminated = True

# pass auxilliary info
info = {}

# Return step information
return obs_space, self.reward, terminated, truncated, info

def render(self):
    pass
```

Listing B.2: Method 3 Microstrip Arrangement RL Environment

B.3 Method 4: Microstrip Arrangement RL Environment

Gymnasium environment for Method 4 RL training.

```
# import libraries
import numpy as np
from typing import *

# import the custom MicroStripArrangement library
from MicroStripArrangement import MicroStripArrangement as msa

# import gym
from gymnasium import Env
from gymnasium.spaces import Box

# Custom environment for scheduling process
class MicroStripArrangementEnv(Env, msa):
    """
    This class initializes a micro strip arrangement RL environment
    """
```

```
"""
    ### Methodology
    Cubic Bezier Curve, Percentage Reward
"""

# initialization function
def __init__(self, V0, hw_arra, ht_arra, ht_subs, hw_micrstr,
             ht_micrstr, er_1, er_2, num_fs, num_pts) -> None:

    # initialize the Micro Strip Arrangement class (measurements
    are in mm scale)
    msa.__init__(self, V0, hw_arra, ht_arra, ht_subs, hw_micrstr,
                 ht_micrstr, er_1, er_2, num_fs)

    action_space_size = 4
    self.num_pts = num_pts

    # minimum energy in lifetime
    self.minimum_energy = np.inf

    # starting g points
    self.g_pts_start_x, self.g_pts_start_y, control_points = self.
        g_points_cubic_bezier(np.array([0,0,0,0]))
    self.energy_init = self.energy(self.potential_coeff_parallel(
        self.g_pts_start_y[1:-1], self.g_pts_start_x[1:-1]))

    # current energy
    self.current_energy = np.inf

    # reward
    self.reward = 0

    # action Space
    #####
    self.action_space = Box(low=-0.2, high=0.2, shape=(
        action_space_size,), dtype=np.float32)
```

```
# observation space
#####
self.observation_space = Box(low=0, high=np.inf, shape=(5,),
                             dtype=np.float32)

def cubic_bezier_curve(self, t: np.ndarray, P0: np.ndarray, P1: np.
ndarray, P2: np.ndarray, P3: np.ndarray) -> np.ndarray:
    # return the cubic bezier curve point for one (x,y) control
    points pair
    return (1 - t)**3 * P0 + 3 * (1 - t)**2 * t * P1 + 3 * (1 - t
        ) * t**2 * P2 + t**3 * P3

# function to return g points
def g_points_cubic_bezier(self, action: np.ndarray) -> Tuple[np.
ndarray, np.ndarray, np.ndarray]:

    # control points
    #####
    P0 = np.array([self.hw_micrstr, 1])
    P10 = self.hw_micrstr + action[0]*(self.hw_arra-self.
        hw_micrstr)
    P1 = np.array([P10, action[1]])
    P20 = self.hw_micrstr + action[2]*(self.hw_arra-self.
        hw_micrstr)
    P2 = np.array([P20, action[3]])
    P3 = np.array([self.hw_arra, 0])

    # Generate equidistant distribution
    #####
    t_values = np.linspace(0,1,self.num_pts)

    # bezier curve
    #####
    curve_points = np.array([self.cubic_bezier_curve(t, P0, P1,
        P2, P3) for t in t_values])
```

```
# Calculate the x and y coordinates using the quadratic
Bezier formula
g_ptsx = curve_points[:, 0]
g_ptsy = curve_points[:, 1]

control_points = np.array([P0,P1,P2,P3])
return g_ptsx,g_ptsy,control_points

# environment reset function
def reset(self, seed = None) -> Tuple[np.ndarray,dict]:

    # reset reward
    #####
    self.reward = 0

    # current energy
    #####
    self.current_energy = np.inf

    # observation space
    #####
    obs_space = self.hw_micrstr
    obs_space = np.append(obs_space,self.hw_arra)
    obs_space = np.append(obs_space,self.ht_arra)
    obs_space = np.append(obs_space,self.ht_subs)
    obs_space = np.append(obs_space,self.er_2)
    obs_space = obs_space.astype(np.float32)

    info = {}
    return obs_space, info

def reward_calc(self,g_ptsx:np.ndarray,g_ptsy:np.ndarray,action:
np.ndarray) -> float:

    # initialize
    #####
    reward = 0
```

```
# to promote some change
#####
if np.all(action == 0):
    # conditon where no chnage happens
    return -100

if np.any(action == 0.2):
    # condition where it gets stuck at line_threshold_upper
    return -100

# is monotone decreasing and convex
#####
if self.monotonically_decreasing(g_ptsy):
    # is convex
    #####
    if self.is_convex(g_ptsy):

        # calculate potential coefficients
        #####
        vn = self.potential_coeff_parallel(g_ptsy[1:-1], g_ptsx
            [1:-1])

        # calculate energy
        #####
        self.current_energy = self.energy(vn)

        # update reward
        #####
        reward += 20 + ((1/self.current_energy)/(1/self.
            energy_init))*1e2 # 20 to keep it above the reward
            of necessary conditions
        # new minimum energy achieved
        #####
        if self.current_energy <= self.minimum_energy:
            # update minimum energy
            self.minimum_energy = self.current_energy

            # update minimum energy action
```

```
        self.action_minimum_energy = action

        # # update reward
        # reward += (1/self.current_energy)*1e-5

    else:
        reward += self.degree_of_monotonicity(g_ptsy) + self.
            degree_of_convex(g_ptsy)

    # not monotone
    #####
    else:
        reward += self.degree_of_monotonicity(g_ptsy)

    # ensure the dtype of reward
    #####
    reward = float(reward)

    return reward

# environment step function
def step(self, action) -> Tuple[np.ndarray,float,bool,bool,dict]:

    # set terminated as False
    #####
    terminated = False
    truncated = False

    action = np.abs(action)
    # get the g points from RL agent
    #####
    g_ptsx,g_ptsy,control_points = self.g_points_cubic_bezier(
        action)

    # set rewards
    #####
    self.reward = self.reward_calc(g_ptsx,g_ptsy,action)
```



```
# observation space
#####
obs_space = self.hw_micrstr
obs_space = np.append(obs_space, self.hw_arra)
obs_space = np.append(obs_space, self.ht_arra)
obs_space = np.append(obs_space, self.ht_subs)
obs_space = np.append(obs_space, self.er_2)
obs_space = obs_space.astype(np.float32)

# set done to True
terminated = True

# pass auxilliary info
info = {}

# Return step information
return obs_space, self.reward, terminated, truncated, info

def render(self):
    pass
```

Listing B.3: Method 4 Microstrip Arrangement RL Environment